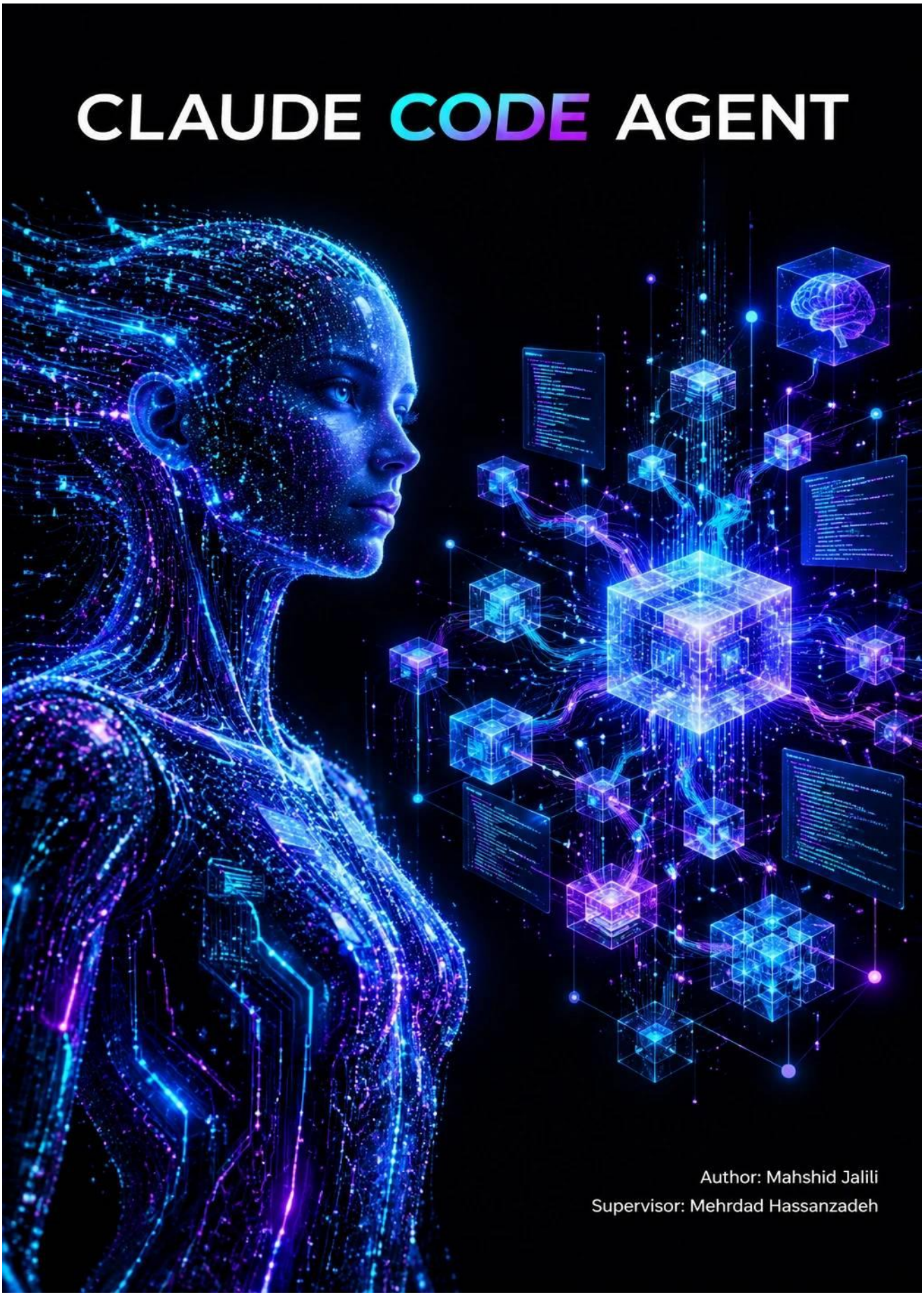
CLAUDE CODE AGENT



Author: Mahshid Jalili
Supervisor: Mehrdad Hassanzadeh

فهرست مطالب

فصل اول: آشنایی با هوش مصنوعی عامل‌محور

- مقدمه
- ۱.۱. هوش مصنوعی عامل‌محور چیست؟
- ۱.۲. تفاوت ایجنت با چت‌بات ساده

فصل دوم: آشنایی با Claude و ... Claude Code

- مقدمه
- معرفی Claude
- معرفی Claude Code
- کاربردهای Claude Code در توسعهٔ نرم‌افزار

فصل سوم: راه‌اندازی نخستین پروژه با ... Claude Code

- ۳.۱. بررسی پیش‌نیازها
- نصب و راه‌اندازی Claude Code
- ایجاد و آماده‌سازی نخستین پروژه
- اجرای نخستین درخواست
- چک‌لیست پایان فصل

فصل چهارم: چرخهٔ کاری پیشنهادی با ... Claude Code

- شناخت وضعیت پروژه
- برنامه‌ریزی پیش از اجرا
- اجرای مرحله‌ای تغییرات
- اجرای تست و بررسی نتیجه
- بازبینی تغییرات
- تصمیم‌گیری انسانی دربارهٔ Commit

فصل پنجم: راهنمای دستورهای Slash و CLI در ... Claude Code

- تفاوت دستورهای Slash و CLI
- دستورهای خط فرمان Claude Code

bitwinit

- دستورهای داخلی و تعاملی
- اجرای فرمان‌های پروژه
- مدیریت نشست‌ها
- چک‌لیست پایان فصل

فصل ششم: کنترل دسترسی و امنیت در ... Claude Code

- مفهوم مجوزها در Claude Code
- کنترل دسترسی به فایل‌ها و ابزارها
- تأیید اجرای فرمان‌ها
- تنظیمات امنیتی
- مدیریت ریسک در اجرای خودکار
- چک‌لیست پایان فصل

فصل هفتم: Git و سه بررسی ضروری پیش از ... Commit

- آشنایی Claude Code با Git
- بررسی وضعیت مخزن
- مشاهده و تحلیل تغییرات
- اجرای تست‌ها پیش از Commit
- بررسی کیفیت و صحت تغییرات
- سه بررسی ضروری پیش از Commit

فصل هشتم: طراحی و مدیریت فایل CLAUDE.md ...

- هدف و کاربرد فایل CLAUDE.md
- ایجاد فایل CLAUDE.md
- ساختار و محتوای پیشنهادی
- دستورالعمل‌های پروژه
- قوانین کدنویسی و اجرای تست
- نگهداری و به‌روزرسانی فایل

فصل نهم: قابلیت‌های پیشرفته در ... Claude Code

bitwithit

- قابلیت‌های تخصصی Claude Code
- عامل‌های فرعی
- تقسیم وظایف میان عامل‌ها
- اتصال ابزارها و منابع خارجی
- ۹.۵. اتصال ابزارها و منابع خارجی با MCP

فصل دهم: انتخاب مدل و سطح استدلال در ... Claude Code

- انتخاب مدل مناسب
- تفاوت مدل‌ها
- تنظیم سطح استدلال
- ایجاد تعادل میان سرعت، هزینه و دقت
- انتخاب مدل بر اساس نوع وظیفه

bitwitit

فصل اول: آشنایی با هوش مصنوعی عامل‌محور

مقدمه

هوش مصنوعی در ابتدا بیشتر برای پاسخ‌دادن به پرسش‌ها، تولید متن و انجام گفت‌وگو استفاده می‌شد. کاربر سؤال یا دستور خود را وارد می‌کرد و مدل، بر اساس اطلاعات موجود در همان گفت‌وگو، پاسخی تولید می‌کرد.

اما **هوش مصنوعی عامل‌محور** یا **Agentic AI** یک گام فراتر می‌رود. در این رویکرد، هوش مصنوعی فقط پاسخ تولید نمی‌کند؛ بلکه می‌تواند برای رسیدن به یک هدف، مسئله را بررسی کند، آن را به چند مرحله تقسیم کند، ابزار مناسب را انتخاب کند، نتیجهٔ عملیات را مشاهده کند و بر اساس نتیجه، برنامهٔ خود را اصلاح کند.

به بیان ساده:

چت‌بات معمولاً به یک درخواست پاسخ می‌دهد، اما ایجنت برای رسیدن به یک هدف، مجموعه‌ای از اقدامات را برنامه‌ریزی و اجرا می‌کند.

۱.۱. هوش مصنوعی عامل‌محور چیست؟

هوش مصنوعی عامل‌محور سیستمی است که برای رسیدن به یک هدف مشخص، چند قابلیت را در کنار هم قرار می‌دهد:

- درک درخواست کاربر؛
- تحلیل مسئله؛
- برنامه‌ریزی مراحل انجام کار؛
- انتخاب ابزار مناسب؛
- اجرای عملیات؛
- مشاهده و ارزیابی نتیجه؛
- اصلاح برنامه در صورت نیاز؛
- ارائهٔ نتیجهٔ نهایی به کاربر.

در یک گفت‌وگوی ساده، مدل ممکن است فقط متنی مانند نمونهٔ زیر تولید کند:

برای رفع این خطا، فایل تنظیمات را بررسی کنید.

اما یک ایجنت می‌تواند، در صورت داشتن مجوز و ابزار مناسب:

1. فایل‌های مرتبط را پیدا کند؛
2. محتوای آن‌ها را بررسی کند؛
3. علت احتمالی خطا را تشخیص دهد؛
4. تغییر پیشنهادی ایجاد کند؛

bitwit

5. تست یا دستور مربوط را اجرا کند؛
6. خروجی را بررسی کند؛
7. در صورت باقی‌ماندن خطا، مرحلهٔ بعدی را پیشنهاد دهد یا اجرا کند.

بنابراین تفاوت اصلی، در **توانایی حرکت از پاسخ متنی به اجرای یک فرایند چندمرحله‌ای** است.

۱.۲. تفاوت ایجنت با چت‌بات ساده

چت‌بات ساده

چت‌بات معمولاً در یک الگوی مستقیم کار می‌کند:

درخواست کاربر ← تولید پاسخ

در این حالت، مدل ممکن است توضیح، کد، پیشنهاد یا متن تولید کند؛ اما لزوماً نمی‌تواند نتیجهٔ اجرای آن را ببیند یا بر اساس آن تصمیم جدیدی بگیرد.

ایجنت

ایجنت معمولاً در چرخه‌ای شبیه این فعالیت می‌کند:

هدف ← برنامه‌ریزی ← انتخاب ابزار ← اجرا ← مشاهده ← اصلاح

برای مثال، اگر کاربر بگوید:

خطای ورود کاربران را در پروژه پیدا و اصلاح کن.

چت‌بات ممکن است چند علت احتمالی و یک قطعه‌کد پیشنهادی ارائه کند. اما ایجنت، در صورت دسترسی مناسب، می‌تواند:

- ساختار پروژه را بررسی کند؛
- فایل‌های مربوط به ورود کاربران را پیدا کند؛
- مسیر اجرای کد را دنبال کند؛
- تست‌های موجود را اجرا کند؛
- تغییر پیشنهادی ایجاد کند؛
- نتیجهٔ تست را مشاهده کند؛
- در صورت شکست تست، راهکار خود را اصلاح کند.

البته این موضوع به ابزارها، مجوزها، محیط اجرا و قابلیت‌های فعال بستگی دارد. داشتن عنوان «ایجنت» به‌تنهایی به معنی دسترسی نامحدود یا عملکرد کاملاً خودکار نیست.

۱.۳. چرخهٔ عامل‌محور

چرخهٔ عامل‌محور، الگوی اصلی عملکرد یک ایجنت است. این چرخه از چند مرحله تشکیل می‌شود.

مرحلهٔ اول: تعیین هدف

ابتدا باید مشخص شود که ایجنت قرار است به چه نتیجه‌ای برسد.

هدف می‌تواند ساده باشد:

یک تابع برای محاسبهٔ میانگین اضافه کن.

یا پیچیده‌تر باشد:

مشکل کندی بارگذاری صفحهٔ داشبورد را بررسی کن، علت را پیدا کن و یک اصلاح محدود ارائه بده.

هرچه هدف دقیق‌تر باشد، احتمال برداشت نادرست کمتر می‌شود. یک هدف خوب معمولاً شامل این موارد است:

- کاری که باید انجام شود؛
- بخش یا فایل موردنظر؛
- محدودیت‌های کار؛
- چیزهایی که نباید تغییر کنند؛
- معیار تشخیص موفقیت.

نمونهٔ دقیق‌تر:

سرعت بارگذاری داشبورد را بررسی کن. فقط فایل‌های مربوط به داشبورد را تغییر بده،

ساختار API را عوض نکن و پس از تغییر، تست‌های مربوط به داشبورد را اجرا کن.

مرحلهٔ دوم: برنامه‌ریزی

پس از دریافت هدف، ایجنت باید مشخص کند که برای رسیدن به نتیجه چه مراحلی لازم است.

برای نمونه:

1. ساختار پروژه را بررسی کن.
2. فایل‌های مربوط به داشبورد را پیدا کن.
3. مسیر دریافت داده را بررسی کن.
4. علت احتمالی کندی را مشخص کن.
5. راه‌حل محدود پیشنهاد بده.
6. تغییر را اعمال کن.
7. تست‌های مرتبط را اجرا کن.

8. نتیجه را گزارش بده.

برنامه‌ریزی کمک می‌کند کار بدون بررسی اولیه و با تغییرات پراکنده انجام نشود.

در Claude Code، استفاده از دستور /plan برای همین مرحله مفید است. این دستور به مدل کمک می‌کند پیش از اعمال تغییر، مسئله و مراحل پیشنهادی را توضیح دهد.

مرحلهٔ سوم: انتخاب ابزار

ایجنت برای اجرای برنامه باید ابزار مناسب را انتخاب کند. این ابزار می‌تواند شامل موارد زیر باشد:

- ابزار خواندن فایل؛
- ابزار جست‌وجو در پروژه؛
- ابزار ویرایش فایل؛
- ابزار اجرای دستور؛
- ابزار اجرای تست؛
- ابزار بررسی وضعیت Git؛
- ابزارهای خارجی از طریق MCP.

انتخاب ابزار باید با هدف و سطح دسترسی موردنیاز متناسب باشد. برای مثال، اگر کار فقط به بررسی یک فایل نیاز دارد، دسترسی گسترده به کل سیستم ضرورتی ندارد.

مرحلهٔ چهارم: اجرای عملیات

در این مرحله، ایجنت اقدام برنامه‌ریزی‌شده را انجام می‌دهد. برای مثال:

- فایل را می‌خواند؛
- نتیجهٔ جست‌وجو را بررسی می‌کند؛
- تغییر پیشنهادی را اعمال می‌کند؛
- تست را اجرا می‌کند؛
- خروجی یک دستور را دریافت می‌کند.

اجرای عملیات ممکن است کاملاً خودکار نباشد. در برخی موارد، ابزار از کاربر تأیید می‌خواهد؛ مخصوصاً برای عملیات حساس مانند:

- حذف فایل؛
- تغییر گسترده در پروژه؛
- اجرای دستور ناشناخته؛
- دسترسی به منابع خارج از پروژه؛

- تغییر تنظیمات مهم.

مرحلهٔ پنجم: مشاهدهٔ نتیجه

تفاوت مهم ایجنت با یک تولیدکنندهٔ متن ساده این است که ایجنت می‌تواند نتیجهٔ اقدام خود را مشاهده کند.

برای نمونه، پس از اجرای تست ممکن است چنین خروجی‌ای دریافت کند:

2 tests passed

1 test failed

ایجنت باید این نتیجه را بررسی کند و بفهمد:

- آیا هدف اصلی محقق شده است؟
- آیا خطای جدیدی ایجاد شده است؟
- آیا هنوز مشکلی باقی مانده است؟
- آیا باید برنامه تغییر کند؟

بدون مرحلهٔ مشاهده، ایجنت نمی‌داند عملیات واقعاً موفق بوده یا فقط ظاهراً اجرا شده است.

مرحلهٔ ششم: اصلاح برنامه

اگر نتیجه مطابق انتظار نباشد، ایجنت باید برنامهٔ خود را اصلاح کند.

برای مثال:

تغییر اعمال شد، اما یکی از تست‌ها به دلیل ناسازگاری نوع داده شکست خورد.

ابتدا نوع دادهٔ ورودی را بررسی می‌کنم و سپس اصلاح محدودتری پیشنهاد می‌دهم.

در این مرحله، ایجنت ممکن است:

- علت خطا را دوباره بررسی کند؛
- ابزار دیگری انتخاب کند؛
- دامنهٔ تغییر را محدودتر کند؛
- راه‌حل قبلی را بازگرداند یا اصلاح کند؛
- از کاربر تأیید بیشتری بخواهد.

چرخه تا زمانی ادامه پیدا می‌کند که یکی از این شرایط برقرار شود:

- هدف با موفقیت انجام شده باشد؛
- ادامهٔ کار به تأیید کاربر نیاز داشته باشد؛
- ابزار یا مجوز کافی وجود نداشته باشد؛

- ایجنت به محدودیت فنی برسد؛
- کاربر اجرای فرایند را متوقف کند.

۱.۴. اجزای اصلی یک ایجنت

یک ایجنت عامل‌محور معمولاً از چند جزء اصلی تشکیل می‌شود.

۱.۴.۱. مدل زبانی

مدل زبانی بخش اصلی تحلیل و تولید پاسخ است. این مدل می‌تواند:

- متن کاربر را تفسیر کند؛
- بین اطلاعات ارتباط برقرار کند؛
- کد و فایل را تحلیل کند؛
- برنامهٔ پیشنهادی تولید کند؛
- نتیجهٔ ابزارها را توضیح دهد؛
- گام بعدی را انتخاب کند.

با این حال، مدل زبانی تضمین نمی‌کند که هر پاسخ صحیح باشد. مدل ممکن است در درک هدف، تحلیل فایل یا نتیجه‌گیری دچار خطا شود.

۱.۴.۲. Context

Context مجموعهٔ اطلاعاتی است که مدل در هنگام انجام کار در اختیار دارد. این اطلاعات ممکن است شامل موارد زیر باشد:

- پیام‌های گفت‌وگو؛
- فایل‌های پروژه؛
- دستورالعمل‌های پروژه؛
- خروجی ابزارها؛
- خطاها و نتایج تست‌ها؛
- تصمیم‌های قبلی کاربر.

اگر اطلاعات مهم در Context وجود نداشته باشد، ایجنت ممکن است تصمیم نادرستی بگیرد. برای مثال، اگر مدل نداند که یک فایل نباید تغییر کند، ممکن است آن را در برنامهٔ خود وارد کند.

۱.۴.۳. حافظه

حافظه به اطلاعاتی گفته می‌شود که برای حفظ پیوستگی کار در طول زمان استفاده می‌شوند.

در پروژه‌های نرم‌افزاری، فایل‌هایی مانند CLAUDE.md می‌توانند بخشی از قوانین و اطلاعات پایدار پروژه را در اختیار Claude Code قرار دهند. این فایل می‌تواند شامل مواردی مانند:

- روش اجرای پروژه؛
- دستورهای تست؛
- قواعد سبک کدنویسی؛
- ساختار پوشه‌ها؛
- محدودیت‌های مهم؛
- دستورالعمل‌های مربوط به Git.

حافظه با Context یکسان نیست. Context بیشتر به اطلاعات جاری و موردنیاز همان جلسه مربوط می‌شود، در حالی که حافظه می‌تواند اطلاعاتی پایدارتر برای جلسات یا وظایف مختلف نگهداری کند.

۱.۴.۴. ابزارها

ابزارها امکان تعامل ایجنت با محیط را فراهم می‌کنند. بدون ابزار، مدل بیشتر در سطح تحلیل و تولید متن باقی می‌ماند.

در Claude Code، ابزارها می‌توانند برای این کارها استفاده شوند:

- خواندن فایل‌ها؛
- جست‌وجو در کد؛
- ایجاد یا ویرایش فایل؛
- اجرای دستورهای ترمینال؛
- اجرای تست؛
- بررسی تغییرات؛
- کار با Git؛
- ارتباط با سرویس‌ها یا ابزارهای خارجی از طریق MCP.

هر ابزار باید با سطح دسترسی مشخص و هدف محدود استفاده شود.

۱.۴.۵. سیستم مجوز و امنیت

سیستم مجوز مشخص می‌کند ایجنت اجازهٔ انجام چه کارهایی را دارد.

برای مثال، ممکن است ایجنت اجازه داشته باشد:

- فایل‌های یک پروژه را بخواند؛
- تست‌های مشخصی را اجرا کند؛

- فایل‌های مشخصی را ویرایش کند.

اما اجازه نداشته باشد:

- فایل‌های خارج از پروژه را بخواند؛
- دستورهای خطرناک اجرا کند؛
- داده‌ای را حذف کند؛
- به سرویس خارجی متصل شود.

مجوزها باید متناسب با کار انتخاب شوند. دادن دسترسی بیشتر همیشه به معنی نتیجهٔ بهتر نیست و می‌تواند خطر تغییر ناخواسته یا افشای اطلاعات را افزایش دهد.

۱.۵. نقش ابزارها در عملکرد ایجنت

یک ایجنت بدون ابزار مناسب، توانایی محدودی دارد. تفاوت میان «پیشنهاد یک راه‌حل» و «اجرای قابل‌بررسی آن راه‌حل» معمولاً به ابزارهای در دسترس مربوط می‌شود.

برای مثال، فرض کنید کاربر می‌پرسد:

چرا تست‌های پروژه شکست می‌خورند؟

اگر ایجنت فقط به متن گفت‌وگو دسترسی داشته باشد، می‌تواند علت‌های احتمالی را بیان کند. اما اگر به پروژه و ابزارهای اجرایی دسترسی داشته باشد، می‌تواند:

1. فایل‌های تست را پیدا کند؛
2. دستور تست را از پروژه استخراج کند؛
3. تست‌ها را اجرا کند؛
4. خروجی خطا را بخواند؛
5. مسیر شکست را بررسی کند؛
6. راه‌حل پیشنهادی ارائه دهد.

بنابراین کیفیت عملکرد ایجنت به دو عامل وابسته است:

- توانایی تحلیل مدل؛
- کیفیت و محدودهٔ ابزارهای در دسترس.

حتی مدل قوی نیز بدون اطلاعات و ابزار کافی نمی‌تواند نتیجهٔ قابل‌اعتمادی تولید کند.

۱.۶. نمونهٔ کامل چرخهٔ عامل‌محور

فرض کنید کاربر از Claude Code می‌خواهد:

یک قابلیت ثبت گزارش خطا به برنامه اضافه کن، اما ساختار فعلی API را تغییر نده.

مرحلهٔ اول: تعیین هدف

هدف این است که قابلیت ثبت گزارش خطا اضافه شود، بدون تغییر در ساختار فعلی API.

مرحلهٔ دوم: برنامه‌ریزی

Claude Code می‌تواند برنامهٔ زیر را پیشنهاد دهد:

1. بررسی ساختار پروژه؛
2. پیدا کردن محل مدیریت خطاها؛
3. بررسی روش فعلی ثبت رویدادها؛
4. پیشنهاد محل مناسب برای ثبت گزارش خطا؛
5. اعمال تغییر محدود؛
6. افزودن یا اصلاح تست‌ها؛
7. اجرای تست‌های مرتبط؛
8. بررسی تفاوت‌های ایجادشده.

مرحلهٔ سوم: انتخاب ابزار

برای اجرای این برنامه، ابزارهای زیر لازم هستند:

- جست‌وجو در فایل‌ها؛
- خواندن فایل‌های مرتبط؛
- ویرایش فایل؛
- اجرای تست؛
- بررسی diff.

مرحلهٔ چهارم: مشاهدهٔ پروژه

ایجنت ساختار پروژه را بررسی می‌کند و ممکن است متوجه شود که مدیریت خطا در چند فایل جدا انجام می‌شود.

مرحلهٔ پنجم: اصلاح برنامه

بر اساس ساختار واقعی پروژه، برنامه دقیق‌تر می‌شود:

مدیریت خطا در دو بخش جدا انجام می‌شود. ابتدا مسیر درخواست‌هایHTTP

و سپس لایهٔ سرویس را بررسی می‌کنم تا ثبت گزارش در محل مناسب انجام شود.

bitwiti

مرحلهٔ ششم: اعمال تغییر

تغییر در فایل‌های محدود اعمال می‌شود و ساختار API دست‌نخورده باقی می‌ماند.

مرحلهٔ هفتم: اجرای تست

تست‌ها اجرا می‌شوند. اگر همهٔ تست‌ها موفق باشند، نتیجه بررسی می‌شود. اگر تستی شکست بخورد، علت شکست باید تحلیل شود.

مرحلهٔ هشتم: بررسی نهایی

در پایان باید مشخص شود:

- کدام فایل‌ها تغییر کرده‌اند؛
- چه رفتاری اضافه شده است؛
- کدام تست‌ها اجرا شده‌اند؛
- آیا ساختار API تغییر نکرده است؛
- آیا کاربر باید تصمیم یا بررسی دیگری انجام دهد.

۱.۷. توانایی‌های ایجنت

یک ایجنت می‌تواند در بسیاری از کارهای فکری و فنی کمک کند، از جمله:

- تبدیل هدف کلی به مراحل مشخص؛
- بررسی اطلاعات موجود؛
- جست‌وجو در فایل‌ها و پروژه؛
- استفاده از ابزارهای مختلف؛
- اجرای عملیات چندمرحله‌ای؛
- تحلیل خروجی ابزارها؛
- تشخیص خطای احتمالی؛
- اصلاح برنامه پس از مشاهدهٔ نتیجه؛
- تهیهٔ گزارش از اقدامات انجام‌شده.

در Claude Code، این توانایی‌ها بیشتر در زمینهٔ توسعهٔ نرم‌افزار کاربرد دارند؛ برای مثال:

- بررسی کد؛
- پیشنهاد تغییر؛

- اجرای تست؛
- بررسی وضعیت پروژه؛
- کمک به رفع خطا؛
- مدیریت بخشی از گردش‌کار توسعه.

۱.۸. محدودیت‌های ایجنت

ایجنت‌ها با وجود توانایی اجرای چندمرحله‌ای، محدودیت‌های مهمی دارند.

۱.۸.۱. احتمال برداشت نادرست

ممکن است مدل درخواست کاربر را به‌درستی متوجه نشود. مثلاً عبارت «بهینه‌سازی عملکرد» می‌تواند به موارد مختلفی اشاره کند:

- کاهش زمان اجرا؛
- کاهش مصرف حافظه؛
- کم‌کردن تعداد درخواست‌ها؛
- بهبود سرعت رابط کاربری.

اگر هدف مشخص نباشد، ایجنت ممکن است مسیر نادرستی را انتخاب کند.

۱.۸.۲. اطلاعات ناقص

اگر فایل، خطا، دستورالعمل یا زمینهٔ کافی در اختیار ایجنت نباشد، تحلیل آن ناقص خواهد بود.

۱.۸.۳. خطای فنی

ایجنت ممکن است:

- کد نادرست تولید کند؛
- بخشی از وابستگی‌ها را نادیده بگیرد؛
- تست نامناسب اجرا کند؛
- خروجی خطا را اشتباه تفسیر کند؛
- تصور کند تغییر موفق بوده، در حالی که مشکل باقی است.

۱.۸.۴. وابستگی به محیط

رفتار Claude Code به عوامل مختلفی بستگی دارد:

- نسخهٔ ابزار؛

- مدل انتخاب‌شده؛
- ابزارهای فعال؛
- مجوزهای موجود؛
- ساختار پروژه؛
- سیستم‌عامل؛
- تنظیمات محیط اجرا.

بنابراین دستور یا قابلیتی که در یک محیط کار می‌کند، ممکن است در محیط دیگر در دسترس نباشد یا رفتار متفاوتی داشته باشد.

۱.۸.۵. خطر اجرای عملیات حساس

اگر مجوزها بیش از حد گسترده باشند، ایجنت ممکن است بتواند عملیاتی انجام دهد که پیامدهای مهم دارند. به همین دلیل، اجرای خودکار نباید با اعتماد کامل و بدون کنترل انجام شود.

۱.۹. مسئولیت کاربر در استفاده از ایجنت

ایجنت ابزار تصمیم‌یار و اجراکننده است، اما مسئولیت نهایی استفاده از نتیجه با کاربر باقی می‌ماند.

کاربر باید:

- هدف را روشن بیان کند؛
- محدودیت‌ها را مشخص کند؛
- برنامهٔ پیشنهادی را بررسی کند؛
- مجوزها را کنترل کند؛
- تغییرات را مرور کند؛
- تست‌ها را اجرا یا نتیجهٔ آن‌ها را بررسی کند؛
- عملیات حساس را تأیید کند؛
- خروجی را پیش از استفادهٔ نهایی ارزیابی کند.

قاعدهٔ مهم این است:

اجرای موفق یک عملیات، به معنی درست‌بودن نتیجه نیست.

برای مثال، ممکن است یک فایل بدون خطای فنی ویرایش شود، اما تغییر ایجادشده با نیاز محصول یا معماری پروژه سازگار نباشد.

۱.۱۰. اصول استفادهٔ مؤثر از هوش مصنوعی عامل‌محور

اصل اول: هدف را دقیق تعریف کنید

به‌جای درخواست کلی:

کد را بهتر کن.

درخواست مشخص‌تری بنویسید:

تابع اعتبارسنجی ایمیل را بررسی کن. فقط همان تابع را تغییر بده،

رفتار سایر اعتبارسنجی‌ها را تغییر نده و تست‌های مرتبط را اجرا کن.

اصل دوم: دامنهٔ تغییر را محدود کنید

درخواست‌های محدود ساده‌تر بررسی می‌شوند و احتمال تغییر ناخواسته را کاهش می‌دهند.

اصل سوم: ابتدا بررسی، سپس تغییر

بهتر است ابتدا از Claude Code بخواهید:

- ساختار مرتبط را توضیح دهد؛
- فایل‌های درگیر را معرفی کند؛
- برنامهٔ پیشنهادی ارائه دهد؛
- ریسک‌های احتمالی را مشخص کند.

سپس، پس از بررسی، اجازهٔ تغییر داده شود.

اصل چهارم: مجوزها را متناسب انتخاب کنید

برای کارهای ساده، دسترسی محدود کافی است. مجوزهای گسترده فقط زمانی باید استفاده شوند که واقعاً ضروری باشند و کاربر پیامدهای آن را بداند.

اصل پنجم: نتیجه را مستقل بررسی کنید

پس از پایان کار، این موارد را بررسی کنید:

- تغییرات ایجادشده؛
- فایل‌های تغییرکرده؛
- خروجی تست‌ها؛
- پیام‌های خطا؛
- اثر تغییر بر بخش‌های دیگر؛
- سازگاری تغییر با هدف اولیه.

اصل ششم: عملیات حساس را بدون تأیید انجام ندهید

در عملیات حذف، تغییر گسترده، دسترسی خارجی یا اجرای دستور ناشناخته، باید تأیید و بررسی بیشتری انجام شود.

۱.۱۱. چک‌لیست شروع کار با یک ایجنت

پیش از شروع یک وظیفه، این پرسش‌ها را بررسی کنید:

- هدف دقیق کار چیست؟
- کدام فایل‌ها یا بخش‌ها در محدودهٔ کار هستند؟
- چه بخش‌هایی نباید تغییر کنند؟
- معیار موفقیت چیست؟
- چه ابزارهایی برای این کار لازم‌اند؟
- چه مجوزهایی لازم است؟
- آیا ابتدا باید برنامه تهیه شود؟
- چگونه نتیجه را بررسی خواهم کرد؟
- کدام تست‌ها باید اجرا شوند؟
- آیا عملیات غیرقابل‌بازگشت وجود دارد؟

جمع‌بندی فصل

هوش مصنوعی عامل‌محور سیستمی است که برای رسیدن به یک هدف، فقط پاسخ تولید نمی‌کند؛ بلکه می‌تواند برنامه‌ریزی کند، از ابزارها استفاده کند، نتیجهٔ عملیات را مشاهده کند و در صورت نیاز مسیر خود را اصلاح کند.

چرخهٔ اصلی آن به شکل زیر است:

هدف

↓

برنامه‌ریزی

↓

انتخاب ابزار

↓

اجرا

↓

مشاهده

↓

اصلاح

اجزای اصلی این چرخه شامل مدل زبانی، Context، حافظه، ابزارها و سیستم مجوز است. Claude Code نمونه‌ای از کاربرد این رویکرد در توسعهٔ نرم‌افزار است و می‌تواند در بررسی پروژه، تغییر کد، اجرای تست و بازبینی تغییرات کمک کند.

با این حال، ایجنت جایگزین بررسی انسانی نیست. استفادهٔ مؤثر از آن به هدف دقیق، Context مناسب، مجوزهای کنترل‌شده، بررسی تغییرات و اجرای تست وابسته است.

bitwithit

فصل دوم: آشنایی با Claude و Claude Code

مقدمه

Claude و Claude Code هر دو بر پایهٔ مدل‌های زبانی هوش مصنوعی کار می‌کنند، اما برای یک نوع استفاده طراحی نشده‌اند. Claude بیشتر به‌عنوان یک دستیار عمومی برای گفت‌وگو، تحلیل، تولید محتوا و حل مسئله استفاده می‌شود؛ در حالی که Claude Code برای تعامل مستقیم‌تر با پروژه‌های نرم‌افزاری و کمک به فرایند توسعه طراحی شده است.

تفاوت اصلی این دو در این است که Claude معمولاً در سطح گفت‌وگو و تولید پاسخ فعالیت می‌کند، اما Claude Code می‌تواند در محیط مناسب، با فایل‌ها، ابزارهای توسعه، دستورات ترمینال و ساختار پروژه تعامل داشته باشد.

قابلیت‌های واقعی هر دو ابزار به نسخه، محیط اجرا، مدل انتخاب‌شده، ابزارهای فعال و سطح دسترسی بستگی دارد.

۲.۱. Claude چیست؟

Claude یک مدل زبانی و دستیار هوش مصنوعی مکالمه‌ای است که می‌تواند درخواست‌های متنی کاربر را تحلیل کند و بر اساس آن پاسخ تولید کند.

کاربر می‌تواند از Claude برای کارهایی مانند موارد زیر استفاده کند:

- پرسش و پاسخ؛
- توضیح مفاهیم پیچیده؛
- خلاصه‌سازی متن؛
- بازنویسی و ویرایش محتوا؛
- تولید ایده؛
- تحلیل اطلاعات؛
- تولید و توضیح کد؛
- برنامه‌ریزی برای انجام یک کار؛
- بررسی یک مسئلهٔ فنی یا محتوایی.

Claude در حالت پایه بیشتر با اطلاعاتی کار می‌کند که در گفت‌وگو یا فایل‌ها و ورودی‌های در دسترس قرار گرفته‌اند. بنابراین کیفیت پاسخ آن به کیفیت درخواست، اطلاعات ارائه‌شده و زمینهٔ مسئله وابسته است.

نمونهٔ استفادهٔ عمومی

تفاوت بین REST API و GraphQL را با یک مثال ساده توضیح بده.

در این حالت، Claude می‌تواند یک توضیح آموزشی، جدول مقایسه یا نمونه‌کد ارائه کند.

نمونهٔ دیگر:

این تابع پایتون را توضیح بده و مشکلات احتمالی آن را مشخص کن.

در اینجا Claude می‌تواند کد را تحلیل کند، اما اگر به محیط اجرای واقعی دسترسی نداشته باشد، نمی‌تواند اجرای آن را به‌طور مستقل بررسی کند.

۲.۲. Claude برای چه کارهایی مناسب است؟

Claude برای کارهایی مناسب است که بیشتر به تحلیل، تولید، سازمان‌دهی و توضیح اطلاعات نیاز دارند.

۲.۲.۱. یادگیری و آموزش

Claude می‌تواند یک مفهوم را در سطوح مختلف توضیح دهد

مفهوم Context Window را ابتدا برای یک فرد مبتدی و سپس برای یک برنامه‌نویس توضیح بده.

در چنین درخواستی، کاربر می‌تواند سطح توضیح، مثال، زبان و میزان جزئیات را مشخص کند.

۲.۲.۲. تحلیل و خلاصه‌سازی

Claude می‌تواند متن‌های طولانی را بررسی و خلاصه کند، نکات اصلی را استخراج کند یا ساختار آن‌ها را بازسازی کند.

برای مثال:

این متن را به سه بخش تقسیم کن:

مسئله، راه‌حل پیشنهادی و محدودیت‌ها.

۲.۲.۳. تولید و ویرایش محتوا

از Claude می‌توان برای تولید متن، بازنویسی، اصلاح ساختار و تنظیم لحن استفاده کرد.

مثال:

این توضیح فنی را برای مستندات پروژه بازنویسی کن.

لحن رسمی باشد و مثال اجرایی هم داشته باشد.

۲.۲.۴. کمک در برنامه‌نویسی

Claude می‌تواند:

- قطعه‌کد تولید کند؛
- کد موجود را توضیح دهد؛

- خطای احتمالی را بررسی کند؛
- برای یک قابلیت برنامه‌ریزی کند؛
- تست پیشنهاد دهد؛
- چند روش پیاده‌سازی را مقایسه کند.

اما پیشنهاد کد با اجرای واقعی و تأیید عملکرد آن یکسان نیست.

۲.۳. Claude Code چیست؟

Claude Code ابزاری تخصصی برای کمک به توسعهٔ نرم‌افزار است که Claude را در کنار محیط پروژه و ابزارهای توسعه قرار می‌دهد.

در یک محیط مناسب، Claude Code می‌تواند با موارد زیر تعامل داشته باشد:

- فایل‌های پروژه؛
- پوشه‌ها و ساختار کد؛
- دستورات ترمینال؛
- تست‌ها؛
- سیستم کنترل نسخه؛
- ابزارهای خارجی؛
- قواعد و دستورالعمل‌های پروژه.

Claude Code برای وظایفی طراحی شده است که فقط به تولید توضیح نیاز ندارند، بلکه به بررسی و اجرای مرحله‌ای کار در یک پروژهٔ واقعی مربوط هستند.

نمونهٔ درخواست به Claude Code

ساختار این پروژه را بررسی کن، محل مدیریت خطاها را پیدا کن

و قبل از هر تغییری یک برنامهٔ پیشنهادی ارائه بده.

در این درخواست، Claude Code باید ابتدا پروژه را بررسی کند و سپس برنامه ارائه دهد.

نمونهٔ دیگر:

خطای تست‌های مربوط به احراز هویت را پیدا کن.

فقط فایل‌های مرتبط با احراز هویت را تغییر بده و بعد تست‌ها را اجرا کن.

اینجا علاوه بر هدف، محدودهٔ تغییر و معیار بررسی نیز مشخص شده است.

۲.۴. تفاوت Claude و Claude Code

موضوع	Claude	Claude Code
نوع استفاده	گفت‌وگو، تحلیل و تولید محتوا	کمک عملی به توسعهٔ نرم‌افزار
محیط اصلی	فضای مکالمه	پروژه و محیط توسعه
دسترسی به فایل‌ها	وابسته به فایل‌های ارائه‌شده	امکان تعامل با فایل‌های پروژه، در صورت فعال‌بودن دسترسی
اجرای دستور	معمولاً محدود یا وابسته به محیط	امکان اجرای دستورات مجاز در محیط پروژه
استفاده از تست‌ها	پیشنهاد یا تحلیل تست	امکان اجرای تست‌های پروژه، در صورت دسترسی
مدیریت تغییرات	تولید پیشنهاد یا کد	بررسی و اعمال تغییرات در فایل‌ها
تعامل با Git	توضیح دستورها و پیشنهاد گردش‌کار	امکان بررسی وضعیت و تفاوت‌ها، در محیط مجاز
ابزارهای خارجی	وابسته به قابلیت‌های فعال	امکان اتصال از طریق ابزارها و MCP

این جدول یک مقایسهٔ کلی است. قابلیت واقعی هر ابزار ممکن است بر اساس نسخه، حساب کاربری، تنظیمات و محیط اجرا متفاوت باشد.

۲.۵. Claude Code چه مسئله‌ای را حل می‌کند؟

در توسعهٔ نرم‌افزار، بسیاری از وظایف نیازمند رفت‌وبرگشت میان چند فعالیت هستند:

1. پیدا کردن فایل مرتبط؛
2. فهم ساختار کد؛
3. بررسی وابستگی‌ها؛
4. اعمال تغییر؛
5. اجرای تست؛
6. بررسی نتیجه؛
7. اصلاح خطا.

اگر این مراحل به‌صورت دستی و بدون برنامه انجام شوند، احتمال فراموشی یا تغییر ناخواسته افزایش پیدا می‌کند.

Claude Code می‌تواند این مراحل را در یک گردش‌کار منسجم‌تر قرار دهد. برای مثال، به جای اینکه کاربر جداگانه بپرسد:

فایل مربوط به ورود کاربران کجاست؟

سپس:

این فایل را توضیح بده.

و بعد:

این خطا را اصلاح کن.

می‌توان هدف را به شکل محدود و مرحله‌ای بیان کرد:

مسیر ورود کاربران را بررسی کن، علت خطای اعتبارسنجی را پیدا کن،

ابتدا برنامهٔ اصلاح را توضیح بده و پس از تأیید من تغییر را اعمال کن.

در این حالت، ابزار باید ابتدا زمینه را جمع‌آوری کند، برنامه بسازد و سپس برای تغییر اقدام کند.

۲.۶. قابلیت‌های اصلی Claude Code

۲.۶.۱. بررسی ساختار پروژه

Claude Code می‌تواند برای فهم اولیهٔ پروژه به بررسی موارد زیر کمک کند:

- نام پوشه‌ها؛
- فایل‌های اصلی؛
- فایل‌های تنظیمات؛
- فایل‌های تست؛
- نقطهٔ شروع برنامه؛
- وابستگی‌ها؛
- دستورهای اجرای پروژه.

در شروع کار، بهتر است از آن بخواهید ساختار را توضیح دهد:

ساختار پروژه را بررسی کن و نقش پوشه‌ها و فایل‌های اصلی را توضیح بده.

هنوز هیچ فایلی را تغییر نده.

این درخواست دامنهٔ کار را مشخص می‌کند و از شروع تغییرات زودهنگام جلوگیری می‌کند.

۲.۶.۲. جست‌وجو در کد

یکی از کاربردهای مهم Claude Code، پیدا کردن بخش‌های مرتبط با یک قابلیت یا خطا است.

مثال:

تمام محل‌هایی را که در آن‌ها احراز هویت کاربر انجام می‌شود پیدا کن

و ارتباط بین آن فایل‌ها را توضیح بده.

نتیجهٔ این جست‌وجو می‌تواند برای برنامه‌ریزی تغییر بعدی استفاده شود.

۲.۶.۳. توضیح کد و ارتباط فایل‌ها

Claude Code می‌تواند فقط یک فایل منفرد را توضیح ندهد، بلکه ارتباط میان چند فایل را نیز بررسی کند.

برای مثال:

مسیر اجرای درخواست از API تا لایهٔ سرویس را دنبال کن

و مشخص کن خطا در کدام مرحله مدیریت می‌شود.

این نوع درخواست برای پروژه‌هایی که ساختار چندلایه دارند مفید است.

۲.۶.۴. پیشنهاد و اعمال تغییر

Claude Code می‌تواند تغییرات پیشنهادی را تولید یا در صورت وجود مجوز، در فایل‌ها اعمال کند.

بهتر است تغییرات بزرگ به بخش‌های کوچک تقسیم شوند. نمونه:

ابتدا فقط تغییرات لازم برای لایهٔ سرویس را پیشنهاد بده.

پس از بررسی، سراغ تست‌ها برو.

این روش امکان بررسی مرحله‌ای را بیشتر می‌کند.

۲.۶.۵. اجرای تست و بررسی نتیجه

یکی از تفاوت‌های مهم Claude Code با یک گفت‌وگوی معمولی، امکان استفاده از دستورهای تست در محیط پروژه است.

مثال:

پس از اعمال تغییر، فقط تست‌های مربوط به احراز هویت را اجرا کن

و نتیجهٔ هر تست را گزارش بده.

با این حال، خروجی موفق یک تست به معنی بی‌نقص‌بودن کامل تغییر نیست. تست‌ها فقط بخش‌هایی از رفتار سیستم را پوشش می‌دهند.

۲.۶.۶. کمک به بررسی تغییرات

Claude Code می‌تواند در بررسی تغییرات ایجادشده نیز کمک کند.

نمونه:

تغییرات ایجادشده را بررسی کن، فایل‌های تغییرکرده را فهرست کن

و تغییرات خارج از محدودهٔ درخواست را مشخص کن.

در گردش‌کار پیشنهادی، استفاده از /diff برای مشاهدهٔ تفاوت‌ها و /review برای بازبینی اهمیت دارد.

۲.۷. نقش Context Window در Claude Code

Claude برای تحلیل درخواست‌ها، فایل‌ها، خروجی ابزارها و پیام‌های گفت‌وگو از Context استفاده می‌کند. مقدار اطلاعاتی که مدل می‌تواند در یک مرحله در نظر بگیرد، محدود است و به آن **Context Window** گفته می‌شود.

در Claude Code، Context ممکن است شامل این موارد باشد:

- پیام اولیهٔ کاربر؛
- برنامهٔ ایجادشده؛
- فایل‌های خوانده‌شده؛
- خروجی دستورات؛
- خطاهای تست؛
- تغییرات ایجادشده؛
- دستورالعمل‌های CLAUDE.md.

مشکل Context بیش‌ازحد

اگر اطلاعات بیش از حد وارد Context شود، ممکن است:

- اطلاعات مهم در میان جزئیات گم شود؛
- پاسخ‌ها کندتر یا کم‌دقت‌تر شوند؛
- بخشی از زمینهٔ قبلی دیگر در دسترس نباشد؛
- مدل ارتباط میان فایل‌ها را نادرست برداشت کند.

برای مدیریت این وضعیت، می‌توان از خلاصه‌سازی و دستورهایی مانند /compact استفاده کرد.

روش بهتر برای مدیریت Context

- فقط فایل‌های مرتبط را بررسی کنید؛
- هدف را در طول کار ثابت و روشن نگه دارید؛
- وظایف بزرگ را به چند مرحله تقسیم کنید؛

- خروجی‌های غیرضروری را وارد جلسه نکنید؛
- قوانین پایدار پروژه را در CLAUDE.md قرار دهید؛
- در صورت طولانی‌شدن جلسه، Context را فشرده کنید.

۲.۸. وابستگی Claude Code به محیط اجرا

Claude Code یک ابزار مستقل از محیط نیست. نحوهٔ عملکرد آن به شرایط اجرایی بستگی دارد.

عوامل مؤثر بر عملکرد

- سیستم‌عامل؛
- نسخهٔ Claude Code؛
- مدل فعال؛
- ابزارهای در دسترس؛
- مجوزهای کاربر؛
- ساختار پروژه؛
- روش نصب وابستگی‌ها؛
- دستورهای تعریف‌شده برای اجرا و تست؛
- اتصال‌های خارجی و MCP.

برای همین، نباید فرض کرد هر دستور یا قابلیتی در همهٔ محیط‌ها دقیقاً یکسان عمل می‌کند.

نمونهٔ وابستگی محیطی

ممکن است یک پروژه با دستور زیر اجرا شود:

```
bash
npm test
```

اما پروژه‌ای دیگر از دستور زیر استفاده کند:

```
bash
pytest
```

Claude Code باید ابتدا دستورهای واقعی همان پروژه را تشخیص دهد یا از کاربر دربارهٔ آن‌ها سؤال کند. اجرای یک دستور عمومی و اشتباه می‌تواند نتیجهٔ گمراه‌کننده ایجاد کند.

۲.۹. توانایی‌ها در برابر محدودیت‌ها

توانایی‌های Claude Code

بسته به سطح دسترسی و ابزارهای فعال، Claude Code می‌تواند:

- پروژه را بررسی کند؛
- فایل‌های مرتبط را پیدا کند؛
- کد را توضیح دهد؛
- برنامهٔ اصلاح پیشنهاد دهد؛
- تغییرات محدود اعمال کند؛
- تست اجرا کند؛
- تفاوت فایل‌ها را بررسی کند؛
- خطاهای احتمالی را تحلیل کند؛
- از ابزارهای خارجی استفاده کند؛
- بخشی از گردش‌کار توسعه را مدیریت کند.

محدودیت‌های Claude Code

Claude Code ممکن است:

- هدف کاربر را نادرست تفسیر کند؛
- فایل اشتباه را تغییر دهد؛
- بخشی از وابستگی‌ها را نادیده بگیرد؛
- تست ناقص یا نامرتبط اجرا کند؛
- خروجی یک تست را اشتباه تفسیر کند؛
- تغییری ایجاد کند که با معماری پروژه سازگار نباشد؛
- به دلیل نداشتن مجوز، نتواند وظیفه‌ای را کامل کند؛
- دربارهٔ قابلیت‌های محیط خود برداشت نادرست داشته باشد.

بنابراین باید میان این دو گزاره تفاوت گذاشت:

Claude Codeتوانست یک تغییر ایجاد کند.

و:

تغییر ایجادشده صحیح، کامل و مناسب پروژه است.

گزارهٔ دوم فقط پس از بررسی و آزمون قابل‌قبول است.

bitwithit

۲.۱۰. چه زمانی از Claude استفاده کنیم و چه زمانی از Claude Code؟

استفاده از Claude

Claude برای این نوع کارها مناسب‌تر است:

- یادگیری یک مفهوم؛
- دریافت توضیح نظری؛
- تحلیل یک متن؛
- تولید مستندات؛
- طراحی اولیهٔ یک راه‌حل؛
- مقایسهٔ روش‌های مختلف؛
- تولید قطعه‌کد مستقل؛
- بازنویسی یا خلاصه‌سازی.

مثال:

سه روش برای پیاده‌سازی کش در یک API پایتون را مقایسه کن.

استفاده از Claude Code

Claude Code برای این نوع کارها مناسب‌تر است:

- بررسی یک ریپازیتوری واقعی؛
- پیدا کردن محل یک قابلیت در پروژه؛
- اعمال تغییر در چند فایل مرتبط؛
- اجرای تست‌ها؛
- بررسی خطاهای اجرایی؛
- مشاهدهٔ تغییرات Git؛
- دنبال‌کردن یک گردش‌کار توسعه.

مثال:

در این پروژه، محل پیاده‌سازی کش را پیدا کن،

روش فعلی را توضیح بده و یک برنامهٔ محدود برای اصلاح آن پیشنهاد کن.

استفادهٔ ترکیبی

bitwithit

در بسیاری از کارها می‌توان از هر دو استفاده کرد:

1. در Claude، مسئله و راه‌حل‌های احتمالی بررسی شود.
2. در Claude Code، ساختار واقعی پروژه تحلیل شود.
3. تغییر محدود اعمال شود.
4. تست و بررسی انجام شود.
5. نتیجه در Claude یا مستندات پروژه ثبت شود.

۲.۱۱. یک نمونهٔ عملی از شروع کار با Claude Code

فرض کنید پروژه‌ای دارید و می‌خواهید خطای ورود کاربران را بررسی کنید.

درخواست نامناسب:

مشکل ورود را حل کن.

این درخواست اطلاعات کافی ندارد. مشخص نیست:

- مشکل کجاست؛
- چه فایل‌هایی در محدوده هستند؛
- آیا باید تغییر اعمال شود؛
- چه چیزی معیار موفقیت است؛
- آیا تستی باید اجرا شود.

درخواست بهتر:

خطای ورود کاربران را در پروژه بررسی کن.

ابتدا فایل‌ها و مسیر اجرای مرتبط با احراز هویت را پیدا و توضیح بده.

هنوز تغییری ایجاد نکن. پس از بررسی، علت احتمالی و برنامهٔ اصلاح را ارائه کن.

ساختار API و رفتار سایر بخش‌ها نباید تغییر کند.

پس از دریافت برنامه، می‌توان درخواست مرحلهٔ بعد را ارسال کرد:

برنامه را تأیید می‌کنم. فقط تغییرات ضروری مربوط به احراز هویت را اعمال کن،

سپس تست‌های مرتبط را اجرا کن و فهرست فایل‌های تغییرکرده را گزارش بده.

در پایان:

تغییرات را با diff بررسی کن و موارد خارج از محدودهٔ درخواست

یا ریسک‌های احتمالی را مشخص کن.

این نمونه، استفادهٔ مرحله‌ای و کنترل‌شده از Claude Code را نشان می‌دهد.

۲.۱۲. قواعد پایه برای استفاده از Claude و Claude Code

قاعدهٔ اول: قابلیت‌ها را قطعی فرض نکنید

برخی قابلیت‌ها به نسخه، محیط یا تنظیمات وابسته‌اند. پیش از استفاده، باید مشخص شود که قابلیت موردنظر در محیط فعلی وجود دارد یا نه.

قاعدهٔ دوم: Claude را منبع بی‌خطا ندانید

پاسخ‌های Claude و Claude Code باید بررسی شوند، به‌خصوص در موارد زیر:

- تغییرات حساس؛
- کد امنیتی؛
- عملیات داده؛
- منطق پزشکی یا مالی؛
- حذف یا جابه‌جایی فایل‌ها؛
- اصلاحات گسترده در پروژه.

قاعدهٔ سوم: تغییرات را محدود نگه دارید

درخواست محدود، کنترل و بازبینی را ساده‌تر می‌کند.

قاعدهٔ چهارم: همیشه نتیجه را بررسی کنید

بعد از هر تغییر، موارد زیر باید بررسی شوند:

- آیا هدف اصلی محقق شده است؟
- آیا فایل‌های اضافی تغییر کرده‌اند؟
- آیا تست‌ها موفق شده‌اند؟
- آیا خطای جدیدی ایجاد شده است؟
- آیا ساختار پروژه حفظ شده است؟

قاعدهٔ پنجم: از تأیید مرحله‌ای استفاده کنید

برای کارهای حساس، بهتر است مراحل به این شکل انجام شوند:

bitwitit

بررسی ← برنامه‌ریزی ← تأیید ← تغییر ← تست ← بازبینی

۲.۱۳. چک‌لیست انتخاب ابزار مناسب

پیش از شروع کار، از خودتان بپرسید:

- آیا فقط به توضیح یا تولید متن نیاز دارم؟
- آیا باید فایل‌های واقعی پروژه بررسی شوند؟
- آیا لازم است دستوری اجرا شود؟
- آیا تست باید اجرا شود؟
- آیا تغییر باید مستقیماً در فایل اعمال شود؟
- آیا کار به Git یا MCP نیاز دارد؟
- آیا اطلاعات کافی برای شروع وجود دارد؟
- آیا مجوز لازم برای اجرای کار فعال است؟

اگر فقط به تحلیل یا توضیح نیاز دارید، Claude معمولی ممکن است کافی باشد. اگر کار به تعامل با پروژه و اجرای مرحله‌ای نیاز دارد، Claude Code گزینهٔ مناسب‌تری است.

جمع‌بندی فصل

Claude یک دستیار هوش مصنوعی عمومی برای گفت‌وگو، تحلیل، تولید محتوا و کمک فنی است. Claude Code نسخه یا ابزار تخصصی‌تری برای کمک به توسعهٔ نرم‌افزار محسوب می‌شود که می‌تواند در محیط مناسب با فایل‌ها، دستورات، تست‌ها و ابزارهای پروژه تعامل داشته باشد.

تفاوت اصلی آن‌ها در سطح تعامل با محیط است:

Claude:
درخواست ← تحلیل و پاسخ

Claude Code:
هدف ← بررسی پروژه ← برنامه‌ریزی ← استفاده از ابزار ← تغییر ← تست ← بازبینی

Claude Code می‌تواند فرایند توسعه را سریع‌تر و منسجم‌تر کند، اما نتیجهٔ آن باید بررسی شود. توانایی‌های ابزار به محیط، مجوزها، مدل، ابزارهای فعال و Context موجود وابسته است. بهترین روش استفاده نیز این است که کار با بررسی آغاز شود، تغییرات محدود باشند و نتیجه با تست و بازبینی انسانی تأیید شود.

فصل سوم: راه‌اندازی نخستین پروژه با Claude Code

هدف این فصل، ایجاد یک **نقطهٔ شروع قابل‌اعتماد** است: Claude Code باید در پروژهٔ درست اجرا شود، وضعیت فعلی Git مشخص باشد و پیش از هر تغییر، دستورهای اجرایی و محدودیت‌های پروژه روشن شوند.

ساختار این فصل از فصل سوم codex2.pdf، صفحات ۱۸ تا ۲۲، گرفته شده است. فایل مرجع دربارهٔ Codex و AGENTS.md است؛ در اینجا همان روش آماده‌سازی را برای Claude Code و CLAUDE.md تطبیق می‌دهیم، نه اینکه این دو ابزار را یکسان فرض کنیم.

۳.۱. بررسی پیش‌نیازها

پیش از شروع، این موارد باید مشخص باشند:

- پوشهٔ پروژه و ریشهٔ ریپوی Git؛
- شاخهٔ فعال و تغییرات ثبت‌نشده؛
- نصب و قابل‌اجرا بودن Claude Code؛
- ابزارهای موردنیاز پروژه، مانند Python، Node.js یا Flutter؛
- روش نصب وابستگی‌ها و اجرای برنامه؛
- فرمان‌های تست، lint و بررسی نوع‌ها.

این فهرست بر اساس پیش‌نیازهای فایل مرجع تنظیم شده است. **منبع: codex2.pdf، ص. ۱۸.**

برای بررسی در دسترس بودن CLI:

```
bash
claude --version
```

اگر فرمان شناسایی نشد، ابتدا نصب یا مسیر اجرایی آن را بررسی کنید. تا وقتی ابزارها و محیط پروژه آماده نیستند، شکست اجرای تست را نباید لزوماً به خطای کد نسبت داد.

۳.۲. انتخاب ریشهٔ پروژه

ریشهٔ پروژه معمولاً پوشه‌ای است که فایل‌های تنظیمات، مستندات و پوشه‌های اصلی در آن قرار دارند:

```
my-project/
├── .git/
├── src/
├── tests/
├── README.md
├── pyproject.toml
└── CLAUDE.md
```

از همین پوشه شروع کنید:

```
bash
cd /path/to/my-project
git rev-parse --show-toplevel
```

فرمان دوم، ریشهٔ ریپوی Git را نمایش می‌دهد. سپس نشست را آغاز کنید:

```
claude
```

شروع از src یا tests ممکن است زمینهٔ اولیه را محدود کند. برای شناخت کل پروژه، ریشهٔ ریپو نقطهٔ شروع مناسب‌تری است؛ مگر اینکه عمداً روی یک زیرپروژه کار کنید.

۳.۳. بررسی وضعیت اولیهٔ Git

پیش از درخواست تغییر، وضعیت فعلی را ببینید:

```
bash
git status
git status --short
git branch --show-current
```

نمونهٔ خروجی کوتاه:

```
M src/auth/login.py
?? tests/test_login.py
```

M نشان‌دهندهٔ تغییر فایل و ?? نشان‌دهندهٔ فایل دنبال‌نشده است. در خروجی کوتاه، دو ستون اول وضعیت staging و پوشهٔ کاری را نشان می‌دهند.

تغییرات موجود را متعلق به کار فعلی فرض نکنید. ممکن است بخشی از کار قبلی شما باشند. فایل مرجع نیز بر شناسایی و حفظ این تغییرات تأکید دارد.

درخواست اولیهٔ مناسب:

ابتدا وضعیت Git و شاخهٔ فعال را بررسی کن.

فایل‌های تغییرکرده و دنبال‌نشده را گزارش بده.

هیچ فایلی را تغییر نده و تغییرات موجود را بازگردانی نکن.

۳.۴. ایجاد شاخهٔ کاری

برای یک تغییر مستقل، شاخهٔ جدا بسازید:

```
bash
git switch -c fix/login-validation
```

شاخهٔ جدا، بازبینی و تفکیک کار را ساده‌تر می‌کند.

نکتهٔ مهم: **ساخت شاخه، تغییرات ثبت‌نشده را پاک یا خودکار ایزوله نمی‌کند.** ابتدا وضعیت Git را بررسی کنید. همچنین عملیات commit، push، merge، rebase و فرمان‌های مخرب را بدون تأیید به عامل نسپارید.

۳.۵. شناسایی فرمان‌های واقعی پروژه

قبل از نصب وابستگی یا اجرای تست، README.md و فایل‌های تنظیمات پروژه را بررسی کنید. نام فرمان را از روی زبان پروژه حدس نزنید.

نوع بررسی	نمونهٔ فرمان، فقط در صورت استفادهٔ پروژه
تست Python	python -m pytest
تست JavaScript	npm test
تست Flutter	flutter test
بررسی lint	ruff check . یا npm run lint
بررسی نوع‌ها	mypy src یا npm run typecheck

این‌ها نمونه‌اند، نه دستورهای عمومی برای همهٔ پروژه‌ها. **منبع نمونه‌های اعتبارسنجی: codex2.pdf، صص. ۳۹–۴۰ و ۵۲.**

درخواست پیشنهادی:

README و فایل‌های تنظیمات را بررسی کن.

فرمان‌های نصب، اجرا، تست، lint و type-check را مشخص کن.

برای هر فرمان، منبع آن را بگو.

فعلاً وابستگی نصب نکن و فرمان تغییردهندهٔ محیط اجرا نکن.

۳.۶. ایجاد یا بررسی CLAUDE.md

CLAUDE.md محل ثبت راهنمای پایدار پروژه برای Claude Code است. آن را کوتاه، دقیق و مبتنی بر واقعیت پروژه نگه دارید.

در فایل مرجع، این نقش با AGENTS.md توضیح داده شده است: معرفی پروژه، فرمان‌های اجرایی، محدودیت‌های تغییر و سیاست امنیتی

نمونهٔ زیر صرفاً برای یک پروژهٔ فرضی Python است:

```
# Project
Python API. Application code is in src/; tests are in tests/.

# Validation
- Tests: python -m pytest
- Lint: ruff check .
```

```
# Change Rules
- Keep changes scoped to the requested task.
- Preserve existing user changes.
- Do not change public API contracts without approval.

# Safety
- Do not read or expose secrets or credentials.
- Ask before installing dependencies or running destructive commands.
- Do not commit, push, merge, or rebase without approval.

# Reporting
Report changed files, checks performed, and unresolved risks.
```

فرمان‌ها و مسیرهای این نمونه را فقط پس از تأیید در پروژهٔ واقعی استفاده کنید. همچنین **CLAUDE.md راهنمای رفتاری است، نه مرز امنیتی؛** محدودیت واقعی دسترسی باید با تنظیمات مجوز و محیط اجرا اعمال شود. اصل کمترین دسترسی لازم در فایل مرجع نیز توصیه شده است.

۳.۷. نخستین درخواست: شناخت، بدون تغییر

پس از آماده‌سازی، اولین درخواست را به شناخت پروژه محدود کنید:

پروژه را بررسی کن، اما هیچ تغییری ایجاد نکن.

ساختار پوشه‌ها، نقطهٔ ورود، مسیر تست‌ها و فرمان‌های اعتبارسنجی را گزارش بده.

CLAUDE.mdرا بررسی کن و ناسازگاری‌های آن با پروژه را مشخص کن.

فایل‌های حساس را نخوان و برای موارد نامشخص حدس نزن.

پاسخ را با ساختار واقعی پروژه مقایسه کنید. هدف، تأیید شناخت اولیه است، نه شروع فوری اصلاحات.

چک‌لیست پایان فصل: ریشهٔ درست، شاخهٔ مشخص، تغییرات قبلی شناسایی‌شده، فرمان‌های واقعی تأییدشده، راهنمای پروژه بررسی‌شده و مجوزهای محدود.

فصل چهارم: چرخهٔ کاری پیشنهادی با Claude Code

هدف این فصل، ارائهٔ یک روند قابل‌اعتماد برای کار با Claude Code است؛ روندی که از شناخت وضعیت فعلی شروع می‌شود و پس از برنامه‌ریزی، اجرای محدود، تست و بازبینی، به تصمیم انسانی دربارهٔ commit می‌رسد.

چرخهٔ کامل به شکل زیر است:

```
شناخت
 ↓
تعریف هدف
 ↓
برنامه‌ریزی
 ↓
اجرای مرحله‌ای
 ↓
تست
 ↓
بازبینی
 ↓
تصمیم برای commit
```

هیچ مرحله‌ای نباید بدون دلیل حذف شود. در تغییرات کوچک می‌توان بعضی مراحل را کوتاه‌تر کرد، اما اصل چرخه باید حفظ شود

نکتهٔ تطبیقی: متن فایل مرجع در برخی بخش‌ها از نام Codex استفاده می‌کند. در این فصل، همان روش برای Claude Code بازنویسی شده است. دستورهایی مانند /plan و /review فقط در صورتی باید استفاده شوند که در محیط Claude Code شما واقعاً در دسترس باشند. در غیر این صورت، از درخواست متنی معادل استفاده کنید.

۴.۱. شناخت وضعیت فعلی

پیش از هر تغییری، باید بدانید پروژه در چه وضعیتی قرار دارد. در این مرحله هدف، **شناخت است، نه اصلاح**.

ابتدا وضعیت Git را بررسی کنید:

```
bash
git status
git branch --show-current
git diff --stat
```

سپس از Claude Code بخواهید بدون دست‌کاری فایل‌ها، ساختار مرتبط با کار را بررسی کند:

ابتدا بدون تغییر هیچ فایلی، محل پیاده‌سازی قابلیت موردنظر را پیدا کن.

فایل‌های مرتبط، تست‌های موجود، وابستگی‌های مهم و نقاط ورود را فهرست کن.

خطرها و فرمان‌های لازم برای اعتبارسنجی را نیز گزارش بده.

Claude Code باید در این مرحله موارد زیر را مشخص کند:

- قابلیت موردنظر در کدام فایل یا ماژول پیاده‌سازی شده است؛
- چه فایل‌هایی مستقیماً با آن ارتباط دارند؛
- چه تست‌هایی برای آن وجود دارد؛
- چه وابستگی‌هایی ممکن است روی تغییر اثر بگذارند؛
- کدام بخش‌های رفتار فعلی نباید تغییر کنند؛
- برای اعتبارسنجی چه تست‌ها یا فرمان‌هایی باید اجرا شوند.

اگر پاسخ کلی یا مبهم بود، هنوز زمان اجرای تغییر نیست. سؤال را دقیق‌تر کنید:

فقط محل route مربوط به پروفایل و middleware احراز هویت را بررسی کن.

تست‌های مربوط به کاربر ناشناس را نیز پیدا کن.

هیچ فایلی را تغییر نده.

شناخت ناقص در ابتدای کار می‌تواند باعث تغییر در فایل اشتباه، شکستن رفتار قبلی یا حذف ناخواستهٔ منطق موجود شود.

۴.۲. تعریف هدف و معیار پذیرش

درخواست «این بخش را بهتر کن» برای یک عامل برنامه‌نویس بیش از حد مبهم است. هدف باید به نتیجهٔ قابل‌بررسی تبدیل شود.

یک تعریف هدف مناسب، این بخش‌ها را دارد:

بخش	پرسش اصلی
هدف	دقیقاً چه چیزی باید تغییر کند؟
محدوده	کدام فایل‌ها یا ماژول‌ها مجاز به تغییر هستند؟
خارج از محدوده	چه بخش‌هایی نباید تغییر کنند؟
معیار پذیرش	از کجا بفهمیم کار درست انجام شده است؟
تست	کدام تست‌ها باید موفق شوند؟
محدودیت	چه کارهایی بدون تأیید مجاز نیست؟

نمونه:

هدف:

اعتبارسنجی ورودی endpoint ورود را اصلاح کن.

محدوده:

فقط فایل handler ورود و تست‌های مستقیم آن.

خارج از محدوده:

مدل کاربر، سیستمsession ، migrationو رابط کاربری.

معیار پذیرش:

-ورودی بدون email معتبر رد شود.

-رمز عبور خالی رد شود.

-پاسخ خطا ساختار فعلی API را حفظ کند.

-تست‌های موجود ورود همچنان موفق باشند.

محدودیت:

-ابتدا فقط برنامه ارائه بده.

-فعلاً هیچ فایلی را تغییر نده.

commit -و push انجام نده.

معیار پذیرش باید تا حد امکان قابل‌آزمایش باشد. عباراتی مانند «کد را تمیزتر کن» یا «امنیت را بهتر کن» به‌تنهایی معیار پذیرش محسوب نمی‌شوند.

۴.۳. برنامه‌ریزی با /plan

پس از روشن شدن هدف، نوبت برنامه‌ریزی است. در محیط‌هایی که /plan پشتیبانی می‌شود، می‌توان از آن استفاده کرد:

/plan

یا:

برای این هدف یک برنامهٔ مرحله‌به‌مرحله ارائه بده.

فعلاً هیچ فایلی را تغییر نده.

یک برنامهٔ خوب باید شامل این موارد باشد:

1. هدف کلی تغییر؛
2. فایل‌ها و ماژول‌های احتمالی؛
3. ترتیب اجرای مراحل؛
4. وابستگی میان مراحل؛
5. تست‌های موردنیاز؛
6. ریسک‌های احتمالی؛
7. تصمیم‌هایی که به تأیید کاربر نیاز دارند.

نمونهٔ درخواست:

بر اساس هدف و محدودهٔ زیر برنامه بده:

هدف:

اعتبارسنجی ورودی login endpoint را اصلاح کن.

لطفاً مشخص کن:

-کدام فایل‌ها باید بررسی شوند؛

-کدام فایل‌ها احتمالاً تغییر می‌کنند؛

-چه تست‌هایی باید اضافه یا اصلاح شوند؛

-ترتیب اجرای کار چیست؛

-چه رفتارهای قبلی باید حفظ شوند؛

-چه ریسک‌هایی وجود دارد.

تا زمان تأیید من، هیچ فایلی را تغییر نده.

برنامه نباید صرفاً فهرستی از فایل‌ها باشد. باید توضیح دهد چرا هر فایل درگیر است و چگونه می‌توان موفقیت تغییر را سنجید.

برای مثال، برنامهٔ مناسب ممکن است چنین ساختاری داشته باشد:

۱ .بررسی route ورود و محل اعتبارسنجی فعلی.

۲ .بررسی تست‌های ورودی معتبر و نامعتبر.

bitwithit

۳ .تعیین حداقل تغییر لازم در.handler

۴ .افزودن تست برای ورودی‌های خالی و نامعتبر.

۵ .اجرای تست‌های مستقیم.endpoint

۶ .بررسی git diff و اطمینان از محدود ماندن تغییرات.

تا پایان مرحلهٔ برنامه‌ریزی، فایل‌ها نباید تغییر کنند.

۴.۴. اجرای محدود و مرحله‌ای

پس از تأیید برنامه، تغییر را در کوچک‌ترین گام منطقی اجرا کنید. درخواست‌های بسیار بزرگ، مانند نمونهٔ زیر، کنترل‌پذیری کمی دارند:

کل سیستم احراز هویت را بازنویسی کن.

این درخواست مشخص نمی‌کند:

- کدام رفتار باید حفظ شود؛
- کدام فایل‌ها مجاز به تغییر هستند؛
- تست موفقیت چیست؛
- بازنویسی تا چه حد باید انجام شود.

در عوض، کار را به بخش‌های کوچک تقسیم کنید:

برنامهٔ پیشنهادی تأیید شد.

اکنون فقط اعتبارسنجی ورودی login endpoint را پیاده‌سازی کن.

محدودهٔ مجاز:

-فایل handler ورود

-تست‌های مستقیم همانendpoint

لطفاً:

-تغییر را محدود نگه دار؛

-ساختار پاسخ‌های فعلی را حفظ کن؛

-پس از تغییر، فایل‌های اصلاح‌شده و خلاصهٔ تغییرات را گزارش بده؛

bitwithit

commit -و push انجام نده.

در برخی محیط‌ها ممکن است دستورهایی مانند /goal در فهرست دستورات وجود داشته باشد:

/goal

اما نباید فرض کرد این دستور در همهٔ نصب‌های Claude Code موجود است. معادل متنی آن قابل‌اعتمادتر است:

هدف تأییدشده را در محدودهٔ مشخص اجرا کن و از تغییر خارج از محدوده خودداری کن.

اجرای مرحله‌ای چند مزیت دارد:

- diff کوچک‌تر و قابل‌فهم‌تر می‌ماند؛
- علت خطا راحت‌تر پیدا می‌شود؛
- بازگردانی تغییر ساده‌تر است؛
- احتمال تغییر ناخواسته کاهش می‌یابد؛
- بررسی هر مرحله امکان‌پذیر می‌شود.

۴.۵. بررسی اولیهٔ تغییرات

پس از اجرای تغییر، بلافاصله سراغ commit نروید. ابتدا وضعیت ریپو و دامنهٔ تغییرات را بررسی کنید:

```
bash
git status
git diff --stat
git diff -- path/to/changed-file
```

از Claude Code نیز بخواهید گزارش مستقلی ارائه دهد:

تغییرات انجام‌شده را گزارش کن.

فایل‌های تغییرکرده، منطق اضافه‌شده و رفتارهای حفظشده را مشخص کن.

فعلاً هیچ فایل دیگری را تغییر نده.

در این مرحله به پرسش‌های زیر پاسخ دهید:

- آیا فقط فایل‌های مجاز تغییر کرده‌اند؟
- آیا فایل یا تغییر نامرتبطی ایجاد شده است؟
- آیا منطق قبلی ناخواسته تغییر کرده است؟
- آیا پیام‌های خطا و قالب پاسخ حفظ شده‌اند؟
- آیا رفتارهای مرزی پوشش داده شده‌اند؟

- آیا فایل حساس یا اطلاعات محرمانه وارد diff نشده است؟

git status مشخص می‌کند چه چیزهایی تغییر کرده‌اند و git diff نشان می‌دهد دقیقاً چه خطوطی تغییر کرده‌اند. هر دو لازم‌اند؛ یکی دامنهٔ تغییر را نشان می‌دهد و دیگری محتوای تغییر را.

۴.۶. اجرای تست‌ها

پس از بررسی اولیه، تست مرتبط را اجرا کنید. فرمان واقعی را از مستندات پروژه یا تنظیمات آن پیدا کنید. نمونه‌ها:

```
bash
python -m pytest
bash
npm test
 bash
flutter test
```

درخواست پیشنهادی به Claude Code:

تست‌های مستقیم قابلیت تغییرکرده را اجرا کن.

اگر تستی شکست خورد:

-خروجی کامل را تحلیل کن؛

-علت احتمالی را مشخص کن؛

-فعلاً کد را تغییر نده؛

-راهکارهای پیشنهادی را گزارش بده.

اگر تست شکست خورد، ابتدا علت را بفهمید. شکست تست می‌تواند ناشی از موارد مختلفی باشد:

- خطا در کد جدید؛
- تست قدیمی یا ناهماهنگ با رفتار مورد انتظار؛
- محیط ناقص؛
- وابستگی نصب‌نشده؛
- تنظیمات نادرست؛
- تعارض رفتار جدید با قرارداد قبلی.

اصلاح کد صرفاً برای سبز شدن تست، بدون فهم علت شکست، قابل‌اعتماد نیست. ممکن است با چنین کاری تست موفق شود اما مشکل اصلی پنهان بماند.

پس از مشخص شدن علت، اصلاح را در دامنه‌ای روشن درخواست کنید:

علت شکست تست، نبود اعتبارسنجی برای مقدار خالی است.

فقط همین مشکل را در handler ورود اصلاح کن.

تست شکست‌خورده و تست‌های مرتبط را دوباره اجرا کن.

فایل‌های خارج از محدوده را تغییر نده.

۴.۷. بازبینی تغییرات با /review

پس از اجرای تست، تغییرات باید جداگانه بازبینی شوند. اگر محیط شما دستور /review دارد، می‌توانید از آن استفاده کنید:

/review

یا از معادل متنی آن استفاده کنید:

تغییرات فعلی را مانند یک بازبین کد بررسی کن.

فعلاً هیچ کدی را تغییر نده.

گزارش را بر اساس شدت مشکل ارائه بده و این موارد را بررسی کن:

-باگ منطقی؛

-شکستن رفتار قبلی؛

-نبود یا ناکافی بودن تست؛

-مشکلات امنیتی؛

-تغییرات خارج از محدوده.

بازبینی باید مستقل از توضیح اولیهٔ عامل انجام شود. هدف این نیست که Claude Code تغییرات خودش را توجیه کند؛ هدف، پیدا کردن اشکال‌هایی است که ممکن است در اجرای اولیه دیده نشده باشند.

گزارش بازبینی باید دست‌کم شامل این موارد باشد:

- شرح مشکل؛
- محل مشکل؛
- شدت یا اثر احتمالی؛
- دلیل اینکه چرا مشکل مهم است؛
- پیشنهاد اصلاح؛
- وضعیت تست مربوط.

نمونهٔ قالب:
یافته ۱
شدت: بالا -

```
- محل: src/auth/login.py
- مشکل: ورودی null قبل از اعتبارسنجی استفاده می‌شود.
- اثر: احتمال خطای زمان اجرا.
- پیشنهاد: اعتبارسنجی ورودی پیش از دسترسی به فیلدها.
- تست مرتبط: test_login_with_missing_payload
```

درخواست بازبینی باید صریحاً بگوید که فعلاً کد تغییر نکند. اصلاح باید پس از بررسی و با تأیید دامنهٔ آن انجام شود.

۴.۸. تصمیم انسانی برای commit

commit پایان خودکار کار نیست؛ یک تصمیم انسانی دربارهٔ ثبت وضعیت فعلی پروژه است.

پیش از commit، این موارد را بررسی کنید:

- هدف اولیه برآورده شده است؛
- تغییرات در محدودهٔ تعیین‌شده باقی مانده‌اند؛
- تست‌های مرتبط موفق شده‌اند؛
- git diff بررسی شده است؛
- اطلاعات حساس در تغییرات وجود ندارد؛
- بازبینی مشکل مهمی پیدا نکرده است؛
- فایل‌های ناخواسته وارد commit نشده‌اند.

در صورت تأیید، commit را دستی اجرا کنید:

```
 bash
git add path/to/file
git commit -m "Fix login validation"
```

در این چرخه، Claude Code نباید بدون تأیید صریح شما عملیات زیر را انجام دهد:

git commit

git push

git merge

git rebase

اگر تست یا بازبینی مشکلی نشان داد:

1. یافتهٔ دقیق را مشخص کنید؛

2. فقط فایل‌های مرتبط با همان مشکل را تغییر دهید؛
3. تست مربوط را دوباره اجرا کنید؛
4. status و diff را دوباره بررسی کنید؛
5. سپس دربارهٔ commit تصمیم بگیرید.

۴.۹. الگوی درخواست کامل

الگوی زیر یک درخواست جامع برای اجرای چرخهٔ کاری است:

ابتدا وضعیت پروژه و شاخهٔ جاری را بررسی کن.

تغییرات موجود را دست‌کاری یا بازگردانی نکن.

هدف:

[هدف دقیق کار را بنویس]

محدودهٔ مجاز:

- [فایل یا ماژول اول]
- [فایل یا ماژول دوم]

خارج از محدوده:

- [بخش‌هایی که نباید تغییر کنند]

معیار پذیرش:

- [معیار اول]
- [معیار دوم]
- [معیار سوم]

ابتدا:

۱. محل پیاده‌سازی، فایل‌های مرتبط و تست‌های موجود را بررسی کن.

۲ .یک برنامهٔ کوتاه و مرحله‌به‌مرحله ارائه بده.

۳ .فعلاً هیچ فایلی را تغییر نده.

پس از تأیید من:

۱ .تغییر را مرحله‌ای و محدود اجرا کن.

۲ .تست‌های مرتبط را اجرا کن.

۳ .فایل‌های تغییرکرده و خلاصهٔ diff را گزارش بده.

۴ .خطاها و محدودیت‌های باقی‌مانده را اعلام کن.

۵ .یک بازبینی مستقل انجام بده، اما کد را در مرحلهٔ بازبینی تغییر نده.

محدودیت‌های نهایی:

commit -انجام نده.

push -انجام نده.

-فایل خارج از محدوده را تغییر نده.

-اطلاعات حساس را نخوان یا گزارش نکن.

این الگو، هدف، محدوده، معیار پذیرش، برنامه، اجرا، تست و گزارش نهایی را در یک درخواست روشن می‌کند.

۴.۱۰. چک‌لیست نهایی فصل

پیش از پایان کار، این فهرست را بررسی کنید:

- [ ] وضعیت اولیهٔ Git بررسی شده است.
- [ ] شاخهٔ فعال مشخص است.
- [ ] تغییرات قبلی کاربر حفظ شده‌اند.
- [ ] هدف به‌صورت دقیق تعریف شده است.
- [ ] معیار پذیرش مشخص است.
- [ ] محدوده و خارج از محدوده تعیین شده‌اند.
- [ ] برنامه پیش از اجرا بررسی شده است.
- [ ] تغییرات مرحله‌ای و محدود انجام شده‌اند.

- [ ] تست‌های مرتبط اجرا شده‌اند.
- [ ] علت شکست تست‌ها بررسی شده است.
- [ ] git status بررسی شده است.
- [ ] git diff بررسی شده است.
- [ ] فایل‌های تغییرکرده با محدودهٔ مجاز مطابقت دارند.
- [ ] بازبینی مستقل انجام شده است.
- [ ] اطلاعات حساس در diff وجود ندارد.
- [ ] تصمیم دربارهٔ commit توسط انسان گرفته شده است.
- [ ] بدون تأیید، push، merge یا rebase انجام نشده است.

جمع‌بندی

چرخهٔ پیشنهادی کار با Claude Code چنین است:

شناخت بدون تغییر
→تعریف هدف و معیار پذیرش
→برنامه‌ریزی
→اجرای محدود
→تست و تحلیل شکست
→بررسی status و diff
→بازبینی بدون تغییر کد
→تصمیم انسانی برایcommit

این روش برای کارهای کوچک، متوسط و پیچیده قابل استفاده است. در کارهای کوچک، برنامه و بازبینی کوتاه‌تر می‌شوند؛ اما شناخت وضعیت، تست، بررسی تغییرات و تصمیم نهایی همچنان باید حفظ شوند

فصل پنجم: راهنمای دستورهای اسلش و CLI در Claude Code

در Claude Code معمولاً با دو نوع دستور سروکار دارید:

1. **دستورهای اسلش** که داخل نشست Claude Code نوشته می‌شوند.
2. **فرمان‌های CLI و ترمینال** که مستقیماً در Shell اجرا می‌شوند.

شناخت تفاوت این دو، از اجرای فرمان اشتباه و برداشت نادرست از وضعیت پروژه جلوگیری می‌کند.

۵.۱. تفاوت دستور اسلش و فرمان CLI

دستورهای اسلش

دستورهای اسلش معمولاً با / شروع می‌شوند و داخل محیط گفت‌وگو یا نشست Claude Code اجرا می‌شوند:

/plan

/review

/status

این دستورها برای کنترل نشست، برنامه‌ریزی، بازبینی یا دریافت اطلاعاتی از محیط ابزار استفاده می‌شوند.

فرمان‌های CLI

فرمان‌های CLI در ترمینال اجرا می‌شوند و مستقیماً با سیستم‌عامل، Git یا ابزارهای پروژه ارتباط دارند:

```
bash
git status
git diff
pytest
npm test
```

مقایسهٔ عملی

نوع دستور	محل اجرا	نمونه	کاربرد
دستور اسلش	داخل نشست Claude Code	/plan	برنامه‌ریزی کار
فرمان Git	ترمینال	git status	بررسی وضعیت پروژه
فرمان تست	ترمینال	pytest	اجرای تست
فرمان جست‌وجو	ترمینال	rg "login"	جست‌وجوی متن

اصل مهم این است:

هر دستوری که با / شروع می‌شود، الزاماً فرمان سیستم‌عامل یا Git نیست.

برای بررسی قطعی وضعیت پروژه، همچنان از فرمان‌های واقعی Git استفاده کنید:

```
bash
git status
git diff
```

۵.۲. دستورهای هدف و برنامه‌ریزی

تعریف هدف

پیش از شروع تغییر، هدف باید دقیق و قابل‌آزمایش باشد.

درخواست ضعیف:

این بخش را بهتر کن.

این درخواست مشخص نمی‌کند:

- چه چیزی باید تغییر کند؛
- کدام فایل‌ها مجاز به تغییر هستند؛
- رفتار قبلی چه بوده است؛
- موفقیت چگونه سنجیده می‌شود.

درخواست بهتر:

اعتبارسنجی ورودی فرم ثبت شماره تلفن را اصلاح کن.

فقط فایل فرم و تست‌های مرتبط را تغییر بده.

فرمت فعلی پیام‌های خطا را حفظ کن.

برای ورودی خالی و شمارهٔ نامعتبر تست اضافه کن.

یک هدف مناسب باید شامل این موارد باشد:

- نتیجهٔ مورد انتظار؛
- فایل‌ها یا مسیرهای مجاز؛
- بخش‌های خارج از محدوده؛
- رفتارهایی که باید حفظ شوند؛
- معیار پذیرش؛
- تست‌های موردنیاز؛
- محدودیت‌های اجرایی.

bitwithit

/goal

در بعضی محیط‌ها ممکن است دستوری مانند /goal برای تعریف یا متمرکزکردن هدف وجود داشته باشد:

/goal

اما در صورت پشتیبانی‌نشدن، همان هدف را به‌صورت متنی بنویسید:

هدف این کار اصلاح اعتبارسنجی ورودی endpoint ورود است.

فقط handler ورود و تست‌های مستقیم آن مجاز به تغییر هستند.

سیستمsession ، مدل کاربر و migration نباید تغییر کنند.

دستور یا روش مورد استفاده مهم نیست؛ مهم این است که هدف، محدوده و معیار موفقیت مبهم نباشند.

/plan

/plan برای دریافت یک برنامهٔ مرحله‌به‌مرحله استفاده می‌شود:

/plan

یا:

برای افزودن قابلیت جست‌وجوی کاربر برنامه‌ریزی کن.

فایل‌های احتمالی، وابستگی‌ها، ریسک‌ها و تست‌های لازم را مشخص کن.

فعلاً هیچ فایلی را تغییر نده.

یک برنامهٔ خوب باید مشخص کند:

- هدف کلی چیست؛
- کدام فایل‌ها احتمالاً درگیر هستند؛
- مراحل اجرا چه ترتیبی دارند؛
- میان مراحل چه وابستگی‌هایی وجود دارد؛
- چه تست‌هایی باید اجرا شوند؛
- چه ریسک‌هایی وجود دارد؛
- برای کدام تصمیم‌ها تأیید کاربر لازم است.

نمونهٔ برنامه:

bitwitit

۱ .بررسی محل پیاده‌سازی جست‌وجوی کاربر.

۲ .بررسیroute ، serviceو تست‌های موجود.

۳ .تعیین حداقل فایل‌های لازم برای تغییر.

۴ .پیاده‌سازی منطق جست‌وجو.

۵ .افزودن تست برای نتیجهٔ خالی و ورودی نامعتبر.

۶ .اجرای تست‌های مستقیم.

۷ .بررسی git diff و وضعیت فایل‌های تغییرکرده.

در مرحلهٔ برنامه‌ریزی، صریحاً اعلام کنید که فایل‌ها نباید تغییر کنند:

فقط پروژه را بررسی کن و برنامه ارائه بده.

تا زمان تأیید من هیچ فایلی را ایجاد، حذف یا ویرایش نکن.

۵.۳. دستورهای وضعیت و بازبینی

/status

در برخی محیط‌ها /status ممکن است وضعیت نشست یا عملیات جاری را نمایش دهد:

/status

اما برای وضعیت قطعی Git از این فرمان استفاده کنید:

```
bash
git status
```

برای خروجی کوتاه‌تر:

```
bash
git status --short
```

نمونهٔ خروجی:

M src/auth/login.py

?? tests/test_login.py

/diff

در بعضی محیط‌ها /diff ممکن است برای نمایش یا تمرکز روی تغییرات استفاده شود:

/diff

اما بررسی قابل‌اعتماد تغییرات Git با فرمان زیر انجام می‌شود:

```
bash
git diff
```

برای دیدن خلاصهٔ تغییرات:

```
bash
git diff --stat
```

برای بررسی یک فایل مشخص:

```
bash
git diff -- src/auth/login.py
```

/review

اگر محیط Claude Code شما از /review پشتیبانی کند، می‌توانید از آن برای بازبینی استفاده کنید:

/review

درخواست کامل‌تر:

تغییرات فعلی را مانند یک بازبین کد بررسی کن.

فعلاً هیچ کدی را تغییر نده.

این موارد را بررسی کن:

-باگ منطقی

-شکستن رفتار قبلی

-مشکلات امنیتی

-ناکافی‌بودن تست‌ها

-تغییرات خارج از محدوده

-ناقص‌بودن پیاده‌سازی

اگر /review در دسترس نبود، از دستور متنی استفاده کنید:

تغییرات فعلی را از نظر باگ منطقی، رفتارهای مرزی،

امنیت، تست‌ها و تغییرات خارج از محدوده بررسی کن.

bitwiti

فقط گزارش بده و کد را تغییر نده.

گزارش بازبینی بهتر است شامل این موارد باشد:

یافته ۱

-شدت: بالا

-محل: src/auth/login.py :

-مشکل: ورودی null پیش از اعتبارسنجی استفاده می‌شود.

-اثر: احتمال خطای زمان اجرا.

-پیشنهاد: اعتبارسنجی ورودی پیش از دسترسی به فیلدها.

-تست مرتبط: test_login_with_missing_payload :

توجه کنید که بازبینی با اصلاح کد تفاوت دارد. ابتدا باید مشکل گزارش شود، سپس دربارهٔ اصلاح آن تصمیم بگیرید.

۵.۴. مدل و سطح استدلال

در بعضی محیط‌ها امکان انتخاب مدل یا سطح تلاش استدلالی وجود دارد. انتخاب مناسب به عوامل زیر بستگی دارد:

- پیچیدگی مسئله؛
- مقدار زمینهٔ موردنیاز؛
- حساسیت پروژه؛
- اهمیت دقت؛
- زمان قابل‌قبول؛
- هزینه یا سهمیهٔ استفاده؛
- نیاز به تحلیل چندمرحله‌ای.

سطح تلاش استدلالی

مفهوم Reasoning Effort میزان منابعی را توصیف می‌کند که مدل برای تحلیل مسئله به کار می‌گیرد. سطح‌های رایج ممکن است چنین باشند:

Low

Medium

High

کارهای ساده مانند تغییر متن یا اصلاح یک شرط معمولاً به تحلیل سنگین نیاز ندارند. در مقابل، تغییر در احراز هویت، migration، معماری یا منطق مالی به بررسی عمیق‌تر نیاز دارد.

نباید وجود یک فرمان یا گزینهٔ ثابت برای تغییر این سطح را در همهٔ محیط‌ها فرض کنید. نام گزینه، محل تنظیم و رفتار آن به نسخه و پیکربندی محیط بستگی دارد.

درخواست مناسب:

پیش از اجرا، پیچیدگی این تغییر و ریسک‌های آن را ارزیابی کن.

اگر به تحلیل عمیق‌تر، بررسی فایل‌های بیشتر یا تست‌های اضافی نیاز است، اعلام کن.

۵.۵. محیط اجرا و سطح دسترسی

Claude Code ممکن است در محیطی با سطح دسترسی محدود یا گسترده اجرا شود. نمونه‌های رایج عبارت‌اند از:

- محیط محلی؛
- محیط ایزوله؛
- محیط ابری؛
- حالت فقط‌خواندنی؛
- محیط دارای اجازهٔ تغییر فایل؛
- محیط نیازمند تأیید برای اجرای فرمان؛
- محیط دارای worktree جداگانه.

انتخاب محیط باید با نوع کار هماهنگ باشد.

حالت فقط‌خواندنی

برای شناخت پروژه:

فقط فایل‌ها را بررسی کن.

هیچ فایلی را تغییر نده.

هیچ فرمانی که داده‌ها یا تنظیمات را تغییر می‌دهد اجرا نکن.

تغییر محدود

فقط این فایل‌ها مجاز به تغییر هستند:

- src/auth/login.py

- tests/test_login.py

فایل‌های خارج از این فهرست را تغییر نده.

bitwithit

بااین‌حال، نوشتن این محدودیت در متن به‌تنهایی یک مرز امنیتی واقعی نیست. دسترسی واقعی باید از طریق تنظیمات محیط، مجوزها و سازوکار تأیید کنترل شود.

همچنین باید تفاوت زیر را در نظر بگیرید:

امکان اجرا ≠ مجوز اجرا

ممکن است Claude Code بتواند فرمانی را اجرا کند، اما اجرای آن برای کار فعلی مجاز نباشد؛ مانند:

```
bash
git push
rm -rf ...
npm publish
```

۵.۶. مدیریت نشست و زمینه

در پروژه‌های طولانی، نشست ممکن است بزرگ شود و بخشی از زمینه فشرده یا خلاصه شود. پس از فشرده‌سازی زمینه، اطلاعات مهم را دوباره کنترل کنید:

برای اطمینان، وضعیت فعلی کار را خلاصه کن:

-هدف اصلی

-فایل‌های مجاز

-فایل‌های ممنوع

-تغییرات انجام‌شده

-تست‌های اجراشده

-خطاهای باقی‌مانده

-معیارهای پذیرش

-وضعیتcommit

اگر پاسخ Claude Code با وضعیت واقعی پروژه مطابقت ندارد، به خروجی Git و فایل‌های پروژه برگردید:

```
bash
git status
git diff
```

شروع نشست جدید

در برخی محیطها ممکن است دستور /new برای شروع نشست جدید وجود داشته باشد:

/new

شروع نشست جدید زمانی مناسب است که:

- هدف پروژه تغییر کرده است؛

- گفت‌وگو با فرض‌های قدیمی ترکیب شده است؛
- زمینهٔ قبلی بیش از حد طولانی شده است؛
- کار جدید باید کاملاً از کار قبلی جدا باشد.

پیش از شروع نشست جدید، خلاصه‌ای مانند زیر ثبت کنید:

پروژه:

هدف انجام‌شده:

فایل‌های تغییرکرده:

تست‌های اجراشده:

خطاهای باقی‌مانده:

وضعیت:commit

پس از شروع نشست جدید، این خلاصه را به‌عنوان زمینهٔ کنترل‌شده ارائه دهید و وضعیت واقعی Git را نیز دوباره بررسی کنید.

۵.۷. ابزارها، فایل‌های راهنما و منابع زمینه

Claude Code ممکن است پاسخ خود را بر اساس چند منبع تولید کند:

- متن گفت‌وگو؛
- فایل‌های پروژه؛
- مستندات محلی؛
- فایل راهنمای پروژه؛
- خروجی فرمان‌ها؛
- نتایج تست؛
- ابزارهای متصل؛
- خلاصهٔ نشست قبلی.

برای هر نتیجهٔ مهم، باید مشخص باشد اطلاعات از کجا آمده است.

درخواست مناسب:

برای هر نتیجه، منبع آن را مشخص کن.

اگر نتیجه بر اساس کد، تست، مستندات یا حدس است، این تفاوت را صریحاً اعلام کن.

در موارد نامشخص حدس نزن.

فایل راهنمای پروژه

فایل راهنما می‌تواند شامل این موارد باشد:

- ساختار پروژه؛
- فرمان‌های نصب و اجرا؛
- فرمان‌های تست؛
- قواعد سبک کدنویسی؛
- محدودیت‌های تغییر؛
- سیاست‌های امنیتی؛
- روش گزارش خطا.

برای Claude Code ممکن است پروژه از فایلی مانند CLAUDE.md استفاده کند. نام فایل و روش بارگذاری آن را باید بر اساس تنظیمات واقعی پروژه بررسی کرد، نه صرفاً بر اساس نامی که در یک ابزار دیگر استفاده می‌شود.

اصل کلی این است:

راهنمای پروژه باید رفتار مورد انتظار را توضیح دهد، اما نباید جایگزین کنترل دسترسی، تست یا بازبینی انسانی شود.

۵.۸. فرمان‌های مهم CLI

وضعیت Git

```
bash
git status
```

نمایش کامل وضعیت فایل‌ها و تغییرات.

```
bash
git status --short
```

نمایش خلاصهٔ تغییرات.

```
bash
git branch --show-current
```

نمایش شاخهٔ فعال.

```
bash
git branch
```

نمایش شاخه‌های موجود.

بررسی تغییرات

```
bash
git diff
```

نمایش تغییرات ثبت‌نشده در فایل‌های tracked.

```
bash
git diff --stat
```

نمایش خلاصهٔ اندازهٔ تغییرات.

```
bash
git diff -- path/to/file
```

نمایش تغییرات یک فایل مشخص.

برای بررسی تغییرات staged:

```
bash
git diff --cached
```

تاریخچهٔ Git

```
bash
git log --oneline -n 10
```

نمایش ۱۰ commit اخیر به‌صورت خلاصه.

```
bash
git show HEAD
```

نمایش جزئیات آخرین commit.

شناسایی ریشهٔ پروژه

```
bash
git rev-parse --show-toplevel
```

نمایش ریشهٔ ریپوی Git.

این فرمان هنگام کار در زیرپوشه‌ها بسیار مفید است و مشخص می‌کند پروژه از کدام مسیر مدیریت می‌شود.

جست‌وجوی متن با rg

اگر ripgrep نصب باشد:

```
bash
rg "login"
```

جست‌وجو در کل مسیر فعلی.

```
bash
rg "login" src tests
```

جست‌وجو فقط در مسیرهای مشخص.

```
bash
rg "TODO|FIXME" .
```

جست‌وجوی چند الگو.

```
bash
rg -n "authenticate" src/
```

نمایش شمارهٔ خط در نتایج.

rg به Claude Code یا ابزار خاصی اختصاص ندارد و یک ابزار عمومی خط فرمان است.

بررسی نسخهٔ ابزارها

```
bash
python --version
node --version
git –version
```

در پروژه‌های Flutter:

```
bash
flutter --version
```

در پروژه‌های Java:

```
bash
java --version
```

فرمان‌های موجود بودن ابزار به‌معنای مناسب‌بودن آن برای پروژه نیستند. فایل‌های تنظیمات و مستندات پروژه باید مشخص کنند که کدام نسخه و فرمان موردنیاز است.

اجرای تست‌ها

نمونه‌های رایج:

```
bash
pytest
bash
python -m pytest
bash
npm test
bash
flutter test
```

این فرمان‌ها عمومی هستند، اما همهٔ پروژه‌ها از آن‌ها استفاده نمی‌کنند. ابتدا فایل‌هایی مانند این موارد را بررسی کنید:

README.md

pyproject.toml

package.json

Makefile

justfile

pubspec.yaml

از Claude Code بخواهید فرمان واقعی پروژه را پیدا کند:

فرمان‌های واقعی تست، lintو type-check را از فایل‌های تنظیمات پروژه پیدا کن.

فرمان‌ها را اجرا نکن؛ ابتدا فقط آن‌ها را گزارش بده.

۵.۹. دستورهای وابسته به محیط در برابر فرمان‌های عمومی

وابسته به محیط Claude Code

موارد زیر ممکن است در یک محیط وجود داشته باشند، اما نباید بدون بررسی وجودشان را قطعی فرض کرد:

/goal

/plan

/review

/diff

/status

/new

همچنین این قابلیت‌ها ممکن است به نسخه یا محیط اجرا وابسته باشند:

- انتخاب مدل از طریق دستور؛
- تغییر سطح تلاش استدلالی؛
- فشرده‌سازی زمینه؛
- worktree؛
- حالت فقط‌خواندنی؛
- محیط ابری یا ایزوله؛
- ابزارهای متصل؛
- حافظه یا زمینهٔ ذخیره‌شده.

عمومی و قابل‌انتقال

فرمان‌های زیر متعلق به Git یا ابزارهای عمومی هستند:

```
bash
git status
git branch
git diff
git log
git show
git rev-parse
rg
pytest
python -m pytest
npm test
flutter test
python --version
node --version
git --version
```

البته عمومی‌بودن فرمان به معنی مناسب‌بودن آن برای همهٔ پروژه‌ها نیست. برای مثال، اجرای pytest در پروژه‌ای که با Jest یا Flutter ساخته شده، انتخاب درستی نیست.

۵.۱۰. الگوی ترکیبی استفاده از اسلش و CLI

یک گردش‌کار عملی می‌تواند چنین باشد:

مرحلهٔ اول: وضعیت پروژه

در ترمینال:

```
bash
```

```
git status
git branch --show-current
```

مرحلهٔ دوم: برنامه‌ریزی در Claude Code

برای اصلاح اعتبارسنجی ورودی login برنامه بده.

فایل‌های درگیر، تست‌های موجود، ریسک‌ها و معیار پذیرش را مشخص کن.

فعلاً هیچ فایلی را تغییر نده.

یا در صورت پشتیبانی:

/plan

مرحلهٔ سوم: اجرای تغییر

برنامه تأیید شد.

فقط handler ورود و تست‌های مستقیم آن را تغییر بده.

ساختار پاسخ‌های فعلی API را حفظ کن.

مرحلهٔ چهارم: بررسی در ترمینال

```
 bash
git status --short
git diff --stat
git diff
```

مرحلهٔ پنجم: اجرای تست

```
bash
python -m pytest tests/test_login.py
```

مرحلهٔ ششم: بازبینی

تغییرات فعلی را از نظر باگ منطقی، امنیت،

رفتارهای مرزی، تست‌ها و تغییرات خارج از محدوده بررسی کن.

کد را تغییر نده و فقط گزارش بده.

مرحلهٔ هفتم: تصمیم نهایی

پس از تأیید انسانی:

```
 bash
git add src/auth/login.py tests/test_login.py
```

```
git commit -m "Fix login validation"
```

در این الگو، Claude Code برای تحلیل، برنامه‌ریزی و کمک به بازبینی استفاده می‌شود و CLI برای کنترل مستقیم وضعیت پروژه، اجرای تست و ثبت تغییرات.

هشدارهای مهم

- دستورهای اسلش را با فرمان‌های Shell یا Git اشتباه نگیرید.
- برای وضعیت قطعی Git از git status استفاده کنید.
- برای مشاهدهٔ تغییرات واقعی از git diff استفاده کنید.
- در برنامه‌ریزی، صریحاً اعلام کنید که فایل‌ها نباید تغییر کنند.
- هدف‌های مبهم، معیار پذیرش قابل‌اعتماد تولید نمی‌کنند.
- وجود یک دستور اسلش به نسخه و محیط Claude Code بستگی دارد.
- مدل و سطح استدلال را بدون بررسی تنظیمات واقعی قطعی فرض نکنید.
- فرمان‌های تست را از پروژه شناسایی کنید و صرفاً نمونه‌ها را کپی نکنید.
- حالت دسترسی و اجرای فرمان را متناسب با ریسک کار انتخاب کنید.
- پس از فشرده‌سازی زمینه، هدف و محدودیت‌ها را دوباره کنترل کنید.
- /review جایگزین بررسی دستی diff و اجرای تست نیست.
- فایل راهنما جایگزین کنترل دسترسی و تأیید انسانی نیست.
- قبل از commit، push، merge یا rebase تأیید صریح دریافت کنید.

چک‌لیست فصل پنجم

- [ ] تفاوت دستور اسلش و فرمان CLI را می‌دانم.
- [ ] می‌دانم /plan باید بدون تغییر فایل اجرا شود.
- [ ] هدف، محدوده و معیار پذیرش را مشخص کرده‌ام.
- [ ] برای بررسی وضعیت از git status استفاده کرده‌ام.
- [ ] برای بررسی تغییرات از git diff استفاده کرده‌ام.
- [ ] فرمان واقعی تست پروژه را پیدا کرده‌ام.
- [ ] دستورهای وابسته به محیط را با فرمان‌های عمومی اشتباه نگرفته‌ام.
- [ ] سطح دسترسی محیط اجرا را بررسی کرده‌ام.
- [ ] پس از فشرده‌سازی زمینه، هدف و محدودیت‌ها را دوباره کنترل کرده‌ام.

- [ ] بازبینی را بدون تغییر کد انجام داده‌ام.
- [ ] فایل‌های تغییرکرده در محدودهٔ مجاز هستند.
- [ ] پیش از عملیات حساس، تأیید انسانی گرفته‌ام.

جمع‌بندی

برای استفادهٔ حرفه‌ای از Claude Code باید سه لایه را از هم جدا نگه دارید:

دستورهای نشست

←برنامه‌ریزی و کنترل همکاری با عامل

فرمان‌هایCLI

←بررسی پروژه، Git، جست‌وجو و اجرای تست

تصمیم انسانی

←تأیید تغییر، commitو عملیات حساس

دستورهای اسلش، ابزار کنترل نشست هستند؛ فرمان‌های CLI، ابزار کار مستقیم با پروژه؛ و بازبینی انسانی، مرحله‌ای است که صحت و محدودهٔ نتیجه را تأیید می‌کند.

bitwitit

فصل ششم: کنترل دسترسی و امنیت در Claude Code

Claude Code برای بررسی فایل‌ها، اجرای فرمان‌ها و اعمال تغییرات در پروژه به دسترسی‌های مختلف نیاز دارد. این دسترسی‌ها اگر بدون کنترل مناسب فعال باشند، می‌توانند باعث تغییر ناخواستهٔ فایل‌ها، افشای اطلاعات حساس، اجرای فرمان‌های مخرب یا ارسال داده به سرویس‌های بیرونی شوند.

هدف این فصل، ایجاد یک روش عملی برای استفادهٔ ایمن از Claude Code است؛ به‌گونه‌ای که ابزار بتواند کارهای لازم را انجام دهد، اما دامنهٔ اختیار آن از نیاز واقعی پروژه بیشتر نشود.

۶.۱. اصل حداقل دسترسی

مهم‌ترین اصل امنیتی در کار با Claude Code این است:

عامل باید فقط به منابع و عملیاتی دسترسی داشته باشد که برای انجام وظیفهٔ فعلی لازم هستند.

اگر هدف فقط بررسی ساختار پروژه است، دسترسی نوشتن لازم نیست.

اگر هدف اصلاح یک فایل است، دسترسی به کل پروژه یا اجرای فرمان‌های مدیریتی ضرورتی ندارد.

اگر هدف اجرای تست است، دسترسی به اینترنت، secrets یا سرویس‌های تولیدی معمولاً لازم نیست.

نمونهٔ نامناسب

کل پروژه را بررسی کن، هر فایلی لازم بود تغییر بده و تست‌ها را اجرا کن.

این درخواست چند ابهام جدی دارد:

- محدودهٔ تغییر مشخص نیست؛
- فایل‌های مجاز تعیین نشده‌اند؛
- فرمان‌های قابل‌اجرا محدود نشده‌اند؛
- عملیات حساس از عملیات عادی جدا نشده‌اند؛
- وضعیت فعلی پروژه مشخص نیست.

نمونهٔ کنترل‌شده

فقط این فایل‌ها مجاز به تغییر هستند:

- src/auth/login.py

- tests/test_login.py

bitwiit

ابتدا فقط فایل‌ها و تست‌های مرتبط را بررسی کن.

تا زمان تأیید من هیچ فایلی را تغییر نده.

فرمان‌های حذف، commit، push، mergeو دسترسی به سرویس بیرونی اجرا نکن.

۶.۲. انواع دسترسی

دسترسی Claude Code فقط به «خواندن» یا «نوشتن» محدود نمی‌شود. بهتر است آن را در چند سطح بررسی کنید.

دسترسی به فایل‌ها

- خواندن فایل؛
- ایجاد فایل جدید؛
- ویرایش فایل؛
- حذف فایل؛
- تغییر نام یا جابه‌جایی فایل؛
- دسترسی به مسیرهای خارج از ریشهٔ پروژه.

دسترسی به فرمان‌ها

- اجرای فرمان‌های فقط‌خواندنی؛
- اجرای تست؛
- نصب وابستگی؛
- تغییر تنظیمات محیط؛
- اجرای اسکریپت‌های پروژه؛
- اجرای فرمان‌های مدیریتی یا مخرب.

دسترسی به Git

- مشاهدهٔ وضعیت و diff؛
- ایجاد branch؛
- stageکردن تغییرات؛
- commit؛
- rebase یا merge؛
- push به مخزنِ راه دور؛

bitwithit

- تغییر تنظیمات Git.

دسترسی شبکه

- اتصال به API؛
- دانلود وابستگی؛
- ارسال داده به سرویس بیرونی؛
- دسترسی به سرویس‌های داخلی؛
- برقراری اتصال با محیط production.

دسترسی به اطلاعات حساس

- کلیدهای API؛
- رمزهای عبور؛
- توکن‌های دسترسی؛
- فایل‌های .env؛
- کلیدهای SSH؛
- اطلاعات کاربران؛
- داده‌های پزشکی یا مالی؛
- credentialهای سرویس‌های ابری.

هرکدام از این موارد باید جداگانه کنترل شوند. دسترسی به فایل‌های پروژه نباید به‌صورت خودکار به معنای دسترسی به اطلاعات محرمانه تلقی شود.

۶.۳. حالت فقط‌خواندنی

در مرحلهٔ شناخت پروژه، بهترین انتخاب معمولاً حالت فقط‌خواندنی است.

در این حالت عامل می‌تواند:

- ساختار پوشه‌ها را بررسی کند؛
- فایل‌ها را بخواند؛
- وابستگی‌ها را شناسایی کند؛
- تست‌ها را پیدا کند؛
- وضعیت Git را مشاهده کند؛
- برنامهٔ پیشنهادی ارائه دهد.

bitwithit

اما نباید:

- فایل ایجاد کند؛
- فایل ویرایش کند؛
- فایل حذف کند؛
- تنظیمات را تغییر دهد؛
- commit یا push انجام دهد؛
- داده‌ای را به بیرون ارسال کند.

درخواست پیشنهادی:

فقط پروژه را بررسی کن و گزارشی از ساختار، فایل‌های مرتبط،

وابستگی‌ها و تست‌های موجود ارائه بده.

هیچ فایلی را تغییر نده و هیچ فرمان تغییردهنده‌ای اجرا نکن.

چرا حالت فقط‌خواندنی مهم است؟

چون مرحلهٔ شناخت هنوز مرحلهٔ تصمیم‌گیری است. اگر عامل قبل از کامل‌شدن شناخت شروع به ویرایش کند، ممکن است:

- فرض نادرست را وارد کد کند؛
- فایل نامرتبطی را تغییر دهد؛
- رفتار موجود را بدون ثبت وضعیت اولیه از بین ببرد؛
- بررسی تغییرات را دشوار کند؛
- دامنهٔ کار را گسترش دهد.

۶.۴. تأیید انسانی

تأیید انسانی باید پیش از عملیات دارای اثر پایدار یا دامنهٔ گسترده انجام شود.

عملیات حساس شامل موارد زیر است:

```
bash
```

```
git commit
git push
git merge
git rebase
rm
```

همچنین این موارد به تأیید نیاز دارند:

- حذف یا جابه‌جایی فایل؛
- تغییر migration؛
- تغییر schema پایگاه داده؛
- تغییر مجوزهای دسترسی؛
- تغییر فایل‌های CI/CD؛
- نصب وابستگی جدید؛
- اجرای اسکریپت ناشناخته؛
- تغییر تنظیمات production؛
- ارسال داده به سرویس بیرونی.

الگوی صحیح تأیید

ابتدا از Claude Code بخواهید عملیات را توضیح دهد:

فرمان‌هایی را که قصد داری اجرا کنی، یکی‌یکی فهرست کن.

برای هر فرمان هدف، اثر جانبی و ریسک آن را توضیح بده.

فعلاً هیچ‌کدام را اجرا نکن.

سپس فقط عملیات لازم را تأیید کنید:

اجرای تست‌های محلی را تأیید می‌کنم.

commitو push ممنوع است.

هیچ فایل دیگری را تغییر نده.

تأیید مبهم مناسب نیست:

هر کاری لازم است انجام بده.

این جمله دامنهٔ اختیار عامل را مشخص نمی‌کند و ممکن است شامل عملیات ناخواسته باشد.

bitwitit

۶.۵. تفاوت پیش‌نمایش و اجرا

پیش از اجرای یک عملیات تغییردهنده، از حالت پیش‌نمایش استفاده کنید.

برای مثال، قبل از حذف یا تغییر گسترده:

ابتدا فقط فهرست فایل‌هایی را که قرار است تغییر کنند ارائه بده.

هیچ تغییری اعمال نکن.

برای Git:

```
bash
git status --short
git diff --stat
git diff
```

برای بررسی فرمان‌های احتمالی:

فرمان پیشنهادی را تحلیل کن و بگو:

-چه چیزی را تغییر می‌دهد؛

-روی کدام مسیرها اثر دارد؛

-آیا برگشت‌پذیر است؛

-آیا به شبکه یا اطلاعات حساس نیاز دارد؛

-آیا برای این کار ضروری است.

اصل عملی:

ابتدا مشاهده، سپس پیش‌نمایش، بعد تأیید و در نهایت اجرا.

۶.۶. فایل‌های حساس و اطلاعات محرمانه

Claude Code ممکن است در جریان تحلیل پروژه با اطلاعات حساسی مواجه شود. وجود این اطلاعات در محیط توسعه به معنای مجازبودن استفاده یا ارسال آن‌ها نیست.

نمونه‌های رایج:

```
.env
.env.local
.env.production
```

```
id_rsa
*.pem
*.key
credentials.json
service-account.json
```

همچنین ممکن است اطلاعات حساس در فایل‌های عادی وجود داشته باشد:

- URLهای دارای token؛
- headerهای احراز هویت؛
- رمزهای عبور hard-codeشده؛
- داده‌های واقعی کاربران؛
- dump پایگاه داده؛
- لاگ‌های حاوی شماره تلفن یا شناسهٔ کاربر.

قواعد کار با secrets

- secrets را در prompt قرار ندهید؛
- مقدار واقعی کلیدها را نمایش ندهید؛
- پیش از اشتراک‌گذاری log، tokenها را حذف کنید؛
- فایل‌های محرمانه را در محدودهٔ تحلیل قرار ندهید؛
- از secret manager یا متغیر محیطی استفاده کنید؛
- در صورت افشای token، آن را revoke یا rotate کنید؛
- خروجی Claude Code را از نظر وجود اطلاعات حساس بررسی کنید.

درخواست مناسب:

فایل‌های تنظیمات پروژه را بررسی کن، اما مقدار واقعیsecret ها،

توکن‌ها، رمزها و کلیدها را نمایش نده.

فقط نام متغیرها و محل استفادهٔ آن‌ها را گزارش کن.

جایگزین‌کردن مقدار حساس

به‌جای این:

API_KEY=sk-actual-secret-value

از این استفاده کنید:

API_KEY=<redacted>

یا:

API_KEY=${API_KEY}

۶.۷. فایل‌های راهنمای پروژه و سیاست امنیتی

فایل راهنمای پروژه، مانند CLAUDE.md، می‌تواند قواعد امنیتی را برای نشست‌های آینده روشن کند.

نمونهٔ محتوای مناسب:

```
# Project Rules

- قبل از تغییر فایل‌های خارج از محدوده، تأیید بگیر.
- هرگز فایل‌های `.env` و کلیدهای خصوصی را نمایش نده.
- اجرا نکن `git push` و `git commit` بدون تأیید صریح.
- migrationها نیازمند بازبینی انسانی هستند.
- تست‌های محلی مجاز هستند.
- اتصال به سرویس production مجاز نیست.
- داده‌های واقعی کاربران نباید در تست استفاده شوند.
- پس از تغییر، `git diff` کن و تست‌های مرتبط را گزارش کن.
این فایل باید کوتاه، روشن و قابل‌اجرا باشد. قواعد متناقض یا بیش از حد کلی، کنترل واقعی ایجاد نمی‌کنند.
```

محدودیت مهم

فایل راهنما به‌تنهایی مرز امنیتی نیست. اگر محیط اجرا اجازهٔ دسترسی به یک secret یا اجرای فرمان خطرناک را بدهد، صرفاً نوشتن «این کار ممنوع است» در فایل راهنما کافی نیست.

کنترل واقعی باید در چند لایه انجام شود:

مجوزهای محیط اجرا

+

محدودیت ابزار

+

فایل راهنمای پروژه

+

تأیید انسانی

+

بازبینی خروجی

۶.۸. اجرای فرمان‌های پرخطر

برخی فرمان‌ها به‌دلیل دامنهٔ اثر یا برگشت‌پذیری کم، پرخطر محسوب می‌شوند.

حذف فایل یا پوشه

```
bash
rm file.txt
rm -rf directory/
```

قبل از اجرای چنین فرمانی باید مشخص باشد:

- مسیر دقیق چیست؛
- چه فایل‌هایی حذف می‌شوند؛
- نسخهٔ پشتیبان وجود دارد یا نه؛
- حذف قابل‌بازگشت است یا نه؛
- این حذف واقعاً لازم است یا روش کم‌خطرتری وجود دارد.

تغییر branch یا تاریخچهٔ Git

```
bash
git reset
git reset --hard
git rebase
git push --force
```

این فرمان‌ها ممکن است تغییرات موجود یا تاریخچهٔ مشترک را از بین ببرند. اجرای آن‌ها بدون بررسی دقیق وضعیت Git مجاز نیست.

نصب وابستگی

```
bash
npm install
pip install ...
flutter pub get
```

نصب وابستگی می‌تواند:

- فایل lock را تغییر دهد؛
- نسخه‌های دیگر را وارد پروژه کند؛
- اسکریپت نصب اجرا کند؛
- به شبکه متصل شود؛
- رفتار محیط را تغییر دهد.

قبل از نصب باید مشخص شود:

- چرا وابستگی لازم است؛

- نسخهٔ دقیق چیست؛
- آیا در فایل lock ثبت می‌شود؛
- آیا نصب به اجرای script نیاز دارد؛
- آیا منبع بسته قابل‌اعتماد است.

اسکریپت‌های پروژه

فرمان‌هایی مانند موارد زیر ممکن است فقط یک کار ساده انجام ندهند:

```
bash
make setup
npm run prepare
./scripts/deploy.sh
```

پیش از اجرا، محتوای اسکریپت و اثر واقعی آن را بررسی کنید:

```
bash
sed -n '1,220p' scripts/deploy.sh
```

یا:

این اسکریپت را بررسی کن و تمام عملیات آن،

دسترسی‌های موردنیاز و اثرات جانبی احتمالی را توضیح بده.

اجرا نکن.

۶.۹. کنترل دسترسی شبکه

اتصال شبکه همیشه بخشی از وظیفهٔ توسعه نیست. اجرای تست یا تحلیل کد معمولاً باید در محیطی با دسترسی شبکهٔ محدود انجام شود، مگر اینکه پروژه واقعاً به سرویس بیرونی نیاز داشته باشد.

ریسک‌های اتصال شبکه

- ارسال اطلاعات محرمانه؛
- دسترسی به سرویس‌های داخلی؛
- تغییر داده در API؛
- دانلود کد یا بستهٔ ناشناخته؛
- ایجاد هزینه؛
- تماس ناخواسته با محیط production؛
- ثبت داده در سرویس ثالث.

درخواست محدود

تست‌ها را فقط در محیط محلی اجرا کن.

از اتصال شبکه استفاده نکن.

بهAPI های بیرونی، productionو پایگاه دادهٔ واقعی دسترسی نداشته باش.

اگر اتصال شبکه ضروری است:

برای اجرای این تست فقط به endpoint زیر دسترسی داشته باش:

https://example.test/api/health

هیچ درخواستPOST ، PUT، PATCHیا DELETE ارسال نکن.

مقدار token را نمایش نده.

در پروژه‌های حساس، محدودسازی شبکه باید در سطح محیط اجرا یا firewall انجام شود، نه فقط در متن درخواست.

۶.۱۰. دادهٔ واقعی و محیط production

استفاده از دادهٔ واقعی کاربران برای آزمایش یا اشکال‌زدایی می‌تواند پیامدهای امنیتی و قانونی داشته باشد.

به‌جای دادهٔ واقعی از این موارد استفاده کنید:

- دادهٔ ساختگی؛
- fixture؛
- mock؛
- sandbox؛
- پایگاه دادهٔ anonymized؛
- محیط staging.

درخواست نامناسب:

به production وصل شو و آخرین کاربران را برای بررسی خطا استخراج کن.

درخواست کنترل‌شده:

ازfixture های محلی برای بازتولید خطا استفاده کن.

به production و دادهٔ واقعی کاربران دسترسی نداشته باش.

اگر خطا فقط در محیط واقعی قابل مشاهده است، باید:

- دامنهٔ داده محدود شود؛
- اطلاعات شخصی حذف یا ناشناس شود؛
- دسترسی ثبت و قابل‌ردیابی باشد؛
- عملیات فقط خواندنی باشد؛
- تأیید مسئول مربوط دریافت شود؛
- خروجی قبل از اشتراک‌گذاری پاک‌سازی شود.

۶.۱۱. مدیریت محدودهٔ تغییرات

یکی از مهم‌ترین کنترل‌های امنیتی، محدودکردن فایل‌هایی است که عامل اجازهٔ تغییر آن‌ها را دارد.

محدودهٔ صریح

فقط این مسیرها مجاز به تغییر هستند:

- src/payment/validator.py

- tests/payment/test_validator.py

تغییر درmigration ، تنظیماتCI ، فایل‌هایdependency ،

فایل‌های محیطی و سایر مسیرها ممنوع است.

بررسی پس از اجرا

```
 bash
git status --short
git diff --name-only
git diff --stat
```

اگر فایل خارج از محدوده تغییر کرده است:

1. اجرای کار را متوقف کنید؛
2. دلیل تغییر را بررسی کنید؛
3. diff را بازبینی کنید؛
4. دربارهٔ نگه‌داشتن یا برگرداندن آن تصمیم بگیرید؛
5. بدون بررسی، تغییر را commit نکنید.

هیچ‌گاه صرفاً به توضیح عامل اعتماد نکنید. فهرست واقعی فایل‌های تغییرکرده را از Git بخوانید.

۶.۱۲. بررسی خروجی عامل

امنیت فقط به فرمان‌هایی که Claude Code اجرا می‌کند مربوط نیست؛ خروجی تولیدشده نیز باید بررسی شود.

مواردی که باید کنترل شوند:

- افشای secret؛
- نمایش اطلاعات شخصی؛
- گزارش نادرست موفقیت؛
- ادعای اجرای تستی که واقعاً اجرا نشده؛
- حذف هشدارها از خروجی؛
- نادیده‌گرفتن خطاهای امنیتی؛
- پیشنهاد فرمان‌هایی با دامنهٔ بیش از نیاز.

درخواست مناسب:

در پایان، دقیقاً مشخص کن:

-کدام فایل‌ها تغییر کردند؛

-کدام فرمان‌ها اجرا شدند؛

-کدام تست‌ها اجرا شدند؛

-کدام تست‌ها اجرا نشدند؛

-چه خطاهایی باقی مانده‌اند؛

-آیا به شبکه یا اطلاعات حساس دسترسی استفاده شد.

اگر عامل می‌گوید تست‌ها موفق بوده‌اند، خروجی واقعی تست را بخواهید:

خلاصهٔ بالا کافی نیست.

فرمان دقیق تست و نتیجهٔ آن را گزارش کن.

اگر تست اجرا نشده، صریحاً بگو اجرا نشده است.

۶.۱۳. سناریوی کامل و ایمن

فرض کنید هدف، اصلاح اعتبارسنجی endpoint ورود است.

مرحلهٔ اول: تعریف محدوده

هدف: اصلاح اعتبارسنجی payload در endpoint ورود.

فایل‌های مجاز:

- src/auth/login.py
- tests/auth/test_login.py

فایل‌های خارج از محدوده:

migration -ها

-فایل‌های محیطی

-تنظیماتCI

-مدل کاربر

-سرویسproduction

تا زمان تأیید من هیچ فایلی را تغییر نده.

مرحلهٔ دوم: بررسی فقطخواندنی

فقطhandler ، تست‌های مرتبط و فایل راهنمای پروژه را بررسی کن.

مقدارsecret ها وtoken ها را نمایش نده.

فرمان تغییردهنده اجرا نکن.

مرحلهٔ سوم: برنامه‌ریزی

بر اساس بررسی انجام‌شده، مراحل تغییر،

ریسک‌ها، تست‌های لازم و معیار پذیرش را ارائه بده.

مرحلهٔ چهارم: تأیید محدود

برنامه تأیید شد.

فقط دو فایل اعلام‌شده را تغییر بده.

bitwithit

ایجاد وابستگی، اتصال شبکه، commitو push ممنوع است.

مرحلهٔ پنجم: بررسی تغییرات

```
bash
git status --short
git diff --name-only
git diff --stat
git diff
```

مرحلهٔ ششم: اجرای تست

```
bash
python -m pytest tests/auth/test_login.py
```

مرحلهٔ هفتم: بازبینی امنیتی

تغییرات را بررسی کن و مشخصاً این موارد را ارزیابی کن:

-دورزدن اعتبارسنجی

-افشای اطلاعات خطا

-ثبت رمز عبور یا token درlog

-تغییر خارج از محدوده

-ناکافی‌بودن تست‌های ورودی نامعتبر

فقط گزارش بده و کد را تغییر نده.

مرحلهٔ هشتم: تصمیم انسانی

پس از بررسی diff و خروجی تست، انسان دربارهٔ commit تصمیم می‌گیرد:

```
bash
git add src/auth/login.py tests/auth/test_login.py
git commit -m "Fix login payload validation"
```

Claude Code می‌تواند در آماده‌سازی commit کمک کند، اما ثبت commit باید بر اساس تأیید وضعیت واقعی پروژه انجام شود.

هشدارهای مهم

- دسترسی زیاد، کیفیت عامل را بهتر نمی‌کند؛ فقط دامنهٔ خطای احتمالی را افزایش می‌دهد.
- متن prompt جایگزین مجوزهای واقعی سیستم‌عامل نیست.
- فایل راهنمای پروژه مرز امنیتی قطعی محسوب نمی‌شود.
- فایل‌های .env، کلیدها و tokenها را در اختیار عامل قرار ندهید مگر با کنترل مشخص.

- خروجی‌های log را پیش از اشتراک‌گذاری از نظر اطلاعات حساس بررسی کنید.
- اجرای تست ممکن است به شبکه، فایل موقت یا سرویس خارجی دسترسی داشته باشد.
- فرمان‌های نصب وابستگی می‌توانند فایل‌های lock و تنظیمات پروژه را تغییر دهند.
- اسکریپت‌های پروژه را پیش از اجرا بخوانید.
- git reset --hard، git push --force و rm -rf را بدون بررسی کامل اجرا نکنید.
- اتصال به production باید استثنا باشد، نه پیش‌فرض.
- دادهٔ واقعی کاربران را با fixture یا دادهٔ ناشناس جایگزین کنید.
- «تغییر ایجاد نشد» را با git status و git diff بررسی کنید.
- ادعای اجرای تست را با فرمان و خروجی واقعی ارزیابی کنید.
- پیش از commit، push، merge، rebase یا تغییر migration تأیید صریح بگیرید.

چک‌لیست فصل ششم

- [ ] هدف و محدودهٔ کار را مشخص کرده‌ام.
- [ ] دسترسی عامل با نیاز واقعی کار متناسب است.
- [ ] مرحلهٔ شناخت را در حالت فقط‌خواندنی انجام داده‌ام.
- [ ] فایل‌های مجاز برای تغییر را مشخص کرده‌ام.
- [ ] فایل‌های خارج از محدوده را نیز تعیین کرده‌ام.
- [ ] فایل‌های .env، tokenها و کلیدهای خصوصی محافظت شده‌اند.
- [ ] دادهٔ واقعی کاربران در اختیار عامل قرار نگرفته است.
- [ ] اتصال شبکه فقط در صورت ضرورت فعال است.
- [ ] فرمان‌های پرخطر پیش از اجرا تحلیل شده‌اند.
- [ ] اسکریپت‌های ناشناخته را قبل از اجرا بررسی کرده‌ام.
- [ ] پیش از عملیات پایدار تأیید انسانی گرفته‌ام.
- [ ] git status و git diff را پس از تغییر بررسی کرده‌ام.
- [ ] فایل‌های تغییرکرده در محدودهٔ مجاز هستند.
- [ ] فرمان و خروجی واقعی تست‌ها مشخص است.
- [ ] commit، push، merge و rebase بدون تأیید انجام نشده‌اند.

جمع‌بندی

امنیت در Claude Code به یک گزینه یا فرمان واحد وابسته نیست. امنیت حاصل ترکیب چند تصمیم است:

حداقل دسترسی

+

محدودهٔ روشن

+

حالت فقطخواندنی در مرحلهٔ شناخت

+

محافظت از اطلاعات حساس

+

محدودسازی شبکه

+

تأیید انسانی

+

بررسی Git و خروجی تست

Claude Code باید مانند یک همکار فنی با اختیار محدود و قابل‌ردیابی استفاده شود. هرچه عملیات اثر گسترده‌تر، برگشت‌پذیری کمتر یا اطلاعات حساس‌تر باشد، نیاز به پیش‌نمایش، محدودسازی و تأیید انسانی بیشتر می‌شود.

bitwithit

فصل هفتم: Git و سه بررسی ضروری پیش از Commit

تغییر کد زمانی کامل محسوب نمی‌شود که فقط تست‌ها سبز شوند یا عامل اعلام کند کار انجام شده است. پیش از ثبت تغییرات در Git باید مشخص شود:

- آیا فایل درست تغییر کرده است؟
- آیا اطلاعات حساس وارد تغییرات نشده است؟
- آیا اندازه و دامنهٔ تغییر منطقی است؟
- آیا تست‌ها و بررسی‌های لازم اجرا شده‌اند؟
- آیا فقط فایل‌های مرتبط stage شده‌اند؟
- آیا commit واقعاً آمادهٔ ثبت است؟

این فصل بر بازبینی مستقل تغییرات پیش از commit تمرکز دارد. فرمان‌های Git در این فصل عمومی هستند، اما سیاست محدودکردن اختیار Claude Code به محیط عامل مربوط می‌شود.

متن منبع در بعضی بخش‌ها بین شماره‌گذاری فهرست و شماره‌گذاری متن اختلاف دارد. در این بازنویسی، ترتیب عملی مراحل یکدست شده است..

۷.۱. وضعیت Git پیش از شروع

پیش از درخواست هرگونه تغییر، وضعیت فعلی پروژه را بررسی کنید:

```
bash
git status
```

برای مشاهدهٔ خلاصهٔ تغییرات:

```
bash
git status --short
```

هدف این بررسی، جداکردن تغییرات قبلی از تغییراتی است که قرار است در جریان کار جدید ایجاد شوند.

پیش از ادامه باید مشخص شود:

- کدام فایل‌ها از قبل تغییر کرده‌اند؟
- کدام تغییرات مربوط به کار قبلی هستند؟
- کدام فایل‌ها باید بدون تغییر باقی بمانند؟
- آیا تغییرات فعلی باید commit یا stash شوند؟
- آیا کار جدید ممکن است با تغییرات قبلی مخلوط شود؟

نمونهٔ وضعیت مشکوک

```
M src/auth/login.py
```

M src/auth/service.py

M package-lock.json

M README.md

?? tmp/debug-output.log

در این مثال، فقط به این دلیل که چند فایل تغییر کرده‌اند نباید همهٔ آن‌ها را بخشی از کار جدید فرض کرد.

- src/auth/login.py ممکن است مستقیماً مرتبط باشد.
- src/auth/service.py باید بررسی شود.
- package-lock.json شاید بر اثر نصب وابستگی تغییر کرده باشد.
- README.md ممکن است تغییر نامرتبط داشته باشد.
- tmp/debug-output.log احتمالاً فایل موقت است، اما حذف آن نیز باید با احتیاط انجام شود.

فصل تأکید می‌کند که علت تغییر هر فایل باید مشخص شود و فایل‌های موقت نباید بدون بررسی حذف شوند؛ زیرا ممکن است اطلاعات مفیدی برای عیب‌یابی داشته باشند. این نکات در صفحهٔ ۵۰ آمده‌اند.

۷.۲. بررسی اول: آیا فایل درست تغییر کرده است؟

پس از اجرای کار، فهرست فایل‌های تغییرکرده را بررسی کنید:

```
bash
git status --short
```

برای مشاهدهٔ فقط نام فایل‌ها:

```
bash
git diff --name-only
```

برای مشاهدهٔ فایل‌های جدید، حذف‌شده یا تغییرکرده باید به نشانه‌های Git توجه کنید:

M file.py فایل تغییرکرده

A file.py فایل افزوده‌شده

D file.py فایل حذف‌شده

?? file.py فایل جدید و ثبت‌نشده

سپس بررسی کنید که هر فایل با هدف درخواست ارتباط دارد یا خیر.

پرسش‌های ضروری

- آیا فایل اصلی موردنظر تغییر کرده است؟
- آیا فایل نامرتبطی نیز تغییر کرده است؟

- آیا فایل تنظیمات، lock file یا مستندات بدون دلیل تغییر کرده است؟
- آیا فایل جدیدی ایجاد شده که در درخواست وجود نداشته است؟
- آیا فایلی حذف شده است؟
- آیا تغییرات قبلی با تغییرات جدید ترکیب شده‌اند؟

برای مشاهدهٔ جزئیات:

```
bash
git diff
```

نباید فقط به خلاصهٔ گزارش عامل اعتماد کرد. وضعیت واقعی باید از Git و diff خوانده شود.

۷.۳. بررسی دوم: آیا فایل حساس سالم است؟

پیش از commit باید diff و فایل‌های تغییرکرده از نظر اطلاعات حساس بررسی شوند.

نشانه‌های رایج اطلاعات حساس عبارت‌اند از:

- password
- secret
- token
- api_key
- private_key
- credential
- اطلاعات اتصال پایگاه داده
- دادهٔ شخصی کاربران

برای جست‌وجوی اولیه می‌توان از این فرمان استفاده کرد:

```
bash
rg -n "password|secret|token|api[_-]?key" .
```

این فرمان فقط یک ابزار کمکی است و ممکن است هشدارهای بی‌مورد یا false positive ایجاد کند. همچنین ممکن است secret به شکلی ذخیره شده باشد که با این الگو پیدا نشود.

بنابراین بررسی نهایی باید با خواندن diff انجام شود:

```
bash
git diff
```

مواردی که باید بررسی شوند

- آیا کلید API وارد کد شده است؟

- آیا token در log یا پیام خطا چاپ می‌شود؟
- آیا رمز عبور در تست hard-code شده است؟
- آیا فایل .env یا credential به staging اضافه شده است؟
- آیا دادهٔ واقعی کاربر در fixture یا نمونهٔ تست قرار گرفته است؟
- آیا URL دارای token در diff وجود دارد؟

اگر مقدار حساسی پیدا شد:

1. آن را در commit ثبت نکنید.
2. مقدار را از کد یا فایل مناسب حذف کنید.
3. در صورت افشای واقعی، token یا کلید را rotate یا revoke کنید.
4. تاریخچهٔ Git را نیز در صورت نیاز بررسی کنید.
5. فقط به حذف مقدار از نسخهٔ فعلی اکتفا نکنید؛ ممکن است در commit قبلی باقی مانده باشد.

۷.۴. بررسی سوم: آیا اندازه و دامنهٔ تغییر منطقی است؟

عنوان رسمی این بخش در فهرست کتابچه «آیا اندازهٔ تغییر منطقی است؟» است. هدف آن بررسی حجم و گسترهٔ واقعی تغییرات است.

برای مشاهدهٔ خلاصهٔ تغییرات:

```
bash
git diff --stat
```

نمونهٔ خروجی:

```
src/auth/login.py     | 12 ++++++------
tests/auth/test_login.py| 18 ++++++++++++++++
2 files changed, 21 insertions(+), 9 deletions(-)
```

اگر درخواست فقط اصلاح یک شرط ساده بوده، اما ده‌ها فایل و صدها خط تغییر کرده‌اند، باید کار متوقف و علت بررسی شود.

تغییرات خارج از هدف

بررسی کنید آیا عامل بخشی از کد را تغییر داده که در درخواست وجود نداشته است.

نمونه‌ها:

- تغییر فایل‌های unrelated؛
- اصلاح همزمان چند ماژول؛
- تغییر تنظیمات CI؛

- تغییر dependency بدون نیاز؛
- بازنویسی بخشی از API؛
- تغییر migration در جریان اصلاح یک تابع ساده.

قالب‌بندی گسترده

گاهی formatter یا ابزار مرتب‌سازی کل فایل را تغییر می‌دهد. این کار تشخیص تغییر واقعی را دشوار می‌کند.

اگر قالب‌بندی کامل فایل لازم نبوده است:

- تغییرات قالب‌بندی را از تغییر منطقی جدا کنید؛
- diff را به‌صورت دقیق بررسی کنید؛
- در صورت امکان، تغییرات غیرضروری را برگردانید؛
- از ترکیب اصلاح رفتاری و format گسترده در یک commit خودداری کنید.

خطوط حذف‌شده

خطوط حذف‌شده را با دقت بخوانید. حذف یک خط ظاهراً ساده ممکن است رفتار مهمی را از بین ببرد:

- import؛
- بررسی مجوز؛
- شرط اعتبارسنجی؛
- مدیریت خطا؛
- ثبت رخداد امنیتی؛
- سازگاری با نسخهٔ قدیمی API.

تغییر قراردادها

موارد زیر بخشی از قرارداد نرم‌افزار محسوب می‌شوند و تغییر آن‌ها باید آگاهانه باشد:

- متن خطا؛
- نام فیلد JSON؛
- کد HTTP؛
- امضای تابع؛
- نام پارامتر؛
- قالب خروجی؛

- رفتار endpoint.

این نکات در صفحهٔ ۵۱ کتابچه توضیح داده شده‌اند.

۷.۵. تست و اعتبارسنجی

پس از بررسی diff باید تست مناسب پروژه اجرا شود. فرمان تست باید از ساختار واقعی همان پروژه شناسایی شود.

نمونه‌های عمومی:

```
bash
npm test
flutter test
go test ./...
```

بررسی‌های تکمیلی نیز ممکن است شامل این موارد باشند:

```
bash
ruff check .
mypy src
npm run lint
npm run typecheck
```

این فرمان‌ها الزامی برای همهٔ پروژه‌ها نیستند. ابزار و فرمان صحیح باید از فایل‌های پروژه، مستندات و تنظیمات موجود شناسایی شود.

تست سبز به‌تنهایی کافی نیست

موفقیت تست‌ها ثابت نمی‌کند که:

- فایل درست تغییر کرده است؛
- همهٔ نیازمندی‌های کاربر پوشش داده شده‌اند؛
- رفتارهای مرزی بررسی شده‌اند؛
- امنیت حفظ شده است؛
- قرارداد API تغییر نکرده است؛
- فایل حساس وارد commit نشده است.

بنابراین نتیجهٔ تست باید در کنار این موارد تحلیل شود:

```
bash
git status
git diff
git diff --stat
```

نمونهٔ درخواست مناسب از Claude Code:

تست‌های مرتبط با این تغییر را شناسایی و اجرا کن.

فرمان دقیق، خروجی واقعی، تست‌های اجراشده،

تست‌های اجرانشده و خطاهای باقی‌مانده را گزارش بده.

هیچ commit یا push انجام نده.

محتوای مربوط به تست و اعتبارسنجی در صفحات ۵۱ و ۵۲ آمده است.

۷.۶. Staging ایمن

پس از بررسی working tree، فقط فایل‌های مرتبط را stage کنید:

```
bash
git add src/auth/login.py tests/auth/test_login.py
```

سپس محتوای staged را بررسی کنید:

```
bash
git diff --cached
```

این مرحله اهمیت زیادی دارد، زیرا git diff معمولاً تغییرات unstaged را نشان می‌دهد، در حالی که -- git diff cached محتوایی را نشان می‌دهد که برای commit آماده شده است.

خارج‌کردن فایل اشتباه از staging

اگر فایلی اشتباهاً stage شده است:

```
bash
git restore --staged path/to/file
```

این فرمان فایل را از staging خارج می‌کند، اما تغییر محلی آن را حذف نمی‌کند.

تفاوت فرمان‌ها:

```
bash
git restore --staged path/to/file
```

تغییر staging را اصلاح می‌کند و تغییر محلی را نگه می‌دارد.

```
bash
git restore path/to/file
```

ممکن است تغییر محلی فایل را بازگرداند و باید با احتیاط اجرا شود.

```
bash
git reset --hard
```

می‌تواند تغییرات ثبت‌نشده را حذف کند و عملیاتی پرریسک است.

توضیح staging و تفاوت این فرمان‌ها در صفحهٔ ۵۳ کتابچه آمده است.

۷.۷. نوشتن پیام Commit

پیام commit باید کوتاه، روشن و قابل‌فهم باشد. پیام باید نتیجه یا هدف تغییر را بیان کند، نه صرفاً فعالیت انجام‌شده را.

مناسب:

```
bash
git commit -m "Fix login validation"
```

نامناسب:

```
bash
git commit -m "Changes"
git commit -m "Update files"
git commit -m "Fix stuff"
```

پیام خوب باید به فردی که در آینده تاریخچهٔ پروژه را می‌خواند کمک کند بفهمد چه تغییری ثبت شده است.

پیش از commit باید این موارد مشخص باشند:

- هدف commit چیست؟
- کدام فایل‌ها در آن قرار دارند؟
- تست‌های مرتبط اجرا شده‌اند؟
- آیا diff staged با پیام commit سازگار است؟
- آیا تغییرات خارج از هدف در commit وجود دارد؟

۷.۸. محدودکردن عملیات Git در Claude Code

Claude Code می‌تواند در بررسی وضعیت، تحلیل diff، اجرای تست و آماده‌سازی تغییرات کمک کند؛ اما عملیات دارای اثر پایدار باید با تأیید انسانی انجام شوند.

بدون تأیید صریح، این عملیات را به عامل نسپارید:

```
git commit

git push

git merge

git rebase

git push --force

git reset

git clean
```

همچنین موارد زیر نیازمند بررسی و تأیید هستند:

- حذف branch؛
- حذف گستردهٔ فایل‌ها؛
- تغییر تاریخچهٔ Git؛
- تغییر migration؛
- stageکردن گستردهٔ فایل‌ها؛
- تغییر remote؛
- push به مخزن مشترک.

درخواست امن:

کد و تست‌ها را آماده کن، اماcommit ، push، merge

و تغییر تاریخچهٔ Git را انجام نده.

در پایان، فرمان‌های پیشنهادی را فقط به‌صورت متن ارائه کن.

اگر قرار است عامل commit انجام دهد، ابتدا این موارد را بررسی کنید:

- پیام commit؛
- فهرست فایل‌های staged؛
- git diff --cached؛
- نتیجهٔ تست؛
- دامنهٔ واقعی تغییرات؛
- وجود یا نبود اطلاعات حساس.

توصیه‌های مربوط به رفتار عامل در Git در صفحهٔ ۵۴ آمده‌اند.

۷.۹. بازگردانی تغییرات با احتیاط

اگر تغییری مناسب نیست، ابتدا وضعیت فعلی را ثبت و بررسی کنید:

```
bash
git status
git diff
```

اگر فایل stage شده است، ابتدا آن را از staging خارج کنید:

```
bash
git restore --staged path/to/file
```

فقط پس از اطمینان از غیرضروری‌بودن تغییر، آن را بازگردانید.

پیش از بازگردانی تغییر مهم، یکی از این اقدامات را انجام دهید:

- گرفتن نسخهٔ پشتیبان؛
- تهیهٔ patch؛
- ایجاد commit موقت؛
- نگه‌داشتن تغییر در branch جداگانه.

برای تهیهٔ patch:

```
bash
git diff > local-changes.patch
```

این کار امکان بررسی یا اعمال دوبارهٔ تغییرات را فراهم می‌کند.

هشدار دربارهٔ بازگردانی

دستورهای زیر اثر یکسانی ندارند:

```
bash
git restore --staged path/to/file
```

فقط staging را اصلاح می‌کند.

```
bash
git restore path/to/file
```

ممکن است تغییر محلی را حذف کند.

```
bash
git reset --hard
```

ممکن است مجموعه‌ای از تغییرات ثبت‌نشده را از بین ببرد.

به‌خصوص reset --hard و git clean نباید برای حل سریع یک وضعیت نامشخص استفاده شوند. ابتدا باید بدانید چه چیزی قرار است حذف شود و آیا نسخهٔ قابل‌بازگشتی از آن وجود دارد.

این بخش در صفحات ۵۴ و ۵۵ کتابچه آمده است.

۷.۱۰. گردش‌کار کامل پیش از Commit

گردش‌کار پیشنهادی این فصل:

1. از ریشهٔ پروژه git status را اجرا کنید.
2. تغییرات قبلی را شناسایی و دسته‌بندی کنید.
3. فایل‌های مجاز برای تغییر را مشخص کنید.
4. git diff را بررسی کنید.
5. فایل‌های حساس و secretها را جست‌وجو کنید.

6. اندازه و دامنهٔ تغییر را با هدف مقایسه کنید.
7. خطوط حذف‌شده و قراردادهای API را بررسی کنید.
8. تست، lint یا type-check مناسب را اجرا کنید.
9. دوباره git status و git diff را بررسی کنید.
10. فقط فایل‌های مرتبط را stage کنید.
11. git diff --cached را بخوانید.
12. پیام commit روشن بنویسید.
13. commit را فقط پس از تصمیم آگاهانه و تأیید انسانی انجام دهید.

خطاهای رایج

- شروع کار بدون اجرای git status؛
- مخلوط‌کردن تغییرات قبلی با تغییر جدید؛
- پذیرش همهٔ فایل‌های تغییرکرده بدون بررسی؛
- نادیده‌گرفتن فایل‌های موقت؛
- اعتماد کامل به گزارش عامل؛
- نخواندن diff؛
- نادیده‌گرفتن خطوط حذف‌شده؛
- پذیرش تغییرات گستردهٔ formatter؛
- تغییر ناخواستهٔ قرارداد API؛
- اعتماد کامل به سبزشدن تست‌ها؛
- stageکردن فایل اشتباه؛
- استفادهٔ بی‌احتیاط از git restore؛
- استفاده از git reset --hard برای حل وضعیت نامشخص؛
- اجازه‌دادن به commit یا push بدون تأیید؛
- بازگرداندن تغییر محلی بدون backup یا patch.

چک‌لیست نهایی پیش از Commit

- [ ] هدف commit مشخص است.

bitwithit

- [ ] شاخهٔ فعلی درست است.
- [ ] تغییرات قبلی از تغییرات جدید جدا شده‌اند.
- [ ] فقط فایل‌های مرتبط تغییر کرده‌اند.
- [ ] git diff به‌طور کامل خوانده شده است.
- [ ] فایل‌های حساس بررسی شده‌اند.
- [ ] secret یا credential در diff وجود ندارد.
- [ ] اندازهٔ تغییر با هدف سازگار است.
- [ ] خطوط حذف‌شده بررسی شده‌اند.
- [ ] تست مناسب پروژه اجرا شده است.
- [ ] نتیجهٔ تست در کنار status و diff تحلیل شده است.
- [ ] فایل‌های staged با هدف commit منطبق‌اند.
- [ ] git diff --cached بررسی شده است.
- [ ] پیام commit روشن و دقیق است.
- [ ] commit، push، merge و rebase بدون تأیید انجام نمی‌شوند.
- [ ] پیش از بازگردانی تغییرات مهم، نسخهٔ پشتیبان یا patch تهیه شده است.

جمع‌بندی

پیش از commit نباید فقط یک سؤال پرسید: «آیا تست‌ها سبز هستند؟»

بررسی کامل‌تر چنین است:

وضعیت اولیهٔGit

+

بررسی فایل‌های تغییرکرده

+

بررسی فایل‌های حساس

+

بررسی اندازه و دامنهٔ تغییر

+

تست و اعتبارسنجی

+

بازبینیstaging

+

پیام commit روشن

+

تأیید انسانی عملیاتGit

Claude Code می‌تواند در تحلیل وضعیت پروژه، خواندن diff، اجرای تست و آماده‌سازی staging کمک کند؛ اما تصمیم نهایی دربارهٔ commit، push، تغییر تاریخچه یا بازگردانی تغییرات باید پس از بررسی انسانی انجام شود.

قاعدهٔ اصلی فصل:

هیچ تغییری را فقط به دلیل موفقیت تست‌ها آمادهٔ commit فرض نکنید؛ فایل‌ها، diff، دامنهٔ تغییر، اطلاعات حساس و وضعیت staging را نیز مستقل بررسی کنید.

bitwitit

فصل هشتم: طراحی و مدیریت CLAUDE.md

فایل CLAUDE.md یکی از مهم‌ترین ابزارهای پایدارسازی رفتار Claude Code در یک پروژه است. این فایل به Claude Code توضیح می‌دهد که پروژه چگونه سازمان‌دهی شده، چه دستورهایی برای توسعه و تست معتبر هستند، چه محدودیت‌هایی باید رعایت شوند و چه کارهایی بدون تأیید کاربر ممنوع‌اند.

CLAUDE.md جایگزین مستندات کامل پروژه، کد یا README نیست. هدف آن این است که اطلاعاتی را در اختیار Claude Code قرار دهد که برای انجام درست کارها، بارها و بارها مورد نیاز هستند.

قاعدهٔ اصلی:

CLAUDE.md باید کوتاه، اجرایی، دقیق و متناسب با واقعیت پروژه باشد.

۸.۱. نقش CLAUDE.md در Claude Code

Claude Code هنگام کار روی پروژه باید پیش از تغییر کد، با ساختار و قواعد آن پروژه آشنا شود. فایل CLAUDE.md می‌تواند این شناخت اولیه را استاندارد کند.

این فایل معمولاً برای ثبت موارد زیر استفاده می‌شود:

- معرفی کوتاه پروژه؛
- فناوری‌ها و زبان‌های اصلی؛
- ساختار پوشه‌ها؛
- روش نصب وابستگی‌ها؛
- روش اجرای برنامه؛
- دستورهای تست؛
- دستورهای lint و type-check؛
- قواعد معماری؛
- محدودیت‌های تغییر کد؛
- قواعد امنیتی؛
- روش مدیریت migration؛
- سیاست مربوط به commit و push؛
- فرمان‌هایی که نیازمند تأیید انسانی هستند؛
- روش گزارش‌دادن نتیجهٔ کار.

برای مثال، اگر تست‌های پروژه با دستور زیر اجرا می‌شوند:

```
bash
pytest
```

این دستور باید در CLAUDE.md ثبت شود، نه اینکه Claude Code مجبور باشد هر بار از روی چند فایل مختلف آن را حدس بزند.

اطلاعات داخل این فایل باید با پروژهٔ واقعی سازگار باشند. ثبت دستورهای حدسی یا قدیمی باعث می‌شود Claude Codeمسیر نادرستی را دنبال کند.

۸.۲ .چه اطلاعاتی در `CLAUDE.md` قرار می‌گیرد؟

معرفی پروژه

در چند خط توضیح دهید پروژه چه کاری انجام می‌دهد و بخش‌های اصلی آن چیست.این بخش نباید به یک متن تبلیغاتی یا مستندات طولانی تبدیل شود. هدف، ایجاد زمینهٔ سریع برای Claude Code است.

فناوری‌های اصلی

فناوری‌هایی را ثبت کنید که انتخاب روش پیاده‌سازی را تحت تأثیر قرار می‌دهند:

```
Stack

- Backend: Python, FastAPI, SQLAlchemy
- Mobile: Flutter and Dart
- Tests: pytest and Flutter test
- Formatting: Ruff and Dart formatter
```

ساختار پوشه‌ها

فقط پوشه‌های مهم و کاربرد آن‌ها را توضیح دهید:

```
Repository Layout

- `backend/`: FastAPI application
- `backend/app/api/`: HTTP routes
```

- `backend/app/services/`: business logic
- `backend/tests/`: backend tests
- `mobile/`: Flutter application
- `docs/`: project documentation

ثبت مسیرهای دقیق باعث می‌شود Claude Code برای پیدا کردن محل مناسب تغییر، به حدس‌های نادرست تکیه نکند.

دستور نصب

دستورهایی را بنویسید که واقعاً برای آماده‌سازی پروژه استفاده می‌شوند:

Setup

Create a virtual environment and install dependencies:
```bash
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Do not commit `.env` or local credential files.

اگر نصب پروژه به چند مرحله نیاز دارد، مراحل را به ترتیب بنویسید.

اجرای برنامه

روش اجرای سرویس یا برنامه باید واضح باشد:

Run Locally

Start the backend with:
```bash
uvicorn app.main:app --reload
```

Start the mobile application with:

bash
flutter run

در صورتی که اجرای پروژه به متغیرهای محیطی نیاز دارد، نام متغیرها را می‌توان توضیح داد، اما مقدار واقعی آن‌ها نباید در فایل نوشته شود.

مناسب:

The backend requires `DATABASE_URL` and `SECRET_KEY`

to be provided through environment variables.

نامناسب:

SECRET_KEY=actual-secret-value

تست

دستورهای تست باید دقیق و قابل اجرا باشند:

Tests

Run backend tests:
```bash
pytest

Run a specific test file:

bash
pytest backend/tests/test_auth.py

Run mobile tests:

bash
flutter test
```

اگر بخشی از تست‌ها نیازمند سرویس خارجی، پایگاه داده یا تنظیم خاصی هستند، آن را نیز توضیح دهید:

Integration tests require a running PostgreSQL instance.

Unit tests do not require external services.

Lintو Type-check

این بررسی‌ها را از تست جدا و روشن ثبت کنید:

Quality Checks

Run Python lint:
```bash
ruff check .
```

Run type checking:

```bash
mypy backend/app
```

Format Python files with:

```bash
ruff format .
```

نباید فرمانی را فقط به دلیل رایج‌بودن آن در پروژه ثبت کنید. برای مثال، اگر پروژه از `black` استفاده نمی‌کند، نباید آن را در `CLAUDE.md` بنویسید.

۸.۳ .قواعد معماری و محدودیت‌های تغییر

Claude Codeممکن است بتواند چند روش مختلف برای انجام یک کار پیدا کند. قواعد معماری کمک می‌کنند راه‌حل انتخاب‌شده با طراحی پروژه سازگار باشد.

نمونه:

Architecture Rules

- Keep HTTP route handlers thin.

- Put business logic in `services/`.

- Access the database through the repository layer.

- Do not call external services directly from route handlers.

- Reuse existing dependency-injection utilities.

- Do not introduce a new framework for a local problem.

این قواعد باید رفتار مورد انتظار را روشن کنند، نه اینکه با جزئیات غیرضروری تمام طراحی سیستم را تکرار کنند.

محدودیت‌های مهم

موارد حساس یا پرریسک را صریح بنویسید:

Change Constraints

- Do not change public API response fields without explicit approval.

- Do not rename database columns casually.

- Do not edit generated files manually.

- Do not update dependencies unless required by the task.

- Do not modify CI configuration unless the task explicitly includes it.

- Preserve backward compatibility for existing API clients.

فایل‌های تولیدشده

اگر بخشی از پروژه با ابزار دیگری تولید می‌شود، Claude Code باید بداند که آن فایل‌ها محل تغییر مستقیم نیستند:

Generated files must not be edited manually.

Update the source configuration and run the generator instead.

این قاعده از تغییراتی جلوگیری می‌کند که در اجرای بعدی ابزار تولیدکننده از بین می‌روند.

۸.۴. قواعد امنیتی در CLAUDE.md

قواعد امنیتی باید کوتاه، مستقیم و بدون ابهام باشند.

نمونه:

Security Rules

- Never print, expose, or commit secrets.
- Do not read production credentials.
- Do not call production services during local development.
- Use redacted values in examples and logs.
- Ask for approval before changing authentication or authorization logic.
- Do not disable security checks to make tests pass.

Secrets

نباید مقادیر زیر در CLAUDE.md یا کد ثبت شوند:

- رمز عبور؛
- API key؛
- access token؛
- private key؛
- connection string واقعی؛
- credential سرویس‌های ابری؛
- اطلاعات شخصی کاربران.

در صورت نیاز، فقط نام متغیر را ثبت کنید:

Required environment variables:

- `DATABASE_URL`
- `JWT_SECRET`
- `SPEECH_SERVICE_API_KEY`

bitwiht

محیط تولید

اگر پروژه به محیط production دسترسی دارد، این محدودیت باید کاملاً شفاف باشد:

Never run commands against production without explicit user approval.

Do not use production data for local testing.

عملیات مخرب

برای جلوگیری از اجرای ناخواستهٔ فرمان‌های خطرناک:

```
The following operations require explicit approval:

- deleting files;
- dropping database tables;
- running destructive migrations;
- force-pushing Git history;
- changing production configuration;
- rotating or revoking credentials.
```

CLAUDE.md جایگزین مجوزهای سیستم‌عامل، تنظیمات sandbox یا سیاست‌های دسترسی نیست؛ اما می‌تواند قواعد رفتاری پروژه را برای Claude Code روشن کند.

۸.۵. کنترل عملیات Git

در فایل راهنما باید مشخص شود Claude Code تا چه حد مجاز به کار با Git است.

نمونهٔ محافظه‌کارانه:

```
Git Workflow

Before making changes, inspect:
bash
git status

Before proposing a commit, inspect:

bash
git diff
```

```
git diff --cached

Do not run `git push`, `git reset --hard`, `git clean`,
`git rebase`, or force-push without explicit approval.

Do not create a commit unless the user asks for it.
```

اگر سیاست پروژه اجازهٔ commit محلی را می‌دهد، باید همان سیاست دقیقاً نوشته شود:

```
Local commits are allowed after tests pass and the staged diff

has been reviewed. Push operations always require approval.
```

قواعد کلی Git باید با سیاست واقعی تیم هماهنگ باشند. نوشتن جمله‌هایی مانند «همیشه commit کن» یا «هر تغییری را push کن» برای بیشتر پروژه‌ها تصمیم مناسبی نیست.

۸.۶. روش گزارش نتیجه

یکی از کاربردهای مهم CLAUDE.md مشخص‌کردن قالب گزارش نهایی است.

نمونه:

```
Reporting

At the end of each task, report:

1. Files changed
2. Behavior changed
3. Tests and checks executed
4. Test results
5. Known limitations
6. Suggested follow-up work
```

Do not claim that a check passed unless it was actually run.

این قانون از گزارش‌های مبهم جلوگیری می‌کند.

گزارش مناسب:

```
Changed:
```

```
- backend/app/services/auth.py
- backend/tests/test_auth.py

Checks:
- pytest backend/tests/test_auth.py: passed
- ruff check backend/app/services/auth.py: passed

Not run:
- full integration suite, because PostgreSQL was unavailable

Remaining limitation:
- No production-like authentication flow was tested
```

گزارش نامناسب:

Everything is fixed and tested.

Claude Code نباید تست یا فرمانی را که اجرا نشده، موفق اعلام کند.

۸.۷. محل قرارگیری فایل‌های راهنما

در یک پروژه ممکن است بیش از یک فایل راهنما وجود داشته باشد. فایل راهنمای اصلی معمولاً در ریشهٔ مخزن قرار می‌گیرد:

```
project/
 ├── CLAUDE.md
 ├── backend/
 ├── mobile/
 └── docs/
```

این فایل برای قواعد مشترک کل پروژه مناسب است.

در پروژه‌های چندبخشی می‌توان برای یک زیرپروژه نیز راهنمای جداگانه داشت:

```
project/
 ├── CLAUDE.md
 ├── backend/
 │    └── CLAUDE.md
 └── mobile/
      └── CLAUDE.md
```

راهنمای ریشه می‌تواند قواعد عمومی را مشخص کند و راهنمای داخل backend/ یا mobile/ قواعد تخصصی همان بخش را توضیح دهد.

قواعد سطحی و محلی

نمونهٔ تقسیم مسئولیت:

CLAUDE.md در ریشه:

```
- Never commit secrets.
- Run tests before reporting completion.
- Do not push without approval.
backend/CLAUDE.md:

- Use FastAPI dependency injection.
- Put business logic in `services/`.
- Run backend tests with `pytest backend/tests`.
mobile/CLAUDE.md:
- Follow Flutter feature-based structure.
- Run `flutter analyze` after Dart changes.
- Do not edit generated localization files manually.
```

فایل‌های محلی نباید قوانین عمومی را بی‌دلیل تکرار کنند. بهتر است هر فایل فقط اطلاعاتی را نگه دارد که در همان محدوده معتبر است.

۸.۸. ایجاد یا به‌روزرسانی CLAUDE.md

در Claude Code می‌توان ابتدا ساختار پروژه را بررسی کرد و سپس فایل راهنما را ایجاد یا اصلاح کرد.

درخواست مناسب:

ساختار پروژه و فایل‌های پیکربندی را بررسی کن.

دستورهای واقعی نصب، اجرا، تست، lint و type-check را پیدا کن.

سپس یک پیش‌نویس کوتاه برای CLAUDE.md پیشنهاد بده.

فعلاً هیچ فایل دیگری را تغییر نده و مقدار secret ها را نمایش نده.

پس از بررسی پیش‌نویس:

پیش‌نویس CLAUDE.md را با ساختار پروژه تطبیق بده.

فقط اطلاعاتی را نگه دار که قابل تأیید و به‌روز هستند.

دستورهای حدسی را حذف کن و قواعد امنیتی و محدودیت‌های Git را اضافه کن.

هنگام ویرایش فایل، Claude Code باید از این موارد پرهیز کند:

- بازنویسی کامل فایل بدون بررسی محتوای قبلی؛

- حذف قواعد معتبر؛
- افزودن دستورهای آزمایش‌نشده؛
- ثبت secretها؛
- اضافه‌کردن قوانین متناقض؛
- تبدیل فایل راهنما به مستندات چندصدخطی.

۸.۹. نمونهٔ کامل CLAUDE.md

نمونهٔ زیر یک الگوی عمومی است و باید با پروژهٔ واقعی تطبیق داده شود:

````
# Project Guide

## Overview

This repository contains a FastAPI backend and a Flutter client.

## Repository Layout

- `backend/app/api/`: API routes
- `backend/app/services/`: business logic
- `backend/app/models/`: database models
- `backend/tests/`: backend tests
- `mobile/`: Flutter application
- `docs/`: documentation

## Setup

Backend:
```bash
python -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt

Mobile:

bash
flutter pub get

## Run
````
````

Backend:

bash
uvicorn app.main:app --reload

Mobile:

bash
flutter run

Verification

Backend tests:

bash
pytest backend/tests

Python lint:

bash
ruff check backend

Mobile checks:

bash
flutter analyze
flutter test

Architecture Rules

- Keep route handlers thin.
- Put business logic in services.
- Reuse existing repository and dependency-injection patterns.
- Do not edit generated files manually.
- Avoid introducing new dependencies unless necessary.

Security

- Never expose or commit secrets.
- Do not access production systems without explicit approval.
- Use redacted values in logs and examples.
- Do not disable authentication or authorization checks.

```
## Git

- Inspect `git status` before starting.
- Review `git diff` before reporting completion.
- Do not push, rebase, force-push, or reset hard without approval.
- Do not create a commit unless explicitly requested.

## Reporting

Report changed files, checks run, results, and remaining limitations.
Do not claim a test passed unless it was actually executed.
```

این نمونه نقطهٔ شروع است، نه فایلی که باید بدون تغییر در هر پروژه‌ای قرار گیرد.

۸.۱۰ .خطاهای رایج

طولانی‌کردن بیش از حد فایل

اگر تمام جزئیاتREADME ، معماری، قرارداد API و راهنمای توسعه در `CLAUDE.md` تکرار شود، اطلاعات مهم در میان متن گم می‌شوند.

راه‌حل:

-اطلاعات پرتکرار و اجرایی را نگه دارید؛

-جزئیات گسترده را در مستندات تخصصی قرار دهید؛

-به فایل‌ها و بخش‌های مرتبط لینک یا مسیر بدهید.

ثبت دستورهای قدیمی

تغییر ابزار یا ساختار پروژه بدون به‌روزرسانی `CLAUDE.md` باعث می‌شود Claude Code فرمان‌های اشتباه اجرا کند.

برای مثال، اگر تست‌ها از `pytest` به ابزار دیگری منتقل شده‌اند، دستور قبلی نباید باقی بماند.

ثبت فرمان‌های حدسی

اینکه یک فرمان در پروژه‌ای دیگر رایج است، دلیل مناسبی برای درج آن در فایل راهنمای پروژه نیست.

فرمان باید از یکی از این منابع تأیید شود:

- `pyproject.toml`;
- `package.json`;
- `Makefile`;

-فایلCI ؛

-مستندات پروژه؛

-اسکریپت‌های موجود؛

-اجرای واقعی در محیط پروژه.

قوانین مبهم

قوانینی مانند موارد زیر ارزش عملی کمی دارند:

- Write good code.
- Follow best practices.
- Be careful.

bitwith.it

- Test everything.

نسخهٔ دقیق‌تر:

- Run `pytest backend/tests` after backend changes.

- Do not modify migrations unless the task requires it.

- Do not expose secrets in logs or test fixtures.

قوانین متناقض

برای مثال، این دو قانون با هم سازگار نیستند:

Always create a commit after every task.

و:

Never create commits without user approval.

قواعد باید اولویت مشخص داشته باشند و در صورت وجود تعارض، سیاست ایمن‌تر و محدودکننده‌تر ملاک قرار گیرد.

تکرار README

CLAUDE.md باید به Claude Code کمک عملی کند، نه اینکه نسخهٔ دوم README باشد. توضیح کامل محصول، تاریخچهٔ پروژه و آموزش عمومی کاربران معمولاً در این فایل ضروری نیست.

ثبت اطلاعات حساس

قرار دادن token یا رمز در فایل راهنما حتی اگر مخزن خصوصی باشد، تصمیم خطرناکی است. فایل‌های راهنما ممکن است در commit، pull request، backup یا ابزارهای دیگر دیده شوند.

تغییر بی‌دلیل فایل

وجود CLAUDE.md به این معنا نیست که در هر کار باید آن را تغییر داد. فقط زمانی آن را به‌روزرسانی کنید که:

- دستورهای پروژه تغییر کرده باشند؛
- ساختار پوشه‌ها تغییر کرده باشد؛
- قانون معماری جدیدی اضافه شده باشد؛
- سیاست امنیتی یا Git تغییر کرده باشد؛
- یک خطای تکرارشونده نیازمند ثبت قاعدهٔ پایدار باشد.

۸.۱۱. بایدها و نبایدها

بایدها

- دستورهای واقعی و قابل اجرا بنویسید.
- مسیر فایل‌های مهم را دقیق مشخص کنید.
- قواعد را کوتاه و اولویت‌بندی‌شده نگه دارید.
- تصمیم‌های معماری مهم را ثبت کنید.
- دستورهای تست و lint را به‌روز نگه دارید.
- قواعد امنیتی را صریح بنویسید.
- محدودیت‌های Git و عملیات حساس را مشخص کنید.
- روش گزارش نتیجه را تعیین کنید.
- از مثال‌های واقعی پروژه استفاده کنید.
- اطلاعات مشترک را در راهنمای ریشه و قواعد تخصصی را در راهنمای محلی قرار دهید.

نبایدها

- secret، token، رمز یا کلید خصوصی را در فایل ننویسید.
- دستورهای آزمایش‌نشده را ثبت نکنید.
- README را بی‌دلیل تکرار نکنید.
- قوانین متناقض ایجاد نکنید.
- فایل را بیش از حد طولانی و مبهم نکنید.
- دستورهای قدیمی را باقی نگذارید.
- برای هر تغییر کوچک، راهنمای پروژه را بی‌دلیل ویرایش نکنید.
- ادعا نکنید فرمانی موفق شده است، مگر اینکه واقعاً اجرا شده باشد.

چک‌لیست نهایی CLAUDE.md

- [ ] هدف فایل برای Claude Code روشن است.
- [ ] معرفی پروژه کوتاه و دقیق است.
- [ ] ساختار مهم پوشه‌ها ثبت شده است.
- [ ] دستورهای نصب واقعی و قابل اجرا هستند.
- [ ] دستور اجرای برنامه مشخص است.

bitwitit

- [ ] دستورهای تست به‌روز هستند.
- [ ] lint و type-check در صورت نیاز مشخص شده‌اند.
- [ ] قواعد معماری ثبت شده‌اند.
- [ ] محدودیت‌های تغییر واضح هستند.
- [ ] قواعد امنیتی صریح هستند.
- [ ] عملیات حساس Git نیازمند تأیید انسانی‌اند.
- [ ] هیچ secret یا credentialی در فایل وجود ندارد.
- [ ] قوانین با README، CI و ساختار فعلی پروژه تناقض ندارند.
- [ ] متن بیش از حد طولانی نیست.
- [ ] دستورهای ثبت‌شده دست‌کم یک‌بار بررسی یا اجرا شده‌اند.
- [ ] روش گزارش نتیجه مشخص است.

جمع‌بندی

CLAUDE.md یک فایل پیکربندی رفتاری و عملیاتی برای Claude Code است. ارزش آن به طول فایل یا تعداد قوانین بستگی ندارد؛ ارزش واقعی آن در دقت و به‌روز بودن اطلاعات است.

یک فایل خوب باید به Claude Code پاسخ دهد:

- پروژه چیست؟
- کد مهم در کجا قرار دارد؟
- برنامه چگونه اجرا می‌شود؟
- تست‌ها چگونه اجرا می‌شوند؟
- چه معماری‌ای باید حفظ شود؟
- چه تغییراتی ممنوع یا حساس‌اند؟
- چه فرمان‌هایی نیازمند تأیید هستند؟
- نتیجهٔ کار چگونه باید گزارش شود؟

فرمول سادهٔ یک CLAUDE.md مناسب:

اطلاعات واقعی

+

bitwitit

دستورهای قابل اجرا

+

قواعد کوتاه و روشن

+

محدودیت‌های امنیتی

+

تأیید انسانی برای عملیات حساس

قاعدهٔ نهایی:

CLAUDE.md را به‌عنوان قرارداد کاری پروژه بنویسید؛ کوتاه، دقیق، قابل بررسی و همیشه هماهنگ با وضعیت واقعی مخزن.

bitwithit

فصل نهم: قابلیت‌های پیشرفته در Claude Code

Claude Code را می‌توان با دستورالعمل‌های تخصصی، عامل‌های فرعی و اتصال به ابزارها و منابع بیرونی برای گردش‌کارهای پیچیده‌تر به کار گرفت. این قابلیت‌ها برای هر کار کوچکی لازم نیستند؛ ابتدا باید مسئله را شناخت و ساده‌ترین راه‌حل مؤثر را انتخاب کرد.

در این فصل، قابلیت‌های پیشرفته در چهار گروه بررسی می‌شوند:

- مهارت‌ها و دستورالعمل‌های قابل‌استفادهٔ مجدد؛
- عامل‌های فرعی برای کارهای قابل‌تفکیک؛
- افزونه‌ها و قابلیت‌های افزودنی؛
- اتصال به ابزارها و منابع خارجی از طریق MCP.

نام، دسترس‌پذیری و روش پیکربندی این قابلیت‌ها ممکن است به نسخه و تنظیمات Claude Code وابسته باشد. پیش از اتکا به هر قابلیت، پشتیبانی آن را در محیط پروژه بررسی کنید و از ثبت فرمان‌های حدسی خودداری کنید.

۹.۱. مهارت‌ها و دستورالعمل‌های تخصصی

مهارت یا دستورالعمل تخصصی، مجموعه‌ای از قواعد و مراحل برای انجام یک نوع کار تکرارشونده است. هدف آن این است که روش انجام کارهای مشابه، هر بار از ابتدا توضیح داده نشود.

نمونه‌هایی از کارهایی که می‌توان برایشان دستورالعمل تخصصی تعریف کرد:

- بازبینی امنیتی API؛
- تولید تست برای کد Python؛
- بررسی کد Flutter؛
- تحلیل migration پایگاه داده؛
- تهیهٔ مستندات endpointها؛
- تولید گزارش استاندارد انتشار نسخه؛
- بررسی دسترس‌پذیری رابط کاربری؛
- تحلیل تصاویر پزشکی بر اساس پروتکل مشخص پروژه.

دستورالعمل تخصصی زمانی مفید است که کار:

- به‌طور مکرر انجام می‌شود؛
- مراحل یا معیارهای پذیرش روشنی دارد؛
- به دانش ویژهٔ پروژه نیاز دارد؛
- خروجی آن را می‌توان با معیارهای مشخص ارزیابی کرد.

این دستورالعمل‌ها جایگزین کد یا تست نیستند. آن‌ها روش کار را روشن می‌کنند؛ نتیجه همچنان باید با شواهد پروژه اعتبارسنجی شود.

نمونهٔ دستورالعمل برای بازبینی API

API Review

- Check authentication and authorization.
- Validate request and response schemas.
- Look for unsafe handling of user input.
- Check error responses and logging.
- Do not change the public API contract without approval.
- Report findings with file paths and supporting evidence.

این نمونه یک الگوی متنی است، نه فرمانی برای فعال‌کردن قابلیت خاصی در Claude Code. آن را فقط در جایگاهی قرار دهید که پروژه و نسخهٔ مورد استفاده واقعاً پشتیبانی می‌کنند.

۹.۲. ارائهٔ تدریجی اطلاعات

هنگام کار روی یک پروژه، ارائهٔ همهٔ مستندات و قواعد به Claude Code از ابتدا معمولاً لازم نیست. اطلاعات باید به اندازهٔ نیاز کار جاری در اختیار آن قرار بگیرد.

برای رعایت این اصل:

1. ابتدا هدف، محدوده و قواعد اصلی را مشخص کنید.
2. فقط فایل‌ها و مستندات مرتبط با همان کار را بررسی کنید.
3. جزئیات تخصصی را زمانی ارائه دهید که مسئله به آن‌ها نیاز دارد.
4. از بارگذاری یا تکرار مستندات نامرتبط خودداری کنید.

این روش کمک می‌کند اطلاعات مهم میان متن‌های فرعی گم نشوند و تحلیل روی بخش‌های مرتبط متمرکز بماند.

۹.۳. عامل‌های فرعی

عامل فرعی می‌تواند برای بررسی یا تحلیل بخشی مستقل از کار به کار گرفته شود، در صورتی که محیط Claude Code مورد استفاده چنین قابلیتی را پشتیبانی کند.

نمونه‌های مناسب برای تقسیم کار:

- بررسی جداگانهٔ فایل‌های backend و گزارش ریسک‌ها؛
- تحلیل تست‌های شکست‌خورده بدون دست‌زدن به کد؛

- بررسی معماری یک زیرسامانه؛
- پیشنهاد راه‌حل جایگزین برای یک باگ؛
- تحلیل مستقل یک تغییر پیش از تصمیم‌گیری دربارهٔ پیاده‌سازی.

عامل فرعی باید وظیفه‌ای محدود و خروجی مشخص داشته باشد. برای مثال، به‌جای «کل پروژه را بررسی کن» می‌توان گفت:

فقط فایل‌های بخش احراز هویت را از نظر کنترل دسترسی بررسی کن.

هیچ فایلی را تغییر نده.

یافته‌ها را با مسیر فایل و شواهد مرتبط گزارش بده.

چه زمانی تقسیم کار مناسب نیست؟

تقسیم کار ممکن است نتیجه را بدتر کند اگر:

- وظیفه کوچک و مستقیم باشد؛
- بخش‌های کار وابستگی زیادی به یکدیگر داشته باشند؛
- چند عامل هم‌زمان فایل‌های مشترک را تغییر دهند؛
- محدودهٔ مسئولیت‌ها روشن نباشد؛
- نتیجهٔ هر بخش بدون زمینهٔ بخش‌های دیگر قابل ارزیابی نباشد.

افزایش تعداد عامل‌ها به‌تنهایی باعث افزایش کیفیت نمی‌شود. تقسیم کار باید زمان یا ریسک بررسی را به شکل معناداری کاهش دهد.

مسئولیت بررسی نهایی

خروجی عامل فرعی را به‌عنوان نتیجهٔ قطعی تلقی نکنید. عامل اصلی یا کاربر باید آن را با کد، تست‌ها و معیارهای پروژه تطبیق دهد. هیچ تغییر حساس یا تصمیم معماری نباید صرفاً به این دلیل پذیرفته شود که یک عامل فرعی آن را پیشنهاد کرده است.

۹.۴. افزونه‌ها و قابلیت‌های افزودنی

برخی محیط‌ها یا گردش‌کارها ممکن است از افزونه‌ها یا قابلیت‌های افزودنی پشتیبانی کنند. این موارد می‌توانند ابزارها، فرمان‌ها یا اتصال‌های بیشتری فراهم کنند. روش نصب و دسترس‌پذیری آن‌ها را باید بر اساس محیط واقعی Claude Code بررسی کرد.

پیش از فعال‌کردن هر افزونه یا ابزار افزودنی، این موارد را ارزیابی کنید:

- منبع آن معتبر و قابل‌شناسایی است؟
- به چه فایل‌هایی دسترسی دارد؟
- آیا می‌تواند فرمان اجرا کند؟

- آیا به شبکه یا سرویس بیرونی وصل می‌شود؟
- چه داده‌هایی را دریافت یا ارسال می‌کند؟
- چه وابستگی‌هایی نصب می‌کند؟
- آیا تغییرات و نسخه‌های آن قابل‌ردیابی‌اند؟
- چگونه می‌توان آن را غیرفعال یا حذف کرد؟

افزونهٔ ناشناخته را صرفاً به دلیل نام جذاب، محبوبیت یا توصیهٔ غیرقابل‌بررسی فعال نکنید.

۹.۵. اتصال ابزارها و منابع خارجی با MCP

MCP می‌تواند راهی برای اتصال Claude Code به ابزارها یا منابع خارجی باشد، مشروط به اینکه در محیط مورد استفاده پشتیبانی و به‌درستی پیکربندی شده باشد.

این اتصال ممکن است دسترسی به مواردی مانند این‌ها فراهم کند:

- پایگاه داده؛
- فایل‌ها یا مستندات؛
- سامانهٔ مدیریت issue؛
- مخزن کد؛
- سرویس‌های داخلی؛
- ابزارهای تحلیل یا پایش.

اتصال به منبع خارجی توانایی بیشتری فراهم می‌کند، اما سطح ریسک را نیز بالا می‌برد. ابزار متصل ممکن است بتواند داده بخواند، تغییر دهد یا به سرویس دیگری ارسال کند. بنابراین، دسترسی آن باید با هدف کار متناسب باشد.

خطرهای اتصال خارجی

- افشای داده‌های داخلی یا اطلاعات شخصی؛
- دسترسی بیش از حد به فایل‌ها یا سامانه‌ها؛
- اجرای عملیات تغییردهنده بدون تأیید؛
- ارسال داده به سرویس ثالث؛
- تغییر ناخواسته در سامانهٔ واقعی؛
- اعتماد به داده‌ای که ناقص، قدیمی یا نامعتبر است.

اصول استفادهٔ امن

1. فقط اتصال‌های شناخته‌شده و ضروری را فعال کنید.

2. دسترسی را به حداقل منابع موردنیاز محدود کنید.
3. دسترسی خواندن را از دسترسی نوشتن جدا نگه دارید.
4. برای عملیات تغییردهنده، تأیید انسانی لازم بدانید.
5. داده‌های حساس و غیرضروری را در اختیار ابزار قرار ندهید.
6. گزارش‌ها و مسیرهای دسترسی را بررسی کنید.
7. محیط آزمایشی را از محیط واقعی جدا نگه دارید.
8. اتصال‌هایی را که دیگر لازم نیستند، غیرفعال کنید.
9. پیش از اجرای عملیات، محدودهٔ اثر آن را بررسی کنید.
10. هیچ داده‌ای را صرفاً به دلیل در دسترس بودن آن به ابزار متصل نسپارید.

۹.۶. انتخاب روش مناسب

نیاز	روش مناسب
کار ساده و یک‌باره	درخواست مستقیم و دقیق
قواعد مشترک پروژه	فایل راهنمای پروژه، مانند CLAUDE.md
کار تخصصی و تکرارشونده	دستورالعمل تخصصی قابل‌استفادهٔ مجدد
تحلیل مستقل یک بخش	عامل فرعی، در صورت پشتیبانی و ضرورت
افزودن قابلیت به محیط	افزونهٔ معتبر و بررسی‌شده، در صورت پشتیبانی
دسترسی به ابزار یا منبع بیرونی	اتصال کنترل‌شده، مانند MCP، در صورت پشتیبانی
اجرای تست یا فرمان محلی	CLI تأییدشدهٔ پروژه
بررسی تغییرات کد	Git و بازبینی diff

قاعدهٔ اصلی این است که برای حل مسئله از ساده‌ترین روش کافی استفاده شود. قابلیت پیشرفته‌تر فقط وقتی توجیه دارد که نیاز مشخصی را برطرف کند.

۹.۷. گردش‌کار پیشنهادی

مرحلهٔ اول: صورت‌بندی مسئله

پیش از انتخاب ابزار، روشن کنید:

- خروجی مورد انتظار چیست؟

- چه فایل‌ها یا منابعی مرتبط‌اند؟
- آیا کار تکرارشونده است؟
- آیا بخش‌های مستقل دارد؟
- آیا به اطلاعات یا سرویس بیرونی نیاز دارد؟
- آیا عملیاتی وجود دارد که نیازمند تأیید باشد؟

مرحلهٔ دوم: انتخاب کم‌پیچیدگی‌ترین راه‌حل

ابتدا درخواست مستقیم یا قواعد موجود پروژه را بررسی کنید. اگر کافی نبود، به‌ترتیب امکان استفاده از دستورالعمل تخصصی، تقسیم کار یا اتصال خارجی را بسنجید.

درخواست مستقیم

→قواعد موجود پروژه

→دستورالعمل تخصصی

→تقسیم کار با عامل فرعی

→اتصال یا افزونهٔ خارجی

این ترتیب قانون ثابت محصول نیست؛ راهنمایی برای جلوگیری از پیچیده‌کردن بی‌دلیل کار است.

مرحلهٔ سوم: تعیین دسترسی و محدوده

پیش از شروع مشخص کنید:

- چه فایل‌هایی قابل بررسی یا تغییرند؛
- ابزار فقط اجازهٔ خواندن دارد یا می‌تواند بنویسد؛
- کدام عملیات نیازمند تأیید است؛
- چه داده‌هایی نباید در اختیار ابزار قرار گیرند؛
- نتیجه با چه شواهدی ارزیابی می‌شود.

مرحلهٔ چهارم: اعتبارسنجی نتیجه

نتیجه را با شواهد واقعی پروژه بررسی کنید:

- فایل‌های تغییرکرده؛
- تست‌ها و بررسی‌های اجراشده؛
- git diff؛
- گزارش‌ها و مستندات مرتبط؛

- وضعیت محیط یا سرویس، در صورت لزوم.

گزارش ابزار یا عامل فرعی به‌تنهایی اثبات نمی‌کند که تغییر درست است.

۹.۸. خطاهای رایج

به‌کارگیری قابلیت پیچیده برای کار ساده

ساخت یک گردش‌کار یا اتصال جدید برای کاری که با یک درخواست روشن انجام می‌شود، هزینه و پیچیدگی غیرضروری ایجاد می‌کند.

اعتماد کامل به خروجی عامل فرعی

عامل فرعی ممکن است زمینهٔ کامل را نداشته باشد یا بر اساس فرض نادرست نتیجه‌گیری کند. یافته‌هایش باید با شواهد بررسی شوند.

اعطای دسترسی بیش از نیاز

اتصال ابزار به کل مخزن، پایگاه دادهٔ واقعی یا محیط تولید، وقتی فقط یک فایل یا دادهٔ آزمایشی لازم است، سطح ریسک را بی‌دلیل بالا می‌برد.

استفاده از افزونهٔ ناشناخته

فعال‌کردن افزونه بدون بررسی منبع، دسترسی‌ها، وابستگی‌ها و رفتار شبکه می‌تواند داده‌ها و محیط پروژه را در معرض خطر قرار دهد.

ارائهٔ همهٔ مستندات از ابتدا

انبوهی از اطلاعات نامرتبط تمرکز کار را کاهش می‌دهد. فقط زمینهٔ لازم برای وظیفهٔ جاری را ارائه کنید.

پذیرفتن نتیجه بدون اعتبارسنجی

پیشنهاد یا گزارش ابزار را با تست، کد و diff بررسی کنید. ادعای موفقیت یک ابزار جایگزین شواهد نیست.

تعارض میان دستورها

اگر درخواست جاری با قواعد فایل راهنمای پروژه یا محدودیت‌های امنیتی تعارض دارد، تعارض را روشن کنید و پیش از انجام تغییر حساس تأیید بگیرید. دستورالعمل‌های تخصصی نیز نباید بدون بررسی، قواعد امنیتی یا محدودیت‌های صریح کاربر را کنار بزنند.

چک‌لیست استفاده از قابلیت‌های پیشرفته

- [ ] آیا این کار واقعاً به قابلیت پیشرفته نیاز دارد؟
- [ ] آیا درخواست مستقیم یا قواعد موجود کافی است؟
- [ ] آیا دستورالعمل تخصصی برای کاری تکرارشونده وجود دارد یا لازم است؟
- [ ] اگر کار تقسیم شده، مرز و خروجی هر بخش روشن است؟
- [ ] آیا افزونه یا اتصال از منبع قابل‌اعتماد می‌آید؟

- [ ] دسترسی فایل، شبکه و فرمان‌های اجرایی بررسی شده است؟
- [ ] فقط منابع ضروری در دسترس‌اند؟
- [ ] دسترسی خواندن و نوشتن متناسب و محدود است؟
- [ ] برای عملیات تغییردهنده تأیید انسانی در نظر گرفته شده است؟
- [ ] داده‌های حساس از دسترس غیرضروری خارج‌اند؟
- [ ] محیط آزمایشی از محیط واقعی جداست؟
- [ ] اتصال‌های بلااستفاده غیرفعال شده‌اند؟
- [ ] خروجی با کد، تست، diff یا مستندات اعتبارسنجی شده است؟
- [ ] نتیجهٔ نهایی، محدودیت‌ها و بررسی‌های انجام‌نشده صریح گزارش شده‌اند؟

جمع‌بندی

قابلیت‌های پیشرفته زمانی ارزشمندند که مسئله واقعاً به آن‌ها نیاز داشته باشد. دستورالعمل‌های تخصصی برای کارهای تکرارشونده، عامل‌های فرعی برای تحلیل‌های محدود و مستقل، و اتصال‌های خارجی برای دسترسی کنترل‌شده به ابزارها و منابع کاربرد دارند.

در همهٔ موارد، سه اصل باید حفظ شوند:

- ساده‌ترین راه‌حل کافی را انتخاب کنید؛
- دسترسی‌ها را به حداقل لازم محدود کنید؛
- خروجی را با شواهد واقعی پروژه بررسی کنید.

bitwiht

فصل دهم: انتخاب مدل و سطح استدلال در Claude Code

انتخاب مدل و میزان تلاش استدلالی باید بر اساس ماهیت کار انجام شود، نه صرفاً بر اساس نام مدل یا بالاترین سطح توانایی آن. مدل قدرتمندتر همیشه انتخاب بهتری نیست. برای یک تغییر کوچک و کم‌ریسک، استفاده از مدل سبک‌تر می‌تواند سریع‌تر و کم‌هزینه‌تر باشد؛ اما برای تغییرات معماری، تحلیل امنیتی یا خطاهای چندلایه، استفاده از مدل توانمندتر توجیه بیشتری دارد.

در این فصل، معیارهای انتخاب مدل، مفهوم سطح استدلال، روش افزایش تدریجی عمق تحلیل و خطاهای رایج بررسی می‌شوند.

۱۰.۱. انتخاب مدل بر اساس نوع کار

مدل مناسب را باید با توجه به چهار عامل انتخاب کرد:

- پیچیدگی کار؛
- میزان ریسک خطا؛
- مقدار عدم‌قطعیت؛
- قابلیت بازگشت تغییرات.

کارهای سبک و کم‌ریسک

برای کارهای زیر معمولاً مدل سبک‌تر یا سریع‌تر کافی است:

- قالب‌بندی کد؛
- اصلاح یک خط یا عبارت؛
- تغییر نام یک متغیر؛
- تولید متن ساده؛
- اصلاح یک خطای نگارشی؛
- تبدیل محدود داده؛
- توضیح کوتاه دربارهٔ یک تابع؛
- اجرای یک تغییر کاملاً مشخص و قابل بازگشت.

در چنین کارهایی، استفاده از مدل قدرتمندتر معمولاً ارزش متناسبی ایجاد نمی‌کند.

کارهای متعادل

برای کارهای زیر به مدلی با توانایی تحلیلی متوسط نیاز است:

- تغییر چند تابع مرتبط؛
- افزودن یا اصلاح تست‌ها؛

- رفع یک خطای معمول؛
- بازبینی یک ماژول؛
- اصلاح یک endpoint؛
- تحلیل وابستگی محدود؛
- به‌روزرسانی مستندات فنی همراه با بررسی کد.

در این حالت، مدل باید بتواند روابط میان چند بخش را بررسی کند، اما مسئله هنوز محدوده‌ای روشن دارد.

کارهای پیچیده و پرریسک

برای کارهای زیر از مدل توانمندتر و تحلیل عمیق‌تر استفاده کنید:

- تغییر معماری؛
- اصلاح قرارداد عمومی API؛
- مهاجرت وابستگی‌ها؛
- تغییر schema یا migration پایگاه داده؛
- تحلیل آسیب‌پذیری امنیتی؛
- بررسی خطای چندفایلی؛
- تحلیل رفتارهای مرزی؛
- تغییر در سامانه‌های دارای دادهٔ واقعی؛
- بررسی وابستگی میان چند زیرسامانه؛
- تصمیم‌گیری دربارهٔ سازگاری نسخه‌ها.

در این نوع کارها، سرعت پاسخ مهم است، اما صحت تحلیل و کاهش احتمال آسیب اهمیت بیشتری دارد.

نکتهٔ مهم دربارهٔ نام مدل‌ها

نام مدل‌ها، سطح دسترسی و قابلیت‌های آن‌ها ممکن است در نسخه‌های مختلف Claude Code یا محیط‌های متفاوت تغییر کند. بنابراین، از نسبت‌دادن نام‌ها یا قابلیت‌های ثابت به مدل‌ها بدون بررسی محیط واقعی خودداری کنید.

معیار نهایی باید رفتار واقعی مدل در پروژه باشد:

- آیا مسئله را درست فهمیده است؟
- آیا محدودیت‌ها را رعایت کرده است؟
- آیا تغییرات پیشنهادی با معماری پروژه سازگارند؟
- آیا خروجی آن با تست و شواهد قابل تأیید است؟

- آیا در کارهای مشابه، عملکرد باثباتی دارد؟

۱۰.۲. مفهوم سطح تلاش استدلالی

سطح تلاش استدلالی نشان می‌دهد مدل برای بررسی مسئله، مقایسهٔ گزینه‌ها و رسیدن به نتیجه چه میزان تحلیل انجام دهد. این مفهوم را نباید با «هوش مطلق مدل» یکی دانست. یک مدل می‌تواند برای کار ساده بیش از حد تحلیل کند، بدون آن‌که کیفیت نهایی به شکل معناداری افزایش یابد.

سطح‌های زیر یک چارچوب مفهومی برای انتخاب عمق مناسب تحلیل هستند. بسته به نسخه و محیط Claude Code، ممکن است تنظیمات واقعی با این نام‌ها وجود نداشته باشند.

سطح سبک

مناسب برای:

- پاسخ‌های کوتاه؛
- توضیح ساده؛
- تغییر یک خط یا تابع؛
- قالب‌بندی؛
- تولید متن کم‌ریسک؛
- اصلاح‌های کاملاً مشخص؛
- بررسی اولیهٔ یک فایل.

در این سطح، مسئله باید روشن، محدود و قابل بازگشت باشد.

سطح متوسط

مناسب برای:

- تغییر چند تابع؛
- افزودن تست؛
- رفع خطای معمول؛
- بازبینی یک ماژول؛
- تحلیل وابستگی محدود؛
- اصلاح چند فایل مرتبط؛
- بررسی سناریوهای اصلی و خطاهای متداول.

در این سطح، مدل باید علاوه بر اجرای درخواست، اثر تغییر بر بخش‌های مرتبط را نیز بررسی کند.

سطح سنگین

bitwithit

مناسب برای:

- باگ‌های چندفایلی؛
- تغییر API؛
- مهاجرت وابستگی؛
- بررسی امنیت؛
- تغییر معماری؛
- تحلیل رفتارهای مرزی؛
- تغییراتی که ممکن است روی چند سرویس اثر بگذارند؛
- کارهایی که نیازمند مقایسهٔ چند راه‌حل هستند.

در این سطح، پیش از اجرای تغییر باید فرض‌ها، وابستگی‌ها، خطرها و روش اعتبارسنجی مشخص شوند.

سطح بسیار سنگین

مناسب برای:

- مسائل بسیار پیچیده؛
- تصمیم‌های معماری پرریسک؛
- تحلیل جامع یک سامانه؛
- تعارض‌های چندلایه میان نیازمندی‌ها؛
- تغییرات حساس در سامانه‌های حیاتی؛
- کارهایی که نیازمند بررسی چندمرحله‌ای، مقایسهٔ گزینه‌ها و ارزیابی پیامدهای بلندمدت هستند.

از این سطح برای کارهای روزمره استفاده نکنید. تحلیل بیش از حد می‌تواند زمان اجرا و مصرف منابع را افزایش دهد، بدون آن‌که ارزش واقعی متناسبی ایجاد کند.

۱۰.۳. سه معیار اصلی برای انتخاب

پیش از انتخاب مدل یا سطح تحلیل، سه سؤال اصلی مطرح کنید.

۱. ریسک کار چقدر است؟

بررسی کنید خطا چه پیامدی دارد:

- آیا فقط یک فایل محلی آسیب می‌بیند؟
- آیا ممکن است یک API از کار بیفتد؟
- آیا دادهٔ واقعی تغییر می‌کند؟

- آیا امنیت یا حریم خصوصی تحت‌تأثیر قرار می‌گیرد؟
- آیا تغییر روی معماری یا چند سرویس اثر دارد؟
- آیا خطا برای کاربران نهایی قابل مشاهده است؟

هرچه پیامد خطا بیشتر باشد، تحلیل عمیق‌تر و کنترل انسانی بیشتری لازم است.

۲. مسئله چقدر روشن است؟

اگر دقیقاً می‌دانید چه فایلی، چه تابعی و با چه نتیجه‌ای باید تغییر کند، کار عدم‌قطعیت کمی دارد.

اما اگر هنوز مشخص نیست:

- علت خطا چیست؛
- کدام فایل مسئول رفتار است؛
- کدام راه‌حل با معماری پروژه سازگارتر است؛
- تغییر چه اثرات جانبی دارد؛
- معیار پذیرش نتیجه چیست؛

ابتدا مرحلهٔ شناخت و برنامه‌ریزی را انجام دهید. افزایش سطح استدلال بدون رفع ابهام، الزاماً نتیجهٔ بهتری ایجاد نمی‌کند.

۳. تغییر تا چه اندازه قابل بازگشت است؟

قابلیت بازگشت تغییر را بررسی کنید:

- آیا می‌توان با یک commit تغییر را برگرداند؟
- آیا داده‌ای تغییر می‌کند؟
- آیا migration اجرا می‌شود؟
- آیا قرارداد عمومی API تغییر می‌کند؟
- آیا وابستگی‌های پروژه به‌روزرسانی می‌شوند؟
- آیا عملیات روی محیط واقعی انجام می‌شود؟

تغییر متن مستندات معمولاً به‌راحتی قابل بازگشت است، اما تغییر schema، migration، دادهٔ واقعی یا قرارداد API ممکن است بازگردانی دشوارتری داشته باشد.

۱۰.۴. روش سه‌مرحله‌ای انتخاب

برای انتخاب مدل و سطح تحلیل، این روند را دنبال کنید.

مرحلهٔ اول: محدودهٔ کار را مشخص کنید

ابتدا روشن کنید:

- هدف دقیق چیست؟
- کدام فایل‌ها مرتبط‌اند؟
- چه فایل‌هایی نباید تغییر کنند؟
- خروجی مطلوب چگونه ارزیابی می‌شود؟
- آیا تست یا مستندات موجود است؟

مرحلهٔ دوم: ریسک و عدم‌قطعیت را بسنجید

کار را بر اساس دو محور بررسی کنید:

وضعیت کار	رویکرد پیشنهادی
ریسک کم و مسئله روشن	مدل سبک‌تر و تحلیل کوتاه
ریسک کم و مسئله مبهم	ابتدا شناخت و برنامه‌ریزی
ریسک متوسط و مسئله روشن	تحلیل متوسط و اجرای کنترل‌شده
ریسک متوسط و مسئله مبهم	بررسی وابستگی‌ها و تهیهٔ طرح
ریسک زیاد و مسئله روشن	مدل توانمندتر، تست و بازبینی دقیق
ریسک زیاد و مسئله مبهم	تحلیل عمیق، تأیید انسانی و اجرای مرحله‌ای

مرحلهٔ سوم: روش اعتبارسنجی را از قبل تعیین کنید

پیش از شروع تغییر، مشخص کنید چگونه موفقیت بررسی می‌شود:

- اجرای تست‌های واحد؛
- اجرای تست‌های یکپارچه؛
- بررسی lint یا type check؛
- مشاهدهٔ git diff؛
- مقایسهٔ رفتار قبل و بعد؛
- بررسی لاگ‌ها؛
- اجرای آزمایشی در محیط جداگانه؛
- بازبینی انسانی تغییرات حساس.

مدلی که نمی‌داند نتیجه چگونه سنجیده می‌شود، ممکن است تغییری ظاهراً درست اما غیرقابل‌اعتماد تولید کند.

۱۰.۵. افزایش تدریجی عمق استدلال

همیشه از بالاترین سطح تحلیل شروع نکنید. رویکرد مناسب، افزایش تدریجی عمق است:

تحلیل سبک

↓اگر کافی نبود

تحلیل متوسط

↓اگر کافی نبود

تحلیل سنگین

↓فقط در صورت نیاز واقعی

تحلیل بسیار سنگین

پس از هر مرحله، نتیجه را بررسی کنید و فقط در صورت وجود دلیل مشخص به مرحلهٔ بعد بروید.

پیش از افزایش سطح، این موارد را بررسی کنید

گاهی مشکل از مدل یا سطح استدلال نیست، بلکه از کیفیت ورودی است:

- آیا درخواست مبهم است؟
- آیا هدف و معیار پذیرش روشن نیست؟
- آیا فایل‌های ضروری در اختیار Claude Code قرار نگرفته‌اند؟
- آیا ساختار پروژه به‌اندازهٔ کافی توضیح داده نشده است؟
- آیا تست یا شواهد لازم وجود ندارد؟
- آیا محدودیت‌های امنیتی و معماری مشخص نشده‌اند؟
- آیا دستورهای پروژه در فایل راهنمای مناسب ثبت نشده‌اند؟
- آیا مدل بر اساس فرض نادرست کار می‌کند؟
- آیا مسئله به چند وظیفهٔ مستقل تقسیم نشده است؟

در بسیاری از موارد، اصلاح درخواست، افزودن فایل مرتبط یا مشخص‌کردن محدودیت‌ها بیش از افزایش سطح استدلال به بهبود نتیجه کمک می‌کند.

۱۰.۶. اصلاح درخواست پیش از افزایش توان مدل

اگر خروجی مطلوب نیست، این ترتیب را امتحان کنید:

1. هدف را دقیق‌تر بنویسید.

2. محدودهٔ فایل‌های قابل تغییر را مشخص کنید.
3. محدودیت‌های معماری و امنیتی را اضافه کنید.
4. معیار پذیرش را تعریف کنید.
5. نمونهٔ ورودی و خروجی ارائه دهید.
6. تست‌ها یا خطاهای واقعی را در اختیار مدل قرار دهید.
7. از Claude Code بخواهید ابتدا برنامه و فرض‌های خود را گزارش کند.
8. سپس، در صورت باقی‌ماندن پیچیدگی، سطح تحلیل را افزایش دهید.

برای مثال، درخواست زیر مبهم است:

احراز هویت را اصلاح کن.

نسخهٔ دقیق‌تر:

علت خطای 401 در endpoint ورود را بررسی کن.

فقط فایل‌های بخش احراز هویت را تغییر بده.

قرارداد API را تغییر نده.

ابتدا علت احتمالی و فایل‌های مرتبط را گزارش کن.

پس از تأیید، اصلاح را انجام بده و تست‌های موجود را اجرا کن.

نسخهٔ دوم اطلاعاتی فراهم می‌کند که مدل برای تحلیل صحیح به آن نیاز دارد و احتمال تغییرات غیرضروری را کاهش می‌دهد.

۱۰.۷. ترکیب انتخاب مدل و سطح استدلال

انتخاب مدل و انتخاب سطح تحلیل دو تصمیم جدا اما مرتبط‌اند. مدل سبک‌تر با تحلیل عمیق‌تر همیشه جایگزین مدل توانمندتر نیست؛ همان‌طور که مدل قدرتمند با تحلیل بیش از حد نیز برای هر کاری لازم نیست.

نمونه‌های کلی:

نوع کار	مدل پیشنهادی	سطح تحلیل
اصلاح قالب‌بندی	سبک‌تر	سبک

نوع کار	مدل پیشنهادی	سطح تحلیل
تغییر یک تابع همراه با تست	سبک یا متوسط	سبک تا متوسط
رفع باگ معمول در یک ماژول	متوسط	متوسط
تغییر چند endpoint مرتبط	متوسط یا توانمندتر	متوسط تا سنگین
مهاجرت وابستگی	توانمندتر	سنگین
تغییر schema یا migration	توانمندتر	سنگین
بررسی امنیتی چندبخشی	توانمندتر	سنگین تا بسیار سنگین
تصمیم معماری پرریسک	توانمندتر	بسیار سنگین

این جدول جایگزین بررسی واقعی پروژه نیست. اندازهٔ کد، کیفیت تست‌ها، تجربهٔ تیم و حساسیت محیط اجرا نیز باید در تصمیم نهایی لحاظ شوند.

۱۰.۸. خطاهای رایج

استفادهٔ همیشگی از قوی‌ترین مدل

این کار ممکن است زمان و هزینهٔ پردازش را افزایش دهد و برای کارهای ساده ارزش اضافی ایجاد نکند.

استفادهٔ همیشگی از سطح بسیار سنگین

تحلیل طولانی برای تغییر کوچک، سرعت گردش‌کار را کاهش می‌دهد و ممکن است باعث پیچیده‌شدن راه‌حل شود.

افزایش سطح به‌جای اصلاح درخواست

اگر مسئله مبهم باشد، افزایش Reasoning معمولاً ابهام را از بین نمی‌برد. ابتدا باید هدف، محدوده و معیار پذیرش روشن شوند.

نادیده‌گرفتن قابلیت بازگشت

تغییری که روی migration، دادهٔ واقعی یا API عمومی اثر می‌گذارد نباید مانند اصلاح یک فایل متنی ساده مدیریت شود.

اعتماد به نام مدل

نام مدل به‌تنهایی کیفیت خروجی را تضمین نمی‌کند. عملکرد مدل باید در محیط و پروژهٔ واقعی بررسی شود.

نادیده‌گرفتن کمبود زمینه

اگر فایل‌ها، تست‌ها، قراردادها یا قواعد معماری در اختیار Claude Code نباشند، انتخاب مدل قدرتمندتر مشکل کمبود اطلاعات را حل نمی‌کند.

چک‌لیست انتخاب مدل و سطح استدلال

- [ ] هدف کار دقیق و قابل‌اندازه‌گیری است.
- [ ] فایل‌های مرتبط و محدودهٔ تغییر مشخص شده‌اند.
- [ ] ریسک خطا ارزیابی شده است.
- [ ] اثر احتمالی بر داده، API، امنیت و معماری بررسی شده است.
- [ ] میزان عدم‌قطعیت مسئله مشخص است.
- [ ] قابلیت بازگشت تغییر ارزیابی شده است.
- [ ] مدل متناسب با پیچیدگی و ریسک انتخاب شده است.
- [ ] سطح تحلیل بیش از نیاز کار انتخاب نشده است.
- [ ] درخواست، محدودیت‌ها و معیار پذیرش را روشن می‌کند.
- [ ] تست یا شواهد لازم در اختیار Claude Code قرار دارد.
- [ ] قبل از افزایش سطح، کیفیت prompt و زمینه بررسی شده است.
- [ ] برای کارهای حساس، تأیید انسانی در نظر گرفته شده است.
- [ ] نتیجه با تست، git diff و شواهد واقعی اعتبارسنجی می‌شود.
- [ ] محدودیت‌ها و بررسی‌های انجام‌نشده در گزارش نهایی ذکر می‌شوند.

جمع‌بندی

انتخاب مدل و سطح استدلال باید متناسب با ریسک، عدم‌قطعیت و قابلیت بازگشت تغییر باشد. مدل قوی‌تر همیشه لازم نیست و تحلیل عمیق‌تر نیز همیشه به نتیجهٔ بهتر منجر نمی‌شود.

روش پیشنهادی این است:

1. ابتدا مسئله و محدوده را روشن کنید.
2. ریسک، عدم‌قطعیت و قابلیت بازگشت را بسنجید.
3. از ساده‌ترین مدل و سطح تحلیل کافی شروع کنید.
4. در صورت نیاز، عمق تحلیل را مرحله‌به‌مرحله افزایش دهید.
5. پیش از افزایش سطح، prompt، فایل‌های مرجع، تست‌ها و محدودیت‌ها را بررسی کنید.
6. نتیجه را با شواهد واقعی پروژه اعتبارسنجی کنید.

به‌طور خلاصه، کیفیت نتیجه فقط به قدرت مدل وابسته نیست؛ کیفیت مسئله‌سازی، زمینهٔ ارائه‌شده، کنترل دسترسی و روش اعتبارسنجی نیز نقش تعیین‌کننده دارند.

bitwithit